

ivdtools handbook

hiox-tech

2026-08-09

Table of contents

Welcome	1
Reader's Guide	1
Version and Citation	1
1 ivdtools Package Overview	3
1.1 Package Positioning	3
1.2 Design Features	3
1.2.1 Standalone Workflows	3
1.2.2 Progressive Workflows	3
1.3 Functional Modules	4
1.4 Quick Start	4
1.4.1 Method Comparison	4
1.4.2 ROC	5
1.4.3 Qualitative Agreement	5
1.4.4 Reference Interval	6
1.4.5 Bottle/Lot ANOVA	6
1.4.6 Standalone Tools	6
1.4.7 Getting Help	6
1.5 AI Automated Analysis	7
1.6 Dependencies	7
1.7 Scope of Use	7
1.8 Project Information	7
2 Installing and Using the Analysis Environment	9
2.1 Purpose of This Document	9
2.2 Installing R	9
2.2.1 Windows and macOS	9
2.2.2 Debian/Ubuntu	9
2.2.3 Fedora	9
2.3 Choosing an Editor	9
2.4 Installing R Packages	10
2.5 Creating a New Project	10
2.6 Importing and Validating Data	11
2.6.1 CSV	11
2.6.2 Excel	12
2.7 Running Code	12
2.8 Exporting Results	13
2.8.1 Tables	13
2.8.2 Figures	13
2.9 Rendering and Saving	13
3 4PLC Equation Calibration Curve Fitting	15
3.1 Analysis Objectives	15
3.2 4PLC Model and Main Functions	15
3.3 Example 1: Basic Fitting Workflow	16
3.3.1 Reading and Validating Data	16

3.3.2	Fitting and Checking Convergence	17
3.3.3	Residuals and Graphics	18
3.3.4	Predicting Response from Concentration	19
3.3.5	Estimating Concentration from Response (Inverse Prediction)	20
3.4	Example 2: Sandwich Assay and Weights	20
3.4.1	Equal-Weight Fit	20
3.4.2	Inverse Response-Squared Weight Fit	21
3.5	Example 3: Competitive Assay	23
3.6	Key Points for Interpreting Results	24
4	Stability Studies	27
4.1	Analysis Objectives	27
4.2	Overview of Functions	27
4.3	Example 1: Establishing an Analysis Plan	28
4.4	Example 2: Bias Analysis	28
4.4.1	Reading and Validating Raw Data	28
4.4.2	Node Bias Computation	29
4.5	Example 3: Regression Analysis Comparing Storage Conditions	31
4.5.1	Reading and Validating Raw Data	31
4.5.2	Comparing Test and Reference Conditions	31
4.6	Example 4: Regression Analysis Relative to the First Node	32
4.6.1	Single-Condition Analysis Relative to the First Node	32
4.7	Example 5: Predicting the Time to Reach a Limit	34
4.8	Example 6: Arrhenius Accelerated Stability Analysis	35
4.8.1	Reading and Validating Raw Data	35
4.8.2	Kinetic and Arrhenius Fitting	36
4.9	Example 7: MKT Computation	37
4.9.1	Reading and Validating Raw Temperature Data	37
4.9.2	Time-Weighted MKT	38
4.10	Splitting Multi-Sample Data	38
4.11	Key Points for Interpreting Results	39
5	Precision Analysis	41
5.1	Analysis Objectives	41
5.2	Overview of Functions	41
5.3	Example 1: EP05 20×2×2 Single-Sample Precision	42
5.3.1	Reading and Validating Data	42
5.3.2	Creating the Analysis Object and Pre-Check	42
5.3.3	Variance Components and Confidence Intervals	43
5.3.4	Converting to an EP05 Terminology Table	44
5.3.5	Graphics	44
5.4	Example 2: Multi-Sample 3×5 and Sadler Profile	46
5.4.1	Reading Data	46
5.4.2	Per-Sample Variance Components	46
5.4.3	Precision Profile	48
5.5	Example 3: Multi-Sample Multi-Site (Site) Precision	49
5.5.1	Reading Data	49
5.5.2	Multi-Column Grouping	50
5.5.3	Splitting by Site for Separate Analysis	50
5.6	Example 4: Custom More Complex Models	50
5.6.1	Reading Data	50
5.6.2	Fitting and Interpretation	51
5.7	Key Points for Interpreting Results	51
6	Linearity Analysis	53
6.1	Analysis Objectives	53
6.2	Overview of Functions	53
6.3	Example 1: Linearity Bias of a Serial Dilution	53
6.3.1	Reading and Validating Data	53

6.3.2	Viewing Available Response Equations	54
6.3.3	Summarizing Replicate Measurements	54
6.3.4	Linear Fitting and Bias	55
6.3.5	Graphics and Prediction	56
6.4	Example 2: Polynomial Analysis of a Twofold Dilution	57
6.4.1	Reading and Validating Data	57
6.4.2	Compute replicate means and introduce weights:	57
6.4.3	Candidate Equation Comparison	58
6.5	Key Points for Interpreting Results	59
7	Analytical Sensitivity	61
7.1	Analysis Objectives	61
7.2	Overview of Functions	61
7.3	Example: LoB / LoD / LoQ Analysis	62
7.3.1	Reading and Validating Data	62
7.3.2	Per-Sample Summarization	62
7.3.3	Computing LoB / LoD / LoQ	63
7.3.4	Returned Object Structure	63
7.3.5	Graphics	64
7.4	Other Input Scenarios	65
7.4.1	Scenario 1: Blank Only (LoB Only)	65
7.4.2	Scenario 2: No Blank (LoQ Only)	66
7.4.3	Scenario 3: LoQ < LoD	66
7.5	Key Points for Interpreting Results	66
8	Bottle-to-Bottle Variability Analysis	69
8.1	Analysis Objectives	69
8.2	Overview of Functions	69
8.3	Example 1: Bottle-to-Bottle Difference of 15 Bottles $\times$ 5 Measurements	70
8.3.1	Reading and Validating Data	70
8.3.2	Step 1: Test for Significant Difference	70
8.3.3	Pairwise Comparison	71
8.3.4	Step 2: Whether the Difference Magnitude Meets Expectations	73
8.4	Example 2: Different Reagent Lots Measuring the Same Quality-Control Material	74
8.4.1	Reading Data	74
8.4.2	Testing the Between-Lot Difference	74
8.5	Key Points for Interpreting Results	76
9	Reference Intervals	77
9.1	Analysis Objectives	77
9.2	Overview of Functions	77
9.3	Example: Reference Intervals Using Three Methods Together	77
9.3.1	Reading and Validating Data	77
9.3.2	Pre-Analysis 1: Outlier Detection	78
9.3.3	Pre-Analysis 2: Normality Test	78
9.3.4	Reference Intervals Using Three Methods Together	80
9.4	Key Points for Interpreting Results	81
10	Quality Control	83
10.1	Analysis Objectives	83
10.2	Overview of Functions	83
10.3	Example 1: Obtaining Rules and Daily QC	83
10.3.1	Viewing the Westgard Rules	83
10.3.2	Reading Daily QC Data	84
10.3.3	Running Quality Judgments	84
10.4	Example 2: Grouped Evaluation on Reagent Lot Change	86
10.4.1	Reading Data	86
10.4.2	Grouping the Whole Sequence	86
10.5	Example 3: Youden Plot for Two-Level QC	87

10.5.1	Reading Data	87
10.5.2	Drawing the Youden Plot	87
10.6	Key Points for Interpreting Results	89
11	Sample Size Calculation	91
11.1	Analysis Objectives	91
11.2	Overview of Functions	91
11.3	Example 1: Sample Size for Bland–Altman Agreement	91
11.3.1	Finding Sample Size from Target Power	91
11.3.2	Finding Power from Sample Size	92
11.4	Example 2: Sample Size for a Single-Proportion Confidence Interval	92
11.4.1	Finding Sample Size from Target Half-Width	92
11.4.2	Finding Half-Width from Sample Size	93
11.5	Example 3: Sample Size for a One-Arm Target-Value Test	93
11.5.1	Finding Sample Size	93
11.5.2	Finding Power	94
11.5.3	Finding the Significance Level (α)	94
11.6	Key Points for Interpreting Results	94
12	Method Comparison	95
12.1	Analysis Objectives	95
12.2	Overview of Functions	95
12.3	Example: A Complete Single-Sample Analysis	95
12.3.1	Reading and Validating Data	95
12.3.2	Creating the Object and Descriptive Statistics	96
12.3.3	Correlation Analysis	97
12.3.4	Regression Analysis	97
12.3.5	Bias at Medical Decision Levels	99
12.3.6	Bland–Altman Analysis	100
12.3.7	Outlier Detection	102
12.3.8	Prediction	102
12.3.9	Subgroup Comparison	102
12.3.10	Summary	104
12.4	Key Points for Interpreting Results	105
13	Qualitative Analysis	107
13.1	Analysis Objectives	107
13.2	Overview of Functions	107
13.3	Example 1: Complete Analysis of Raw Paired Data	108
13.3.1	Reading and Validating Data	108
13.3.2	Building the Four-Fold Table	108
13.3.3	Describing the Raw Data	109
13.3.4	Diagnostic Accuracy Metrics	109
13.3.5	Kappa Agreement	110
13.3.6	McNemar Test	111
13.3.7	Summary	111
13.4	Example 2: Building Directly from TP/FP/TN/FN	113
13.5	Example 3: Converting Quantitative Data to Qualitative	115
13.5.1	Reading Data	115
13.5.2	Converting to Binary	115
13.5.3	Building the Four-Fold Table and Analyzing	116
13.6	Key Points for Interpreting Results	117
14	ROC Curve and Cutoff Determination	119
14.1	Analysis Objectives	119
14.2	Overview of Functions	119
14.3	Example: Single-Marker ROC Analysis	120
14.3.1	Reading and Validating Data	120
14.3.2	Creating the Object and Description	120

14.3.3	AUC	121
14.3.4	Optimal Cutoff	121
14.3.5	Graphics and Summary	125
14.4	Key Points for Interpreting Results	129
15	ROC Curve and Multivariate Regression	131
15.1	Analysis Objectives	131
15.2	Overview of Functions	131
15.3	Example: Joint Multi-Marker Analysis	131
15.3.1	Reading and Validating Data	131
15.3.2	Creating the Object and Single-Marker AUC	132
15.3.3	Multivariate Logistic Regression	132
15.3.4	Multi-Curve ROC Plot	133
15.3.5	Predicting New Samples	134
15.3.6	Summary	134
15.4	Key Points for Interpreting Results	135
16	Introduction to Algorithm Principles	137
16.1	Introduction	137
16.2	General Conventions and Notation	137
16.3	Outlier and Distribution Algorithms	137
16.3.1	Outliers	137
16.3.2	Normality	138
16.4	Method Comparison and Qualitative Evaluation Algorithms	138
16.4.1	Regression, Correlation, and Bias	138
16.4.2	Four-Fold Table	138
16.5	ROC Algorithms	138
16.6	Precision, ANOVA, and Reference Interval Algorithms	139
16.6.1	Variance Components	139
16.6.2	ANOVA and Reference Intervals	139
16.7	QC, Stability, and Equation Fitting Algorithms	139
16.7.1	QC and Stability	139
16.7.2	Equation Fitting	140
16.8	Module-to-Guideline Mapping	140
16.9	Scope of Use	140
17	AI-Automated Analysis	143
17.1	Overview	143
17.2	Applicable Analysis Tasks	143
17.3	Standard Workflow	144
17.3.1	1. Create a Dedicated Output Directory	144
17.3.2	2. Data Precheck	144
17.3.3	3. Determine the Analysis Plan	144
17.3.4	4. Execute the Analysis and Render the Report	144
17.3.5	5. Verify the Results	144
17.4	Output Contents	145
17.5	Installation and Usage	145
17.5.1	Runtime Environment	145
17.5.2	Use in Claude Code	145
17.5.3	Use in OpenAI Codex	145
17.5.4	Recommended Details When Submitting a Task	145
17.6	Data Handling Principles	146
17.7	Frequently Asked Questions	146
18	References	147
18.1	Introduction	147
18.2	Module-to-CLSI-Guideline Mapping	147
18.3	Methodological References	148
18.4	Usage Suggestions	148

Welcome

This book is an executable, auditable usage guide for `ivdtools`, intended for IVD practitioners, statisticians, and readers new to R. It covers everything from data entering the project to result export, experimental scenarios, and the statistical algorithms actually implemented in the package source code. Pedagogical recommendations are not equivalent to regulatory or standard requirements.

! Scope of Use

This book is for teaching statistical workflows and verifying implementations. It is not a substitute for product intended use, risk analysis, current regulations, or the original text of CLSI standards. Any performance claims should be supported by approved protocols, real samples, and independent review.

Reader's Guide

- Beginners read Part 1 in sequence, then move on to the function chapters by task.
- Experienced analysts can directly look up function signatures, object fields, and boundary behavior.

Version and Citation

```
packageVersion("ivdtools")
```

```
[1] '0.1.1'
```

In papers or reports, at least record: package name and version, R version, functions and key parameters, data snapshot checksums, and analysis date. The software may be cited as: hiox-tech (2026), *ivdtools: Statistical Tools for in Vitro Diagnostic Method Evaluation*, version 0.1.1. Please submit issues to the project's issue page.

Chapter 1

ivdtools Package Overview

1.1 Package Positioning

`ivdtools` is an R statistical toolkit for in vitro diagnostic (IVD) reagent evaluation, providing a set of reproducible workflows spanning data inspection, statistical analysis, and result summarization and visualization. The current version is 0.1.1, requires R $\geq 4.1.0$, and is released under the MIT license.

The package covers method comparison, ROC, qualitative agreement, precision, reference intervals, stability, quality control, curve fitting, analytical sensitivity, outlier and normality assessment, and sample-size calculation. Statistical results should be interpreted in the context of a pre-specified study protocol, applicable standards, laboratory requirements, and clinical or analytical allowable limits, and cannot substitute for professional judgment.

1.2 Design Features

1.2.1 Standalone Workflows

Standalone interfaces complete one task at a time, for example:

```
normal_test(dat, col = "candidate")
reference_interval(dat, col = "candidate")
```

1.2.2 Progressive Workflows

Modules such as method comparison, ROC, qualitative analysis, and precision organize the analysis process using S3 objects:

1. A constructor function creates an analysis object and stores the raw data and metadata;
2. An analysis method receives the object, computes results, and returns the updated object;
3. `summary()` summarizes the analyses that have been completed;
4. `plot()` produces plots from results already present in the object;
5. `predict()` performs prediction in supported modules.

```
data -> constructor -> analysis method -> updated object -> summary/plot/predict
```

The progressive interface requires reassignment:

```
obj <- analysis_method(obj)
```

If you call only `analysis_method(obj)` without the `obj <-`, the new results will not accumulate into the original object.

1.3 Functional Modules

Module	Main entry	Key features
Method comparison	<code>mcr()</code>	Descriptive statistics, correlation, OLS/WLS, Deming, weighted Deming, Passing–Bablok, Bland–Altman, bias and outlier analysis
ROC	<code>roc()</code>	ROC curve, AUC, optimal cutoff, multivariate logistic regression and prediction
Qualitative analysis	<code>raw_to_table()</code> , <code>counts_to_table()</code>	Four-fold table, diagnostic accuracy metrics, Kappa agreement and McNemar test
Precision	<code>precision()</code>	Variance components, confidence intervals, outliers, normality and Sadler precision profile
Reference interval	<code>reference_interval()</code>	Percentile, parametric and robust reference intervals
Bottle/lot ANOVA	<code>bottle_anova()</code>	One-way or nested ANOVA, Tukey HSD and compact letter display
Stability	<code>stability_bias()</code> , <code>stability_regression()</code>	Bias, stability regression, time to exceed limit, stability plan, MKT and Arrhenius models
Quality control	<code>qc_chart()</code> , <code>youden_plot()</code>	Levey–Jennings chart, Westgard rules and Youden plot
Curve fitting	<code>fit_equation()</code>	Linear, exponential, 4PLC, 5PLC and other equation fits, weighted/constrained fits and model comparison
Analytical sensitivity	<code>lob_lob_loq()</code>	Limit of blank, limit of detection, limit of quantitation determination
Outliers and distribution	<code>outliers_test()</code> , <code>normal_test()</code>	Grubbs, ESD, Dixon, IQR and multiple normality tests
Sample size	<code>sample_size_*</code>	Bland–Altman agreement, single-proportion confidence interval and one-arm target-value test

1.4 Quick Start

The following code uses the deterministic example data bundled with the package. See [Installing and Using the Analysis Environment](#) for installation and loading instructions.

1.4.1 Method Comparison

```
library(ivdtools)

mc <- mcr(
  ivd_mcr_example,
  id = "sid",
```

```

    candidate = "test",
    reference = "ref"
  )
mc <- describe(mc)
mc <- correlation(mc, method = "pearson")
mc <- regression(mc, method = "deming")
mc <- bland_altman(mc, type = "difference")

summary(mc)
plot(mc, type = "regression")
plot(mc, type = "bland_altman")

```

Regression methods include OLS, WLS, Deming, weighted Deming, and Passing-Bablok. The specific method, weights, confidence level, and outlier-handling strategy should be decided before analysis.

1.4.2 ROC

```

ro <- roc(
  ivd_roc_example,
  cols = c("x1", "x2"),
  reference = "ref",
  positive = 1
)
ro <- describe(ro)
ro <- auc(ro)
ro <- cutoff(ro)
ro <- mlr(ro, cols = c("x1", "x2"), name = "combined")

summary(ro)
plot(ro, mlr = "all", cutpoint = "Youden")

```

The reference variable must be binary. If the original reference result is continuous, first use `continuous_to_binary()`, and explicitly record the threshold and the positive direction.

1.4.3 Qualitative Agreement

```

qa <- raw_to_table(
  ivd_qualitative_example,
  candidate = "new",
  reference = "gold",
  id = "id",
  positive = "positive"
)
qa <- describe(qa)
qa <- diagnostics(qa)
qa <- kappa(qa)
qa <- mcnemar(qa)

summary(qa)

```

When TP, FP, TN, FN counts are already available, you can also create an object with `counts_to_table()`.

1.4.4 Reference Interval

```
set.seed(20260806)
ri_data <- data.frame(value = rnorm(160, mean = 100, sd = 15))

ri <- reference_interval(
  ri_data,
  col = "value",
  method = "all"
)
print(ri)
plot(ri)
```

The percentile, parametric, or robust method should be chosen based on sample source, sample size, distribution characteristics, and the study protocol.

1.4.5 Bottle/Lot ANOVA

```
ba <- bottle_anova(ivd_bottle_example, value ~ bottle)
print(ba)

posthoc <- tukey(ba)
print(posthoc)
```

1.4.6 Standalone Tools

```
# Outliers and normality
outliers_test(ri_data, "value", method = "grubbs")
normal_test(ri_data, "value", method = "shapiro")

# View available fit equations and Westgard rules
list_equation()
list_westgard()

# Sample size / power
sample_size_bland_altman(
  power = 0.80,
  mu = 0.2,
  sd = 1,
  delta = 2.5
)
```

1.4.7 Getting Help

View the function documentation:

```
help(mcr, "ivdtools")
```

View the help documentation and browse topic-specific files.

```
vignette("ivdtools")
```

Visit the project homepage to view/download the complete function manual and help documentation.

1.5 AI Automated Analysis

The `ivdtools` R package provides statistical functions, while the `ivdtools-analysis` Skill provides a constrained automated analysis workflow for AI. Once the user supplies the data file, study design, and analysis requirements, the Skill uses `ivdtools` as its statistical engine to organize data pre-check, parameter confirmation, statistical analysis, result verification, and Chinese report generation.

The value of using the Skill in a conversation is not to “replace statistical judgment” but to standardize repetitive technical steps, and to save the data source, analysis parameters, excluded records, warnings, result tables, figures, and software environment together, reducing the risk of omissions and manual copy errors.

`ivdtools-analysis` is not installed with `ivdtools`; it can be obtained from the project homepage.

1.6 Dependencies

`ivdtools` mainly depends on `ggplot2`, `ggrepel`, `VCA`, `VFP`, `minpack.lm`, `nloptr`, `nls2`, `nortest`, and `rlang`.

1.7 Scope of Use

- Software output does not automatically constitute a pass/fail determination; acceptance criteria should be defined before analysis.
- Before use in regulated scenarios, applicable software validation, process verification, and result review should be completed.

1.8 Project Information

- Project homepage: <https://github.com/hiox-tech/ivdtools>
- Issue feedback: <https://github.com/hiox-tech/ivdtools/issues>
- License: MIT

Chapter 2

Installing and Using the Analysis Environment

2.1 Purpose of This Document

This document explains how to set up the `ivdtools` analysis environment, create an R Markdown project, import and validate data, run a progressive analysis, and save reproducible results.

2.2 Installing R

`ivdtools` 0.1.1 requires R $\geq$ 4.1.0. It is recommended to use a still-supported newer R version and to keep the same major version and dependency versions across all people in the project.

2.2.1 Windows and macOS

Download the installer for your system from CRAN: <https://cran.r-project.org/>.

- Windows: install R; Rtools of a matching version is needed only when building packages containing native code from source.
- macOS: choose the installer according to your processor architecture; Xcode Command Line Tools may be needed when building from source.

2.2.2 Debian/Ubuntu

The base R can be installed from the system repository:

```
sudo apt update
sudo apt install r-base r-base-dev
```

2.2.3 Fedora

```
sudo dnf install R R-devel
```

The R version in Linux distribution repositories may be older; if the project needs a newer R, configure the software source according to the CRAN instructions for the corresponding distribution.

2.3 Choosing an Editor

It is recommended to use `.Rmd` or `.qmd` files to record code, text, tables, and figures.

Software	Windows	Linux	macOS	Suitable for
RStudio				Works out of the box; suitable for beginners and routine analysis
Positron				Suitable for users who want the new-generation Posit IDE
Visual Studio Code				Requires configuring R, Quarto and other extensions
Jupyter				Requires Python, Jupyter and an R kernel
PyCharm				Requires corresponding R support and extra configuration

RStudio and Positron can be obtained from <https://posit.co/downloads/>; Visual Studio Code can be obtained from <https://code.visualstudio.com/Download>.

2.4 Installing R Packages

Once `ivdtools` has been released on the CRAN mirror you use:

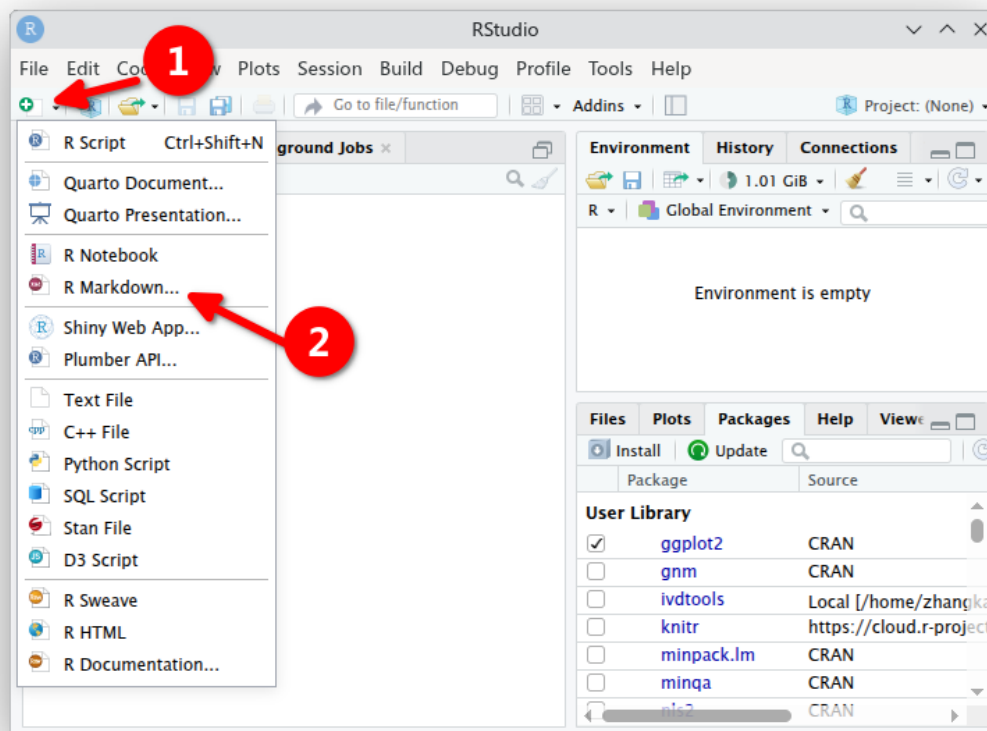
```
install.packages(c("ivdtools", "rmarkdown", "knitr"))
```

Additional packages can be installed as needed for data import:

```
install.packages(c("readr", "readxl"))
```

2.5 Creating a New Project

In RStudio, choose **File > New File > R Markdown**.



Minimal document template:

```
---
title: "IVD Analysis Report"
author: "Analyst"
date: "Enter the report date"
output:
  html_document:
    toc: true
    number_sections: true
---
```

Then insert an R code chunk named `setup` with `include=FALSE` set, containing the following:

```
knitr::opts_chunk$set(collapse = TRUE, comment = "#>")
library(ivdtools)
```

2.6 Importing and Validating Data

2.6.1 CSV

Base R:

```
dat <- read.csv(
  "data-raw/method-comparison.csv",
  stringsAsFactors = FALSE,
  check.names = FALSE
)
```

Using `readr`:

```
dat <- readr::read_csv(
  "data-raw/method-comparison.csv",
  show_col_types = FALSE
)
```

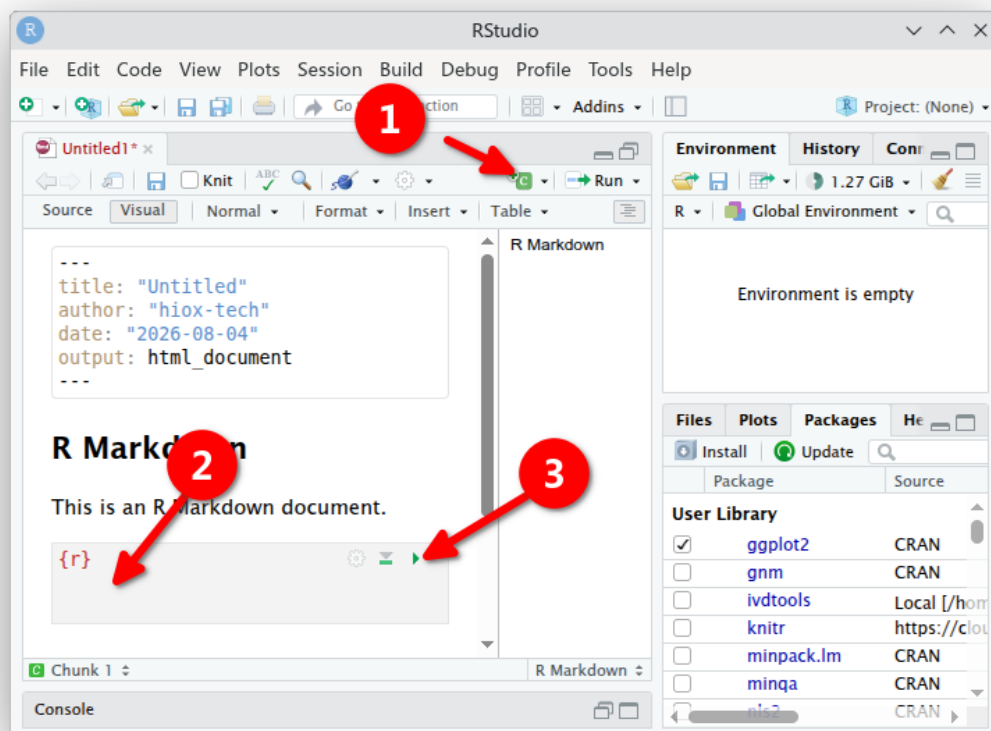
2.6.2 Excel

```
dat <- readxl::read_excel(
  "data-raw/method-comparison.xlsx",
  sheet = "data"
)
```

If the Excel file contains formulas, merged cells, or manual formatting, first confirm that the imported values, column names, and data types match the original sheet.

2.7 Running Code

In a `.Rmd`, insert an R code chunk and use the button at the top-right of the chunk to run the current chunk, or run all code from top to bottom.



To ensure reproducibility, the final report should be rendered from scratch in a fresh R session, rather than relying on objects already present in the Console.

2.8 Exporting Results

2.8.1 Tables

After reading the result fields described on the help page, you can write out a CSV:

```
# Example: the actual fields should be confirmed against the help page of the corresponding obj
result_table <- mc$regression
saveRDS(result_table, "tables/regression-result.rds")
```

Complex objects are best saved as RDS to preserve their class and attributes; when exchanging with other software, tidy them into flat data frames and export as CSV.

2.8.2 Figures

```
p <- plot(mc, type = "bland_altman")
ggplot2::ggsave(
  "figures/bland-altman.png",
  plot = p,
  width = 7,
  height = 5,
  dpi = 300
)
```

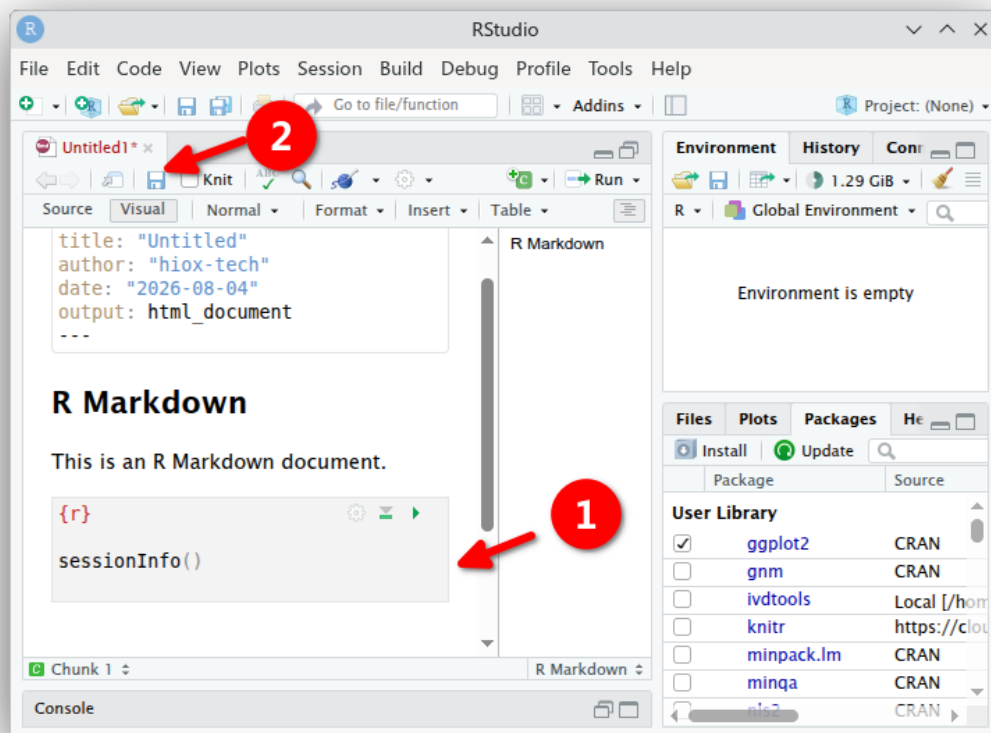
2.9 Rendering and Saving

In RStudio, click **Knit**, or run:

```
rmarkdown::render(
  "report/analysis.Rmd",
  output_format = "html_document",
  clean = TRUE
)
```

After the analysis is complete, record the environment information:

```
sessionInfo()
```



Recommended for archiving:

1. The unmodified raw data;
2. The `.Rmd` source file;
3. The rendered HTML/PDF;
4. Readable results and figures;
5. The `sessionInfo()` environment information;

Chapter 3

4PLC Equation Calibration Curve Fitting

3.1 Analysis Objectives

This document demonstrates how to fit a four-parameter logistic (4PLC) calibration curve using `fit_equation()` from the `ivdtools` package, and how to examine convergence status, residuals, weight effects, and forward and inverse prediction.

4PLC is a nonlinear model. A successfully returned fit object does not imply the model is adequate; formal analysis should also incorporate the calibration point design, lower and upper asymptote coverage, parameter stability, residual structure, repeat-measurement precision, and pre-specified acceptance criteria.

```
library(ivdtools)
library(readr)
```

3.2 4PLC Model and Main Functions

In `ivdtools`, 4PLC corresponds to E07 and can also be invoked by the name "4PLC":

$$y = A + \frac{B - A}{1 + (x/C)^D}$$

Here, A and B denote the two asymptotic plateaus, C denotes the concentration scale at the curve midpoint, and D controls the curve direction and steepness. The meaning of the parameters should be interpreted in the context of the actual signal direction.

Function	Purpose
<code>list_equation()</code>	View the built-in equation registry
<code>replicate_to_mean()</code>	Summarize replicate measurements and optionally compute weights
<code>fit_equation()</code>	Fit built-in or custom equations
<code>coef()</code>	Extract model parameters
<code>residuals()</code>	Extract residuals (observed minus fitted)
<code>predict()</code>	Forward prediction of response or inverse estimation of concentration
<code>plot()</code>	Plot observed points, fitted curve, and intervals
<code>compare_equation()</code>	Compare candidate equations

View the response-curve models:

```
list_equation(category = "Response")
```

ID	Name	Formula	Params	nP	Engine
E01	Linear	$a + b \cdot x$	a, b	2	lm
E02	Quadratic	$a + b \cdot x + c \cdot x^2$	a, b, c	3	lm
E03	ExpoGrowth	$a \cdot \exp(b \cdot x)$	a, b	2	nls
E04	ExpoDecay	$a \cdot \exp(-b \cdot x)$	a, b	2	nls
E05	Power	$a \cdot x^b$	a, b	2	nls
E06	Log	$a + b \cdot \log(x)$	a, b	2	lm
E07	4PLC	$A + (B - A) / (1 + (x / C)^D)$	A, B, C, D	4	nls
E08	5PLC	$A + (B - A) / (1 + (x / C)^D)^E$	A, B, C, D, E	5	nls
E09	Gaussian	$a \cdot \exp(-(x - b)^2 / (2 \cdot c^2))$	a, b, c	3	nls
E10	Michaelis	$V_{\max} \cdot x / (K_m + x)$	Vmax, Km	2	nls
E11	Sinusoidal	$a \cdot \sin(b \cdot x + c) + d$	a, b, c, d	4	nls
E12	Cubic	$a + b \cdot x + c \cdot x^2 + d \cdot x^3$	a, b, c, d	4	lm

`fit_equation()` can use automatic starting values or take them via `start`; `lower`, `upper`, and `constraints` set parameter bounds or constraints. `weights` accepts a numeric vector or the built-in schemes "equal", "1/y", "1/y^2", "1/x", "1/x^2".

Note: Weights should reflect the measurement error or variance structure; they should not be chosen only because the curve is positively or negatively correlated. It is best to pre-specify weights based on repeat-measurement precision, residual diagnostics, or the study protocol.

3.3 Example 1: Basic Fitting Workflow

3.3.1 Reading and Validating Data

```
u1 <- read_csv("./data/4PLC-U1.csv", show_col_types = FALSE)
u1
```

```
# A tibble: 6 x 3
  Sample    Con   RLU
  <chr>    <dbl> <dbl>
1 s1         0   1581
2 s2        4.08  3857
3 s3       24.9 12931
4 s4       92.8 36153
5 s5      339. 106083
6 s6     1018. 303726
```

```
str(u1)
```

```
spc_tbl_ [6 x 3] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ Sample: chr [1:6] "s1" "s2" "s3" "s4" ...
 $ Con   : num [1:6] 0 4.08 24.93 92.75 338.54 ...
 $ RLU   : num [1:6] 1581 3857 12931 36153 106083 ...
- attr(*, "spec")=
.. cols(
..   Sample = col_character(),
..   Con = col_double(),
..   RLU = col_double()
.. )
- attr(*, "problems")=<externalptr>
```



```
summary(u1[c("Con", "RLU")])
```

Con		RLU	
Min. :	0.000	Min. :	1581
1st Qu.:	9.293	1st Qu.:	6126
Median :	58.840	Median :	24542
Mean :	246.308	Mean :	77388
3rd Qu.:	277.092	3rd Qu.:	88600
Max. :	1017.550	Max. :	303726

```
anyDuplicated(u1$Sample)
```

```
[1] 0
```

```
sum(!is.finite(u1$Con) | !is.finite(u1$RLU))
```

```
[1] 0
```

This data contains 6 concentration levels. Because 4PLC requires estimating 4 parameters, the residual degrees of freedom are very few; in addition, the highest concentration does not yet clearly cover the upper plateau, so the model parameters may be unstable. This example is mainly intended to illustrate the operating workflow and should not be used directly as a formal calibration design example.

If each concentration contains replicates, you can fit all observations directly, or first summarize them with `replicate_to_mean()`. Whether to summarize and how to weight should be stated in the analysis plan.

3.3.2 Fitting and Checking Convergence

```
fit_u1 <- fit_equation(
  "4PLC",
  data = u1,
  x = "Con",
  y = "RLU"
)
print(fit_u1)
```

E07 (4PLC)

Formula: $A + (B - A) / (1 + (x / C)^D)$

Call:

```
fit_equation(eq = "4PLC", data = u1, x = "Con", y = "RLU")
```

Residuals:

Min	1Q	Median	3Q	Max
-1991.214	-1335.541	-252.849	960.620	2652.956

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
A	3.10445e+03	2.09875e+03	1.47919	0.277197
B	6.13447e+07	2.60964e+09	0.02351	0.983380
C	2.60778e+05	1.18118e+07	0.02208	0.984391
D	-9.58182e-01	1.20691e-01	-7.93917	0.015498 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2775.94 on 2 degrees of freedom

AIC: 115.58 Iterations: 50

Do not ignore the optimizer's convergence information:

```
fit_u1$fit$convInfo[c("isConv", "finIter", "stopCode", "stopMessage")]

$isConv
[1] FALSE

$finIter
[1] 50

$stopCode
[1] -1

$stopMessage
[1] "Number of iterations has reached `maxiter' == 50."
```

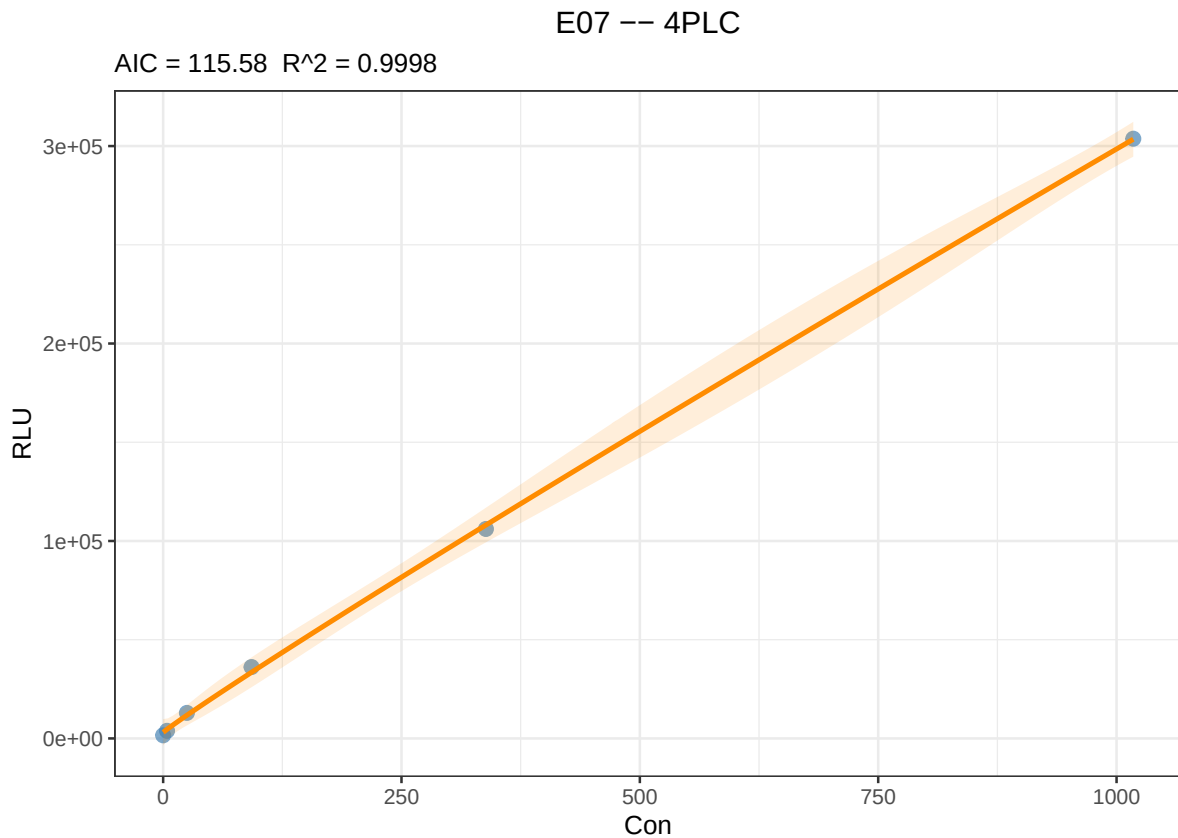
The default fit for this data reaches the iteration limit without declaring convergence. Increasing the maximum number of iterations sometimes helps, but cannot resolve a calibration range that does not cover a plateau or parameters that are not identifiable. When convergence is not achieved, inspect the data, model direction, starting values, parameter bounds, and calibration point design in turn, rather than accepting the result based only on the curve appearance.

3.3.3 Residuals and Graphics

```
u1_diagnostics <- data.frame(
  Sample = u1$Sample,
  Con = u1$Con,
  RLU = u1$RLU,
  Residual = residuals(fit_u1),
  Relative_residual_pct = residuals(fit_u1) / u1$RLU * 100
)
knitr::kable(u1_diagnostics, digits = 2)
```

Sample	Con	RLU	Residual	Relative_residual_pct
s1	0.00	1581	-1523.45	-96.36
s2	4.08	3857	-771.83	-20.01
s3	24.93	12931	1192.12	9.22
s4	92.75	36153	2652.96	7.34
s5	338.54	106083	-1991.21	-1.88
s6	1017.55	303726	266.13	0.09

```
plot(fit_u1, interval = "confidence", level = 0.95)
```



Residuals should be examined on both the original response scale and the relative scale, and interpreted together with the repeat-measurement variance. For responses spanning several orders of magnitude, looking at original residuals alone may let high-signal points dominate the judgment.

3.3.4 Predicting Response from Concentration

```
predict(
  fit_u1,
  newdata = data.frame(Con = c(20, 200, 1000)),
  interval = "confidence",
  level = 0.95
)
```

x	y	y_ci_lwr	y_ci_upr
20	10095.70	5194.229	14997.17
200	66540.68	59494.141	73587.22
1000	298518.29	289967.527	307069.05

The mean confidence interval describes the uncertainty of the fitted mean; the prediction interval also includes single-observation error and is usually wider:

```
predict(
  fit_u1,
  newdata = data.frame(Con = c(20, 200, 1000)),
  interval = "prediction",
  level = 0.95
)
```

x	y	y_pi_lwr	y_pi_upr
20	10095.70	-7.215948	20198.61
200	66540.68	55240.318293	77841.05
1000	298518.29	286223.575001	310813.01

3.3.5 Estimating Concentration from Response (Inverse Prediction)

```
predict(
  fit_u1,
  newdata = data.frame(RLU = c(20000, 100000, 250000)),
  inverse = TRUE
)
```

x	y
50.2395	20000
311.3672	100000
828.5559	250000

Prediction boundaries: Avoid extrapolating beyond the calibration concentration range. Inverse prediction is very sensitive to response error in the plateau region, and non-monotonic models can also have multiple solutions. The current example fit did not converge reliably, so the predictions are used only to demonstrate the interface and cannot be used for quantitative conclusions.

3.4 Example 2: Sandwich Assay and Weights

3.4.1 Equal-Weight Fit

Sandwich assay data typically show increasing signal with increasing concentration. This example first performs an equal-weight fit:

```
u2 <- read_csv("./data/4PLC-U2.csv", show_col_types = FALSE)

fit_u2_equal <- fit_equation(
  "4PLC",
  data = u2,
  x = "Con",
  y = "RLU"
)
print(fit_u2_equal)
```

E07 (4PLC)

Formula: $A + (B - A) / (1 + (x / C)^D)$

Call:

```
fit_equation(eq = "4PLC", data = u2, x = "Con", y = "RLU")
```

Residuals:

Min	1Q	Median	3Q	Max
-1034493.852	-58331.509	17644.023	45830.410	785037.098

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
A	-4.19069e+04	2.74298e+05	-0.15278	0.88288329
B	4.01980e+08	9.51404e+08	0.42251	0.68532695
C	1.12895e+03	3.16059e+03	0.35720	0.73147078
D	-1.02539e+00	1.32241e-01	-7.75390	0.00011121 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 565331 on 7 degrees of freedom

AIC: 327.64 Iterations: 24

3.4.2 Inverse Response-Squared Weight Fit

When the response variance clearly increases with signal, an inverse-response-squared weight can be evaluated:

```
fit_u2_weighted <- fit_equation(
  "4PLC",
  data = u2,
  x = "Con",
  y = "RLU",
  weights = "1/y^2"
)
print(fit_u2_weighted)
```

E07 (4PLC)

Formula: $A + (B - A) / (1 + (x / C)^D)$

Call:

```
fit_equation(eq = "4PLC", data = u2, x = "Con", y = "RLU", weights = "1/y^2")
```

Residuals:

Min	1Q	Median	3Q	Max
-1038926.254	-2031.562	491.283	35707.649	809345.965

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
A	5.66828e+03	2.30614e+02	24.57907	4.7016e-08 ***
B	2.00185e+08	6.72603e+07	2.97627	0.020624 *
C	4.83741e+02	1.74222e+02	2.77658	0.027433 *
D	-1.08082e+00	1.02400e-02	-105.54858	1.8062e-12 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.042025 on 7 degrees of freedom

AIC: 265.29 Iterations: 7

Compare the relative residuals of the two models at each calibration point:

```
u2_comparison <- data.frame(
  Sample = u2$Sample,
  Con = u2$Con,
  RLU = u2$RLU,
  Equal_weight_pct = residuals(fit_u2_equal) / u2$RLU * 100,
  Inverse_y2_weight_pct = residuals(fit_u2_weighted) / u2$RLU * 100
)
knitr::kable(u2_comparison, digits = 2)
```

Sample	Con	RLU	Equal_weight_pct	Inverse_y2_weight_pct
c1	0.00	5637	843.43	-0.55
c2	0.05	16014	275.49	3.07
c3	0.20	47520	67.84	-4.70
c4	0.51	125038	14.11	-1.46
c5	1.03	277621	4.57	4.70
c6	5.19	1498023	-4.39	0.98
c7	10.92	3334387	-1.53	1.70
c8	20.83	6148959	-6.53	-5.30
c9	49.90	16515788	4.75	4.13
c10	95.47	28504677	-3.63	-3.64

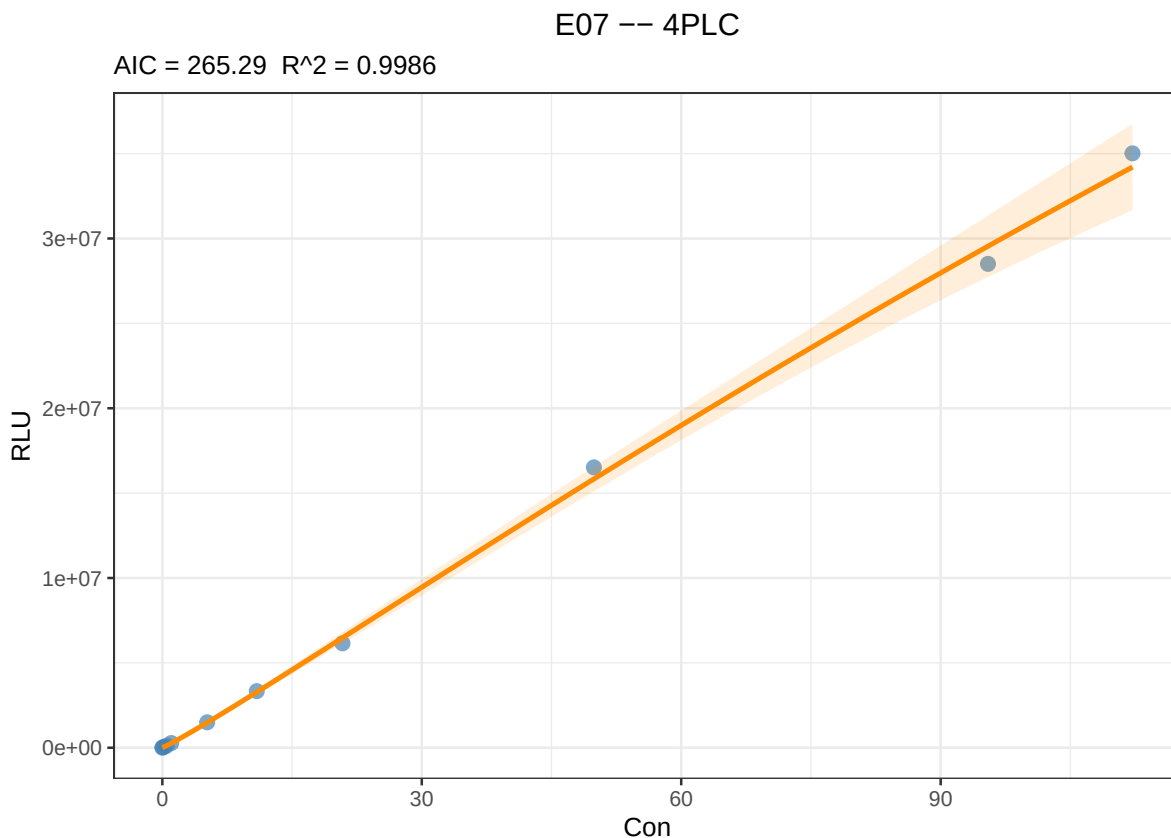
Sample	Con	RLU	Equal_weight_pct	Inverse_y2_weight_pct
c11	112.18	35017203	1.75	2.31

```
data.frame(
  Model = c("Equal weight", "1/y^2 weight"),
  Median_absolute_relative_residual_pct = c(
    median(abs(u2_comparison$Equal_weight_pct)),
    median(abs(u2_comparison$Inverse_y2_weight_pct))
  ),
  Maximum_absolute_relative_residual_pct = c(
    max(abs(u2_comparison$Equal_weight_pct)),
    max(abs(u2_comparison$Inverse_y2_weight_pct))
  )
) |>
  knitr::kable(digits = 2)
```

Model	Median_absolute_relative_residual_pct	Maximum_absolute_relative_residual_pct
Equal weight	4.75	843.43
1/y^2 weight	3.07	5.30

In this example data, the $1/y^2$ weight clearly reduces the relative residuals at low-signal points; this only indicates that this weight better matches the relative-error evaluation objective adopted for this data. The formal choice still needs to combine the variance of the replicates at each concentration, the back-calculated concentration bias, model parameter stability, and pre-specified acceptance criteria. The residual standard errors and AIC under different weights have different scales and should not be compared directly without an error model.

```
plot(fit_u2_weighted, interval = "confidence", level = 0.95)
```



3.5 Example 3: Competitive Assay

Competitive assay data typically show decreasing signal with increasing concentration:

```
u3 <- read_csv("./data/4PLC-U3.csv", show_col_types = FALSE)

fit_u3 <- fit_equation(
  "4PLC",
  data = u3,
  x = "Con",
  y = "RLU"
)
print(fit_u3)
```

E07 (4PLC)

Formula: $A + (B - A) / (1 + (x / C)^D)$

Call:

```
fit_equation(eq = "4PLC", data = u3, x = "Con", y = "RLU")
```

Residuals:

Min	1Q	Median	3Q	Max
-1937.581	-454.210	-28.126	625.582	1807.765

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
A	-2.81576e+04	2.17005e+03	-12.9755	1.1793e-06 ***
B	1.40035e+06	1.08896e+03	1285.9562	< 2.22e-16 ***
C	1.29783e+02	6.49733e-01	199.7479	4.4162e-16 ***
D	7.82733e-01	2.79342e-03	280.2058	< 2.22e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1093.1 on 8 degrees of freedom

AIC: 207.11 Iterations: 5

```
fit_u3$fit$convInfo[c("isConv", "finIter", "stopCode", "stopMessage")]
```

\$isConv

[1] TRUE

\$finIter

[1] 5

\$stopCode

[1] 1

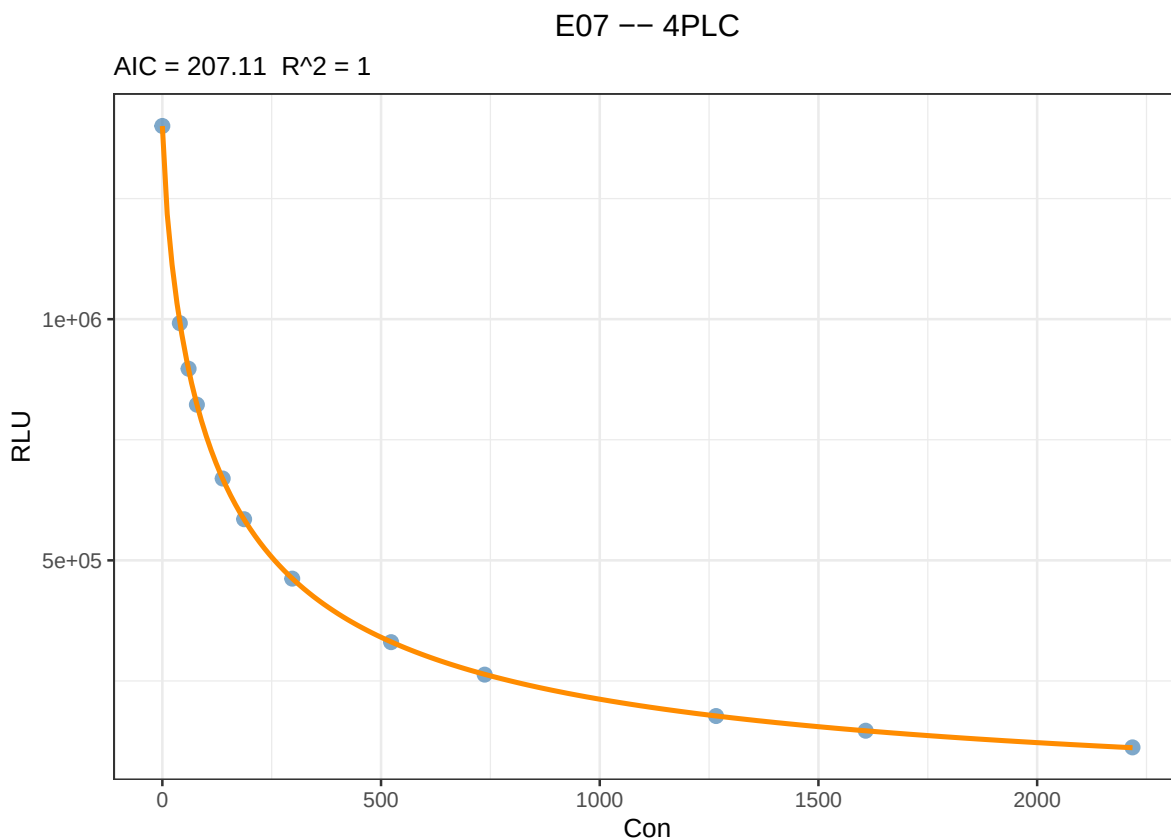
\$stopMessage

[1] "Relative error in the sum of squares is at most `ftol'."

```
u3_diagnostics <- data.frame(
  Sample = u3$Sample,
  Con = u3$Con,
  RLU = u3$RLU,
  Relative_residual_pct = residuals(fit_u3) / u3$RLU * 100
)
knitr::kable(u3_diagnostics, digits = 2)
```

Sample	Con	RLU	Relative_residual_pct
m1	0	1400548	0.01
m2	40	991716	-0.20
m3	60	897247	0.20
m4	79	822749	-0.05
m5	138	669646	0.11
m6	187	585293	0.10
m7	297	462119	-0.07
m8	523	330399	-0.19
m9	737	263095	-0.25
m10	1266	177260	-0.12
m11	1608	146827	0.11
m12	2218	112252	0.61

```
plot(fit_u3, interval = "confidence", level = 0.95)
```



In this example, the equal-weight fit has already converged and the in-sample relative residuals are small, but you should still decide whether weights are needed based on the replicate measurement variance. The signal direction itself is not a basis for choosing between equal-weight and weighted fitting.

3.6 Key Points for Interpreting Results

1. **Calibration design:** Concentration points should cover the intended working range and, where possible, provide information about the upper and lower plateaus.
2. **Convergence status:** Check `convInfo`; do not treat reaching the maximum iterations as successful convergence.
3. **Parameter plausibility:** Judge stability in light of the plateaus, curve midpoint, direction, standard errors, and parameter correlations.

4. **Residual structure:** Examine the original residuals, relative residuals, and their variation with concentration or fitted value.
5. **Weight rationale:** Prefer variance structures estimated from replicate measurements, and specify the selection rule before analysis.
6. **Back-calculation performance:** Evaluate the back-calculated concentration bias and precision at each calibration point as required by the project.
7. **Prediction range:** Avoid extrapolation, especially unstable inverse prediction in the plateau region.
8. **Reproducibility:** Save the raw data, code, model parameters, package version, figures, and complete session information.

Chapter 4

Stability Studies

4.1 Analysis Objectives

Stability studies evaluate the extent to which measurement results change over time for reagents, calibrators, or quality-control materials under specified storage, transport, or use conditions. This document demonstrates how to use the `ivdtools` package to establish a stability analysis plan, complete node bias, regression of trends within and between conditions relative to the first node, prediction of the time to reach an allowable limit, Arrhenius accelerated stability analysis, and mean kinetic temperature (MKT) computation.

```
library(ivdtools)
library(readr)
```

All data in this document are deterministic teaching examples used only to validate the analysis workflow. The 5% allowable drift, 10% accelerated degradation limit, target temperature, and activation energy in the examples are not universal acceptance criteria; formal studies must use parameters pre-specified in the protocol, risk assessment, or applicable standards. Whether to choose a concentration value or a signal value as the response should be decided according to the protocol.

4.2 Overview of Functions

Function	Main purpose	Key input or output
<code>stability_plan()</code>	Plan the minimum number of time points for a regression series	Allowable drift, variability, replicates, expected drift and power
<code>stability_bias()</code>	Compute absolute/relative bias of each condition relative to the first node	Node summary, within-condition bias, between-condition bias and limit status
<code>stability_regression()</code>	Regression of a single-condition trend or test/reference paired bias	Slope, one-sided confidence limit, node results and diagnostics
<code>stability_time()</code>	Predict the time when the point estimate or one-sided confidence limit reaches the allowable bias	Predicted time, whether extrapolated, and search range
<code>arrhenius()</code>	Extrapolate the shelf life at target temperature from degradation rates at multiple elevated temperatures	Kinetic order, rate, activation energy and predicted shelf life

Function	Main purpose	Key input or output
<code>mkt()</code>	Compute equally spaced or time-weighted mean kinetic temperature	Per-probe and combined MKT

`stability_regression()` supports two modes:

- `mode = "self"`: regress the measurement results of a single condition over time;
- `mode = "compare"`: compute the bias of the test condition relative to the reference condition at matched time points, then regress the bias over time.

Before analysis, clarify the condition mapping, time unit, bias scale, expected direction of change, and allowable limits. Conclusions about acceptability can be drawn only when the protocol provides acceptance criteria.

4.3 Example 1: Establishing an Analysis Plan

Suppose the study protocol specifies an allowable relative drift of 5%, an expected drift of 2.5%, repeatability variability of 1.5%, and a target power of 90%. Compare the minimum number of time points needed when measuring 1, 2, or 3 times at each time point:

```
plan <- ivdtools::stability_plan(
  allowable_drift = 5,
  variability = 1.5,
  replicates = c(1, 2, 3),
  expected_drift = 2.5,
  power = 0.9,
  mode = "simple",
  bias_type = "relative"
)
print(plan)
```

Stability Study Time-Point Plan

Allowable drift: 5.0000; expected drift: 2.5000

Success target: 90%; method: CLSI EP25 Appendix A Table A2

Replicates	Residual	Allow/Resid	Min nodes
1	1.5000	3.3333	n/a
2	1.0607	4.5455	28
3	0.8660	5.7471	18

n/a -- No feasible entry in EP25 table.

4.4 Example 2: Bias Analysis

4.4.1 Reading and Validating Raw Data

```
bias_data <- read_csv(
  "./data/stability-bias-example.csv",
  show_col_types = FALSE
)
bias_data
```

```
# A tibble: 30 x 4
  Condition Day Replicate Result
  <chr>      <dbl>      <dbl>  <dbl>
1 2-8C      0          1  100.
2 2-8C      0          2  99.8
3 2-8C      0          3  100
4 2-8C      7          1  99.9
5 2-8C      7          2  100.
6 2-8C      7          3  99.8
7 2-8C     14          1  99.7
8 2-8C     14          2  99.9
9 2-8C     14          3  99.6
10 2-8C     21          1  99.5
# i 20 more rows

dim(bias_data)

[1] 30  4

str(bias_data)

spec_tbl_ [30 x 4] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ Condition: chr [1:30] "2-8C" "2-8C" "2-8C" "2-8C" "2-8C" ...
 $ Day      : num [1:30] 0 0 0 7 7 7 14 14 14 21 ...
 $ Replicate: num [1:30] 1 2 3 1 2 3 1 2 3 1 ...
 $ Result   : num [1:30] 100.2 99.8 100 99.9 100.1 ...
- attr(*, "spec")=
.. cols(
..   Condition = col_character(),
..   Day = col_double(),
..   Replicate = col_double(),
..   Result = col_double()
.. )
- attr(*, "problems")=<externalptr>
```

4.4.2 Node Bias Computation

Using 2-8C as the reference condition, normalize to each condition's own first node, and evaluate whether the relative bias is within the illustrative $\pm 5\%$ limit:

```
bias_result <- ivdtools::stability_bias(
  data = bias_data,
  condition = "Condition",
  time = "Day",
  value = "Result",
  reference_condition = "2-8C",
  limit = 5,
  bias_type = "relative",
  time_unit = "d"
)
print(bias_result)
```

```
Stability Bias Analysis
Conditions:      2 (2-8C, 25C)
Reference:      2-8C
Time unit:      d
Node comparisons:20
```

Acceptance: +/- 5.0000 (relative); 10/10 nodes passed

Within-condition bias (vs first node)

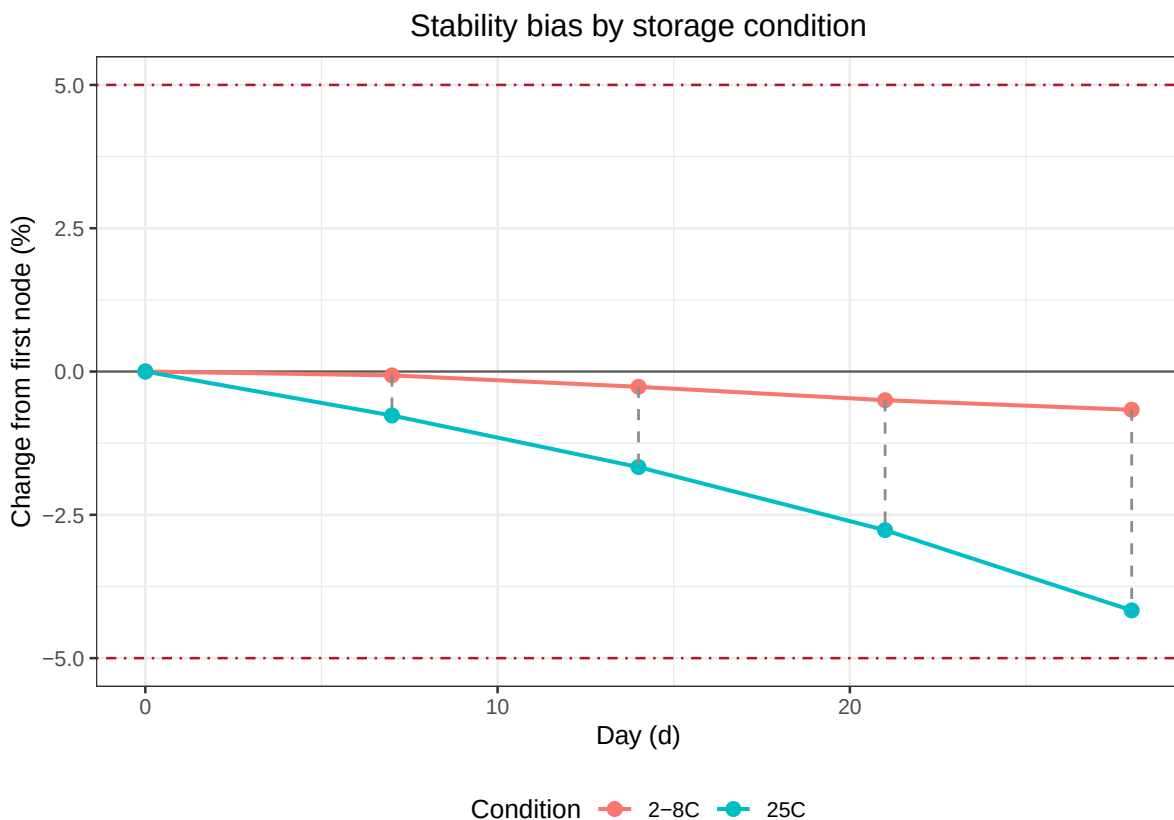
Cond	Time	Value	Bias(%)	Status
2-8C	0.0000	100.0000	0.0000	Pass
2-8C	7.0000	99.9333	-0.0667	Pass
2-8C	14.0000	99.7333	-0.2667	Pass
2-8C	21.0000	99.5000	-0.5000	Pass
2-8C	28.0000	99.3333	-0.6667	Pass
25C	0.0000	100.0000	0.0000	Pass
25C	7.0000	99.2333	-0.7667	Pass
25C	14.0000	98.3333	-1.6667	Pass
25C	21.0000	97.2333	-2.7667	Pass
25C	28.0000	95.8333	-4.1667	Pass

Between-condition bias (reference: 2-8C)

Time	Condition	TestVal	Bias(%)
0.0000	25C	100.0000	0.0000
7.0000	25C	99.2333	-0.7005
14.0000	25C	98.3333	-1.4037
21.0000	25C	97.2333	-2.2781
28.0000	25C	95.8333	-3.5235

Within-condition bias uses each condition's own first node as the baseline; between-condition bias compares the test and reference conditions at the same time points.

```
plot(bias_result)
```



4.5 Example 3: Regression Analysis Comparing Storage Conditions

4.5.1 Reading and Validating Raw Data

```
regression_data <- read_csv(
  "./data/stability-regression-example.csv",
  show_col_types = FALSE
)
regression_data
```

A tibble: 36 x 4

	Condition	Day	Replicate	Result
	<chr>	<dbl>	<dbl>	<dbl>
1	2-8C	0	1	100.
2	2-8C	0	2	99.9
3	2-8C	0	3	100
4	2-8C	7	1	100
5	2-8C	7	2	99.8
6	2-8C	7	3	100.
7	2-8C	14	1	99.9
8	2-8C	14	2	100
9	2-8C	14	3	99.8
10	2-8C	21	1	99.8

i 26 more rows

4.5.2 Comparing Test and Reference Conditions

Compute the relative bias of 25C relative to 2-8C at each matched time point, and regress the bias over time. The example pre-specifies that results decrease over time, so `direction = "decrease"` is used:

```
regression_result <- ivdtools::stability_regression(
  data = regression_data,
  time = "Day",
  value = "Result",
  condition = "Condition",
  mode = "compare",
  test_condition = "25C",
  reference_condition = "2-8C",
  bias_type = "relative",
  direction = "decrease",
  conf.level = 0.95,
  limit = 5,
  time_unit = "d"
)
print(regression_result)
```

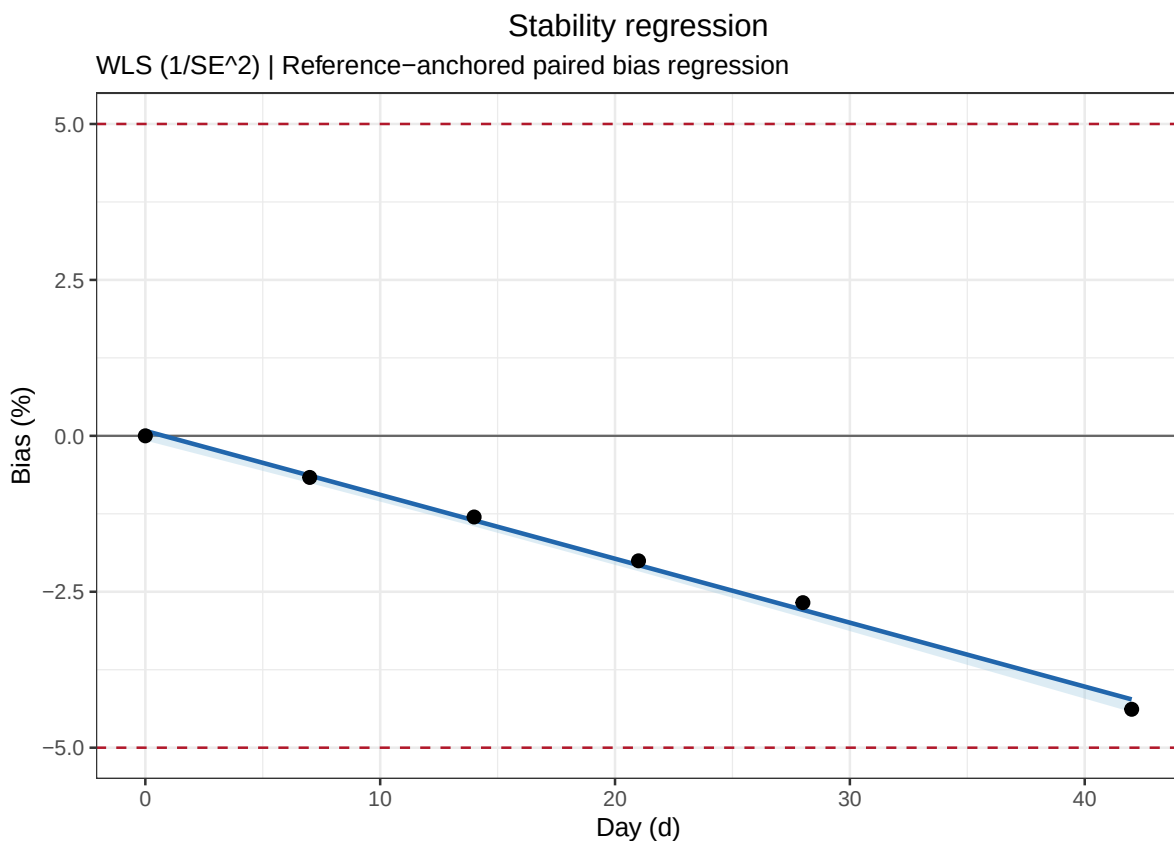
Stability Regression

```
Mode: compare
Test condition: 25C
Reference: 2-8C
Fit: WLS (1/SE^2); intercept = 0.0799, slope = -0.1025
Direction: decrease
One-sided CI: 95.0%
R-squared: 0.9956
```

Cook distance: row(s) 6 > 1; not removed

Time	ObsBias	Estimate	CI_limit	Status
0.0000	0.0000	0.0799	-0.0750	
7.0000	-0.6669	-0.6376	-0.7566	
14.0000	-1.3013	-1.3552	-1.4522	
21.0000	-2.0040	-2.0728	-2.1717	
28.0000	-2.6747	-2.7904	-2.9140	
42.0000	-4.3842	-4.2256	-4.4296	

```
plot(regression_result)
```



The model uses a one-sided confidence limit to evaluate the least favorable direction. This example identifies a node with a large Cook's distance, but the function does not automatically delete it; you should go back to the original records, experimental procedure, and protocol requirements, and perform an include/exclude sensitivity analysis when there is sufficient justification.

4.6 Example 4: Regression Analysis Relative to the First Node

4.6.1 Single-Condition Analysis Relative to the First Node

In addition to the test/reference paired regression of Example 3, `mode = "self"` can analyze how a single condition changes over time on its own. The regression example data is still used below, but only the 25C condition is selected; the `observed_bias` at each node is computed relative to that condition's first-node mean:


```

self_regression_result <- ivdtools::stability_regression(
  data = regression_data,
  time = "Day",
  value = "Result",
  condition = "Condition",
  mode = "self",
  test_condition = "25C",
  bias_type = "relative",
  direction = "decrease",
  conf.level = 0.95,
  limit = 5,
  time_unit = "d"
)
print(self_regression_result)

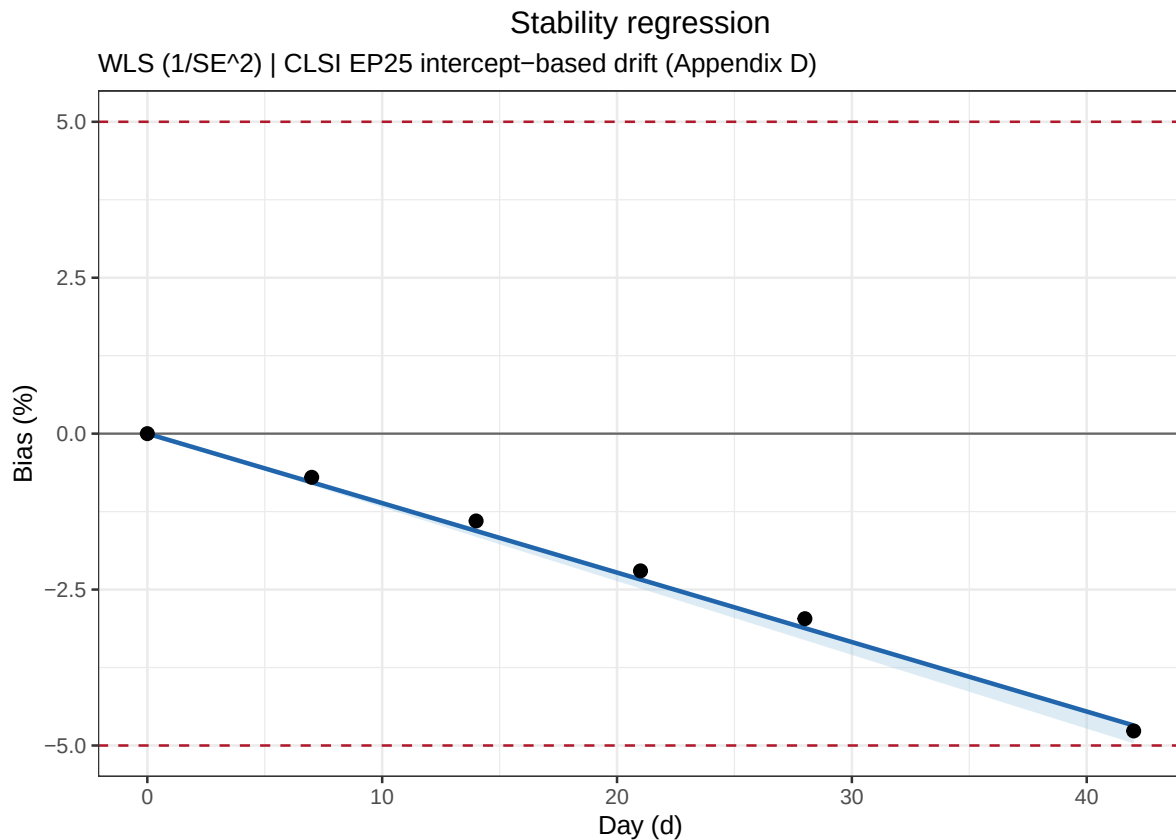
```

Stability Regression

Mode: self
 Test condition: 25C
 Fit: WLS (1/SE²); intercept = 100.0849, slope = -0.1115
 Direction: decrease
 One-sided CI: 95.0%
 R-squared: 0.9966
 Cook distance: row(s) 1, 6 > 1; not removed

Time	ObsBias	Estimate	CI_limit	Status
0.0000	0.0000	-0.0000	-0.0000	
7.0000	-0.7000	-0.7797	-0.8278	
14.0000	-1.4000	-1.5594	-1.6555	
21.0000	-2.2000	-2.3391	-2.4833	
28.0000	-2.9667	-3.1188	-3.3110	
42.0000	-4.7667	-4.6782	-4.9665	

```
plot(self_regression_result)
```



The observed bias at the first node is fixed at 0; the model estimates, however, are fitted jointly from all time points and do not require the regression intercept to pass exactly through the first node. A one-sided confidence limit is used to evaluate the pre-specified least favorable direction; the illustrative $\pm 5\%$ limit still cannot replace the study protocol requirements.

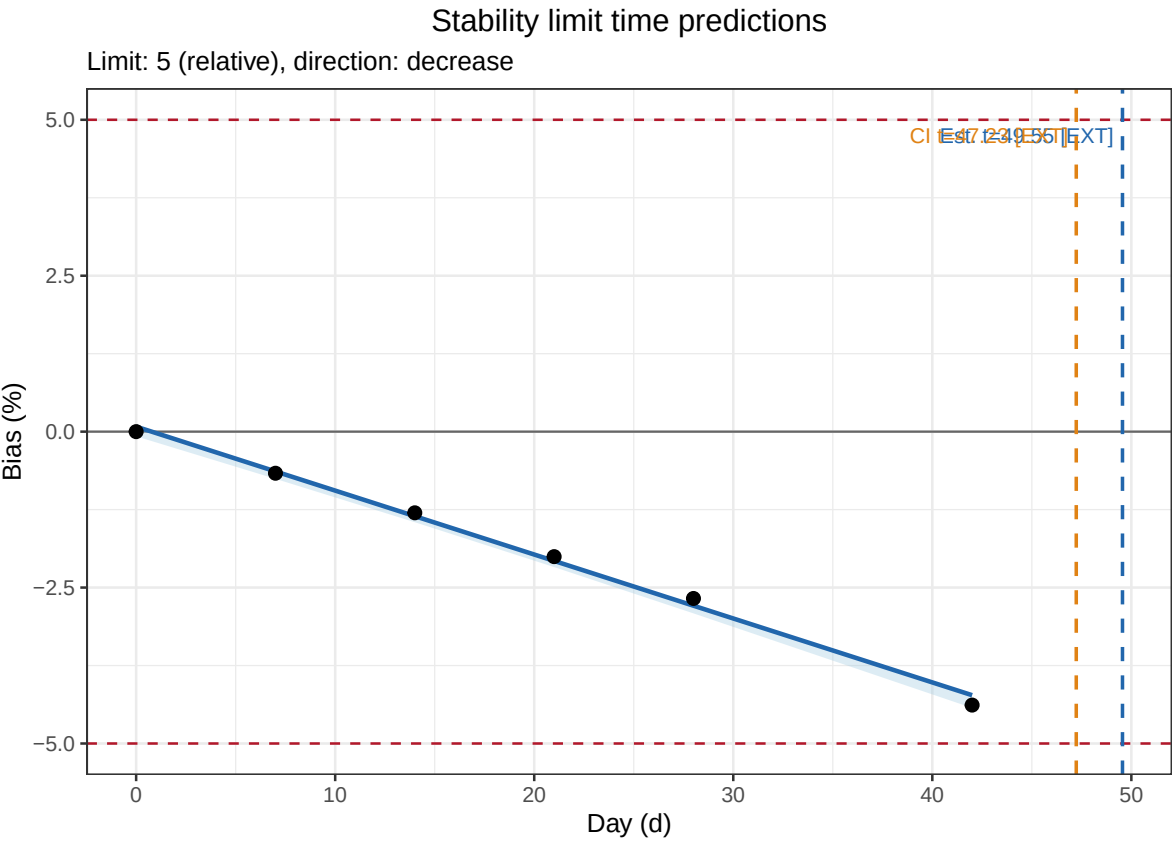
4.7 Example 5: Predicting the Time to Reach a Limit

`stability_time()` uses the regression object from Example 3 to compute the times at which the point estimate and the one-sided confidence limit first reach a -5% bias:

```
time_result <- ivdtools::stability_time(
  regression_result,
  limit = 5,
  max_time = 90
)
print(time_result)
```

```
Stability Limit Time
Limit: 5.0000 (relative); direction: decrease
estimate      49.5548 d [EXTRAPOLATED]
one_sided_ci   47.2316 d [EXTRAPOLATED]
```

```
plot(time_result)
```



In this example, the longest actual observation time is 42 days, while the predicted time to reach the limit exceeds the observation range, so the output is marked **EXTRAPOLATED**. Extrapolated results rely on the assumption that the linear trend continues beyond the unobserved interval and cannot be treated as measured stability evidence; report the extrapolation distance and uncertainty prominently, and prefer confirmation with longer actual observation data.

4.8 Example 6: Arrhenius Accelerated Stability Analysis

4.8.1 Reading and Validating Raw Data

```
arrhenius_data <- read_csv(  
  "./data/stability-arrhenius-example.csv",  
  show_col_types = FALSE  
)  
arrhenius_data
```

A tibble: 24 x 4

	Temperature	Day	Replicate	Result
	<dbl>	<dbl>	<dbl>	<dbl>
1	30	0	1	100.
2	30	0	2	99.8
3	30	7	1	99.6
4	30	7	2	99.4
5	30	14	1	99.1
6	30	14	2	98.9
7	30	21	1	98.5
8	30	21	2	98.3
9	37	0	1	100.
10	37	0	2	99.9

i 14 more rows

4.8.2 Kinetic and Arrhenius Fitting

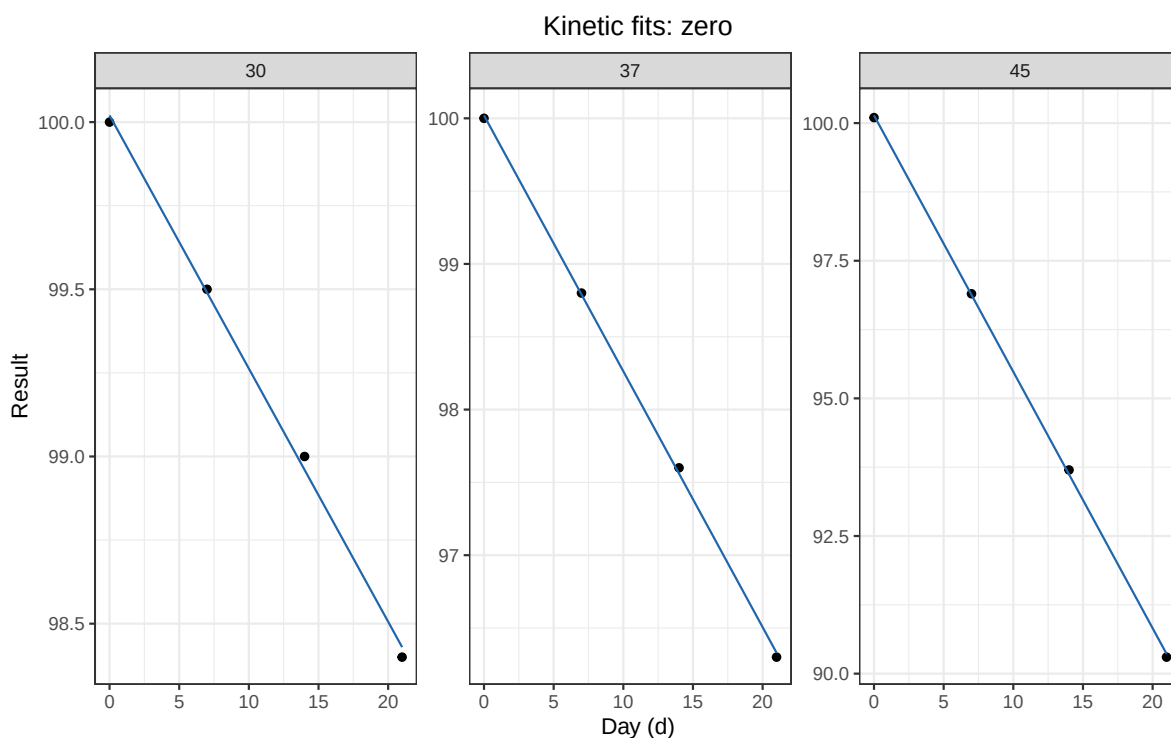
The example uses accelerated data at 30, 37, and 45 °C, extrapolated to 5 °C; allows a relative change of 10%, and pre-specifies the degradation direction as decreasing:

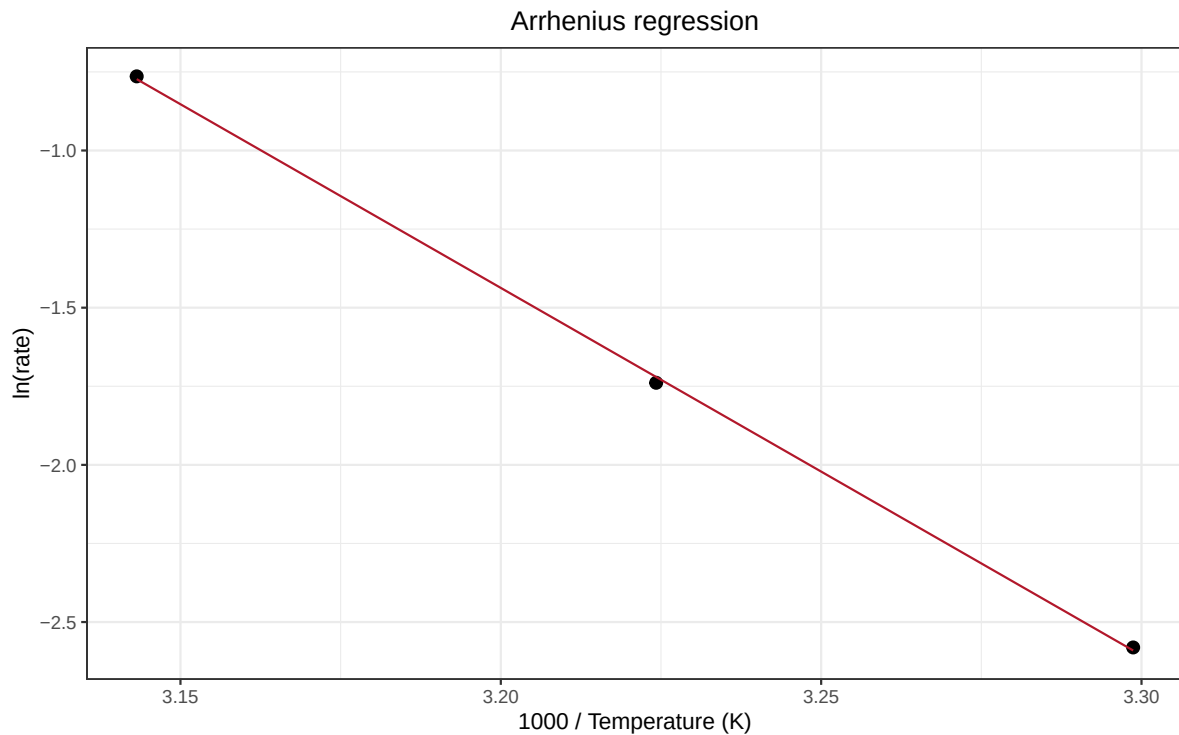
```
arrhenius_result <- ivdtools::arrhenius(
  data = arrhenius_data,
  temperature = "Temperature",
  time = "Day",
  value = "Result",
  target_temp = 5,
  order = "auto",
  direction = "decrease",
  limit = 10,
  bias_type = "relative",
  temp_unit = "C",
  time_unit = "d",
  conf.level = 0.95
)
print(arrhenius_result)
```

Arrhenius Stability Analysis

```
Kinetic order:  zero (auto-selected)
Direction:      decrease
Activation Ea:  97.159 kJ/mol
Target rate:    0.002346 /d
Limit time:     4263.7878 d [1585.3589, 11467.3633]
```

```
plot(arrhenius_result)
```





`order = "auto"` selects a model from the zero-order and first-order kinetic candidates according to the implementation rules. Formal analysis should determine the candidate orders under experimental design and mechanistic knowledge, and verify the degradation direction, rate significance, extent of degradation, and model residuals at each temperature. Extrapolating from elevated temperatures to the target storage temperature may span a large range; the predicted shelf life and its interval must be interpreted together with the model assumptions, the actual time data, and validation studies.

4.9 Example 7: MKT Computation

4.9.1 Reading and Validating Raw Temperature Data

```
mkt_data <- read_csv(
  "./data/stability-mkt-example.csv",
  show_col_types = FALSE
)
mkt_data$Timestamp <- as.POSIXct(
  mkt_data$Timestamp,
  format = "%Y-%m-%d %H:%M:%S",
  tz = "UTC"
)
mkt_data
```

```
# A tibble: 8 x 3
  Timestamp          Probe_A Probe_B
<dtm>          <dbl>   <dbl>
1 2026-07-01 00:00:00      25    24.5
2 2026-07-01 06:00:00      26    25.5
3 2026-07-01 12:00:00      29    28.5
4 2026-07-01 18:00:00      28    27.5
5 2026-07-02 00:00:00      25    24.8
6 2026-07-02 08:00:00      24    24.2
7 2026-07-02 16:00:00      27    26.5
8 2026-07-03 00:00:00      26    25.8
```

4.9.2 Time-Weighted MKT

Using two temperature probes and the actual time intervals, with temperature in degrees Celsius and an example activation energy of 83.144 kJ/mol:

```
mkt_result <- ivdtools::mkt(
  data = mkt_data,
  temp_cols = c("Probe_A", "Probe_B"),
  time = "Timestamp",
  temp_unit = "C",
  ea = 83.144
)
print(mkt_result)
```

Mean Kinetic Temperature

Ea: 83.144 kJ/mol

Method: Time-weighted trapezoidal MKT

Probe_A	299.538 K	26.388 C
Probe_B	299.194 K	26.044 C
Combined	299.367 K	26.217 C

MKT is not a simple arithmetic mean of temperature; high-temperature exposure receives greater weight through the Arrhenius relationship. Formal computations should confirm probe calibration status, time ordering, missing intervals, temperature units, exposure duration, and the activation energy assumption. MKT can only summarize temperature exposure; it cannot replace product-specific stability studies.

4.10 Splitting Multi-Sample Data

When different concentration levels, sample types, or lots need to be evaluated separately, you should retain the sample identifier, split the data first, and then analyze each sample with the same pre-specified parameters. Read and validate the multi-sample raw data:

```
multiple_samples_data <- read_csv(
  "./data/stability-multiple-samples-example.csv",
  show_col_types = FALSE
)
multiple_samples_data
```

A tibble: 36 x 4

	Sample	Day	Replicate	Result
	<chr>	<dbl>	<dbl>	<dbl>
1	Sample_A	0	1	100.
2	Sample_A	0	2	99.8
3	Sample_A	0	3	100
4	Sample_A	7	1	99.7
5	Sample_A	7	2	99.5
6	Sample_A	7	3	99.8
7	Sample_A	14	1	99.2
8	Sample_A	14	2	99.4
9	Sample_A	14	3	99.1
10	Sample_A	21	1	98.7

i 26 more rows

Use `split()` to build a list of data named by sample, and first look at one of the samples:

```
sample_data_list <- split(
  multiple_samples_data,
  multiple_samples_data$Sample
)
names(sample_data_list)
```

```
[1] "Sample_A" "Sample_B"
```

```
sample_a_data <- sample_data_list[["Sample_A"]]
sample_a_data
```

```
# A tibble: 18 x 4
  Sample      Day Replicate Result
<chr>    <dbl>     <dbl>  <dbl>
1 Sample_A      0         1  100.
2 Sample_A      0         2  99.8
3 Sample_A      0         3  100.
4 Sample_A      7         1  99.7
5 Sample_A      7         2  99.5
6 Sample_A      7         3  99.8
7 Sample_A     14         1  99.2
8 Sample_A     14         2  99.4
9 Sample_A     14         3  99.1
10 Sample_A     21         1  98.7
11 Sample_A     21         2  98.5
12 Sample_A     21         3  98.8
13 Sample_A     28         1   98.
14 Sample_A     28         2  98.3
15 Sample_A     28         3  98.1
16 Sample_A     42         1  96.1
17 Sample_A     42         2  96.4
18 Sample_A     42         3  96.2
```

Perform the analysis on each sample in turn.

4.11 Key Points for Interpreting Results

1. **Protocol first:** Before analysis, specify conditions, nodes, replicates, time units, direction, bias scale, and acceptance limits.
2. **Raw data read-only:** Report missing, duplicate, non-numeric records, and every excluded row at each step; do not automatically delete anomalous records.
3. **Correct pairing:** Condition comparisons must be made at matched time points, with the test and reference conditions explicitly identified.
4. **Model diagnostics:** Report slope, confidence limits, fitting method, residuals, influential points, and whether a WLS fallback occurred.
5. **Prominent extrapolation:** Predictions of stability time beyond the observed range should be clearly labeled and not treated as measured evidence.
6. **Explicit temperature assumptions:** Arrhenius analysis and MKT must record Celsius/Kelvin units, time units, and activation energy.
7. **Separate statistics from acceptability:** Statistical significance is not the same as product acceptability; judgments can only be based on pre-specified allowable limits.
8. **Independent review:** Clinical, regulatory, or release decisions should be independently reviewed by appropriate professionals.

Chapter 5

Precision Analysis

5.1 Analysis Objectives

Precision evaluates the degree of agreement among results measured on the same sample by the same measurement system under repeatable conditions, usually expressed as the standard deviation (SD) or coefficient of variation (CV%). CLSI EP05 subdivides precision into variance components by condition, such as within-run, between-run, and between-day, and provides estimates, confidence intervals, and acceptance limits for judging each component.

This document demonstrates how to use the `precision()` function in the `ivdtools` package to establish a precision variance-component analysis, complete outlier checking, normality assessment, estimation of variance components and confidence intervals, Sadler precision profile fitting, and rearrangement of the results into an EP05-style variance component table. The example data are deterministic teaching data used only to demonstrate the analysis workflow; formal studies must use the experimental design, acceptance criteria, and analysis parameters pre-specified in the protocol or standard.

```
library(ivdtools)
library(readr)
```

Data format requirement: `precision()` ultimately calls `VCA::anovaVCA()`, which requires a plain `data.frame`. Since `readr::read_csv()` reads in a tibble, this document consistently converts to a `data.frame` first, then converts the experimental factor columns (day, run, replicate, etc.) to factors.

5.2 Overview of Functions

Function	Main purpose	Key input or output
<code>precision()</code>	Create a precision variance-component analysis object	Nested formula, by grouping, balance check
<code>outlier()</code>	Per-sample outlier detection	Grubbs or IQR method
<code>normal()</code>	Per-sample normality test	Shapiro-Wilk etc., level controls the confidence level
<code>variance()</code> / <code>vc()</code>	Estimate variance components	VC, %Total, SD, CV%, NegVC passed here
<code>ci()</code>	Confidence intervals for SD/CV of each component	Satterthwaite approximation
<code>profile()</code>	Sadler precision profile fitting	10 candidate models, AIC selection
<code>list_sadler()</code>	View the 10 Sadler models	Model formula and type

<code>summary()</code> / <code>plot()</code>	Summarize and plot	<code>plot(type=...)</code> selects the graphic
--	--------------------	---

After `precision()` creates the object, analysis results accumulate into the object step by step through `outlier()`, `normal()`, `variance()`, `ci()`, and `profile()`, and **must be reassigned**:

```
precision() -> outlier() -> normal() -> variance() -> ci() -> profile()
```

Only analyses that have been executed appear in `summary()`, and some `plot()` graphics depend on prior analyses.

5.3 Example 1: EP05 20×2×2 Single-Sample Precision

5.3.1 Reading and Validating Data

20×2×2 means 20 days, 2 runs per day, and 2 replicates per run, for a total of 80 results; it is one of the classic EP05 designs:

```
ep05 <- read_csv("./data/precision-ep05.csv", show_col_types = FALSE)
ep05 <- as.data.frame(ep05)
str(ep05)
```

```
'data.frame':  80 obs. of  4 variables:
 $ day  : num  1 1 1 1 2 2 2 2 3 3 ...
 $ run  : num  1 1 2 2 1 1 2 2 1 1 ...
 $ rep  : num  1 2 1 2 1 2 1 2 1 2 ...
 $ value: num  96.9 93.5 99.3 101.5 103 ...
```

The experimental factor columns are integers in the CSV and are converted to factors after reading:

```
ep05$day <- factor(ep05$day)
ep05$run <- factor(ep05$run)
ep05$rep <- factor(ep05$rep)
```

```
dim(ep05)
```

```
[1] 80  4
```

```
summary(ep05$value)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
86.06   94.83   99.33   99.11  103.53  115.01
```

`value ~ day/run` means the variance components are nested layer by layer as between-day (day) -> within-day run (run). / is the nesting notation, equivalent to treating each deeper factor as a factor nested within the factor above it.

5.3.2 Creating the Analysis Object and Pre-Check

```
p <- precision(ep05, value ~ day/run)
p
```

Precision analysis object

```
Data class      : data.frame
Total rows      : 80
Formula         : value ~ day/run
```

```

Samples                : 1

Factor levels
  day      : 1 (4), 2 (4), 3 (4), 4 (4), 5 (4), 6 (4), 7 (4), 8 (4), 9 (4), 10 (4), 11 (4), 12
  run      : 1 (40), 2 (40)
  rep      : 1 (40), 2 (40)

```

Analysis status

```

[ ] outlier
[ ] normal
[ ] variance
[ ] ci
[ ] profile

```

The balance check output reports the ratio of the largest to smallest cell size; when the data are highly unbalanced, VCA will suggest switching to REML.

Perform per-sample outlier detection and normality testing:

```
p <- outlier(p, method = "grubbs")
```

Outlier detection -- Grubbs (alpha = 0.05)
No outliers detected.

```
p <- normal(p, method = "shapiro")
```

Normality test -- Shapiro-Wilk

Sample	n	W	p-value
all	80	0.9868	0.5858

Parameter notes: The method of `outlier()` can be "grubbs" (default) or "iqr", and `alpha` is the Grubbs significance level. For `normal()`, `method="auto"` selects the test based on sample size (Shapiro-Wilk for $n \leq 50$, Anderson-Darling for larger samples); when `method="shapiro"` is specified, the reported `W` column is the true Shapiro-Wilk `W` statistic. For single-sample data with $n=80$, either approach is acceptable; this example specifies it explicitly for illustration.

5.3.3 Variance Components and Confidence Intervals

```
p <- variance(p)
```

Variance components

sample	component	VC	%Total	SD	CV[%]
all	day	18.8358	42.6	4.3400	4.3788
all	day:run	17.4343	39.4	4.1754	4.2128
all	error	7.9482	18.0	2.8193	2.8445

The `NegVC` parameter needs to be passed at `variance()`:

```
p <- variance(p, NegVC = TRUE) # allow negative variance components (default FALSE)
```

```
p <- ci(p)
```

Confidence intervals (SD)

sample	component	estimate	lower	upper
--------	-----------	----------	-------	-------

all	total	6.6497	5.4288	8.5842
all	day	4.3400	0.0000	6.2255
all	day:run	4.1754	2.0129	5.5513
all	error	2.8193	2.3147	3.6073

Confidence intervals (%CV)				
sample	component	estimate	lower	upper

all	total	6.7091	5.4773	8.6609
all	day	4.3788	0.0000	6.2812
all	day:run	4.2128	2.0309	5.6009
all	error	2.8445	2.3353	3.6395

`ci()` uses the Satterthwaite effective-degrees-of-freedom approximation to compute confidence intervals for the SD and CV of each component.

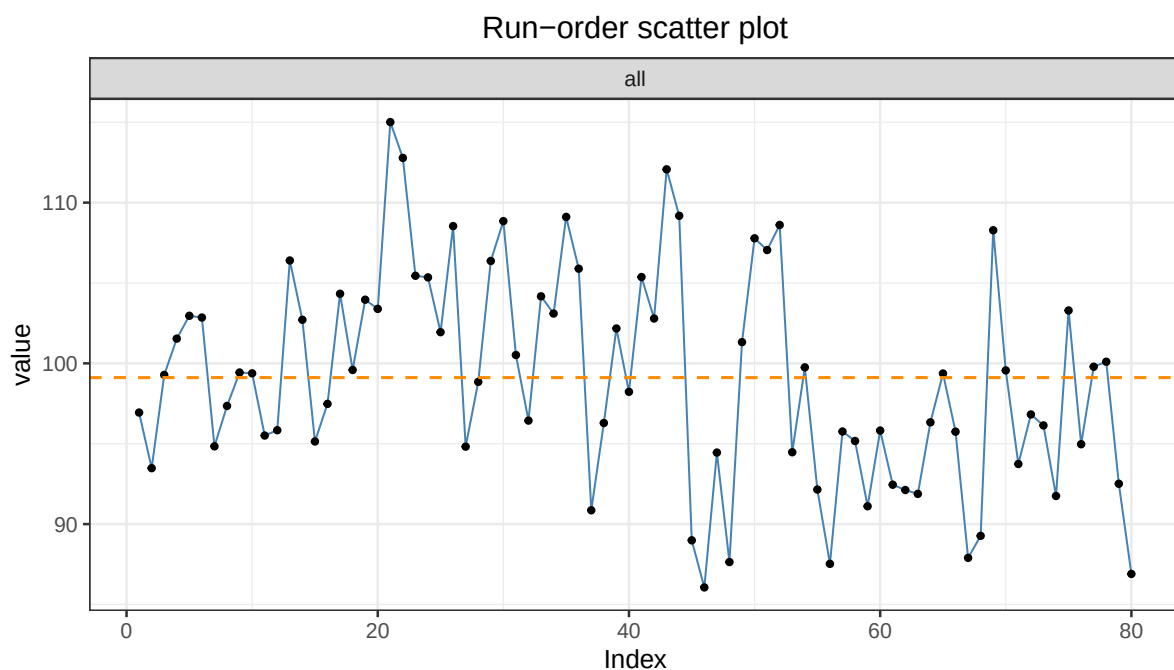
5.3.4 Converting to an EP05 Terminology Table

Classify the variance components into within-run, between-run, and between-day by the nesting depth of the component name, compute the total SD/CV, and further merge the component confidence intervals to form a complete EP05 report table with intervals. **Whether the statistical results are acceptable must be judged against the precision limits pre-specified in the protocol.**

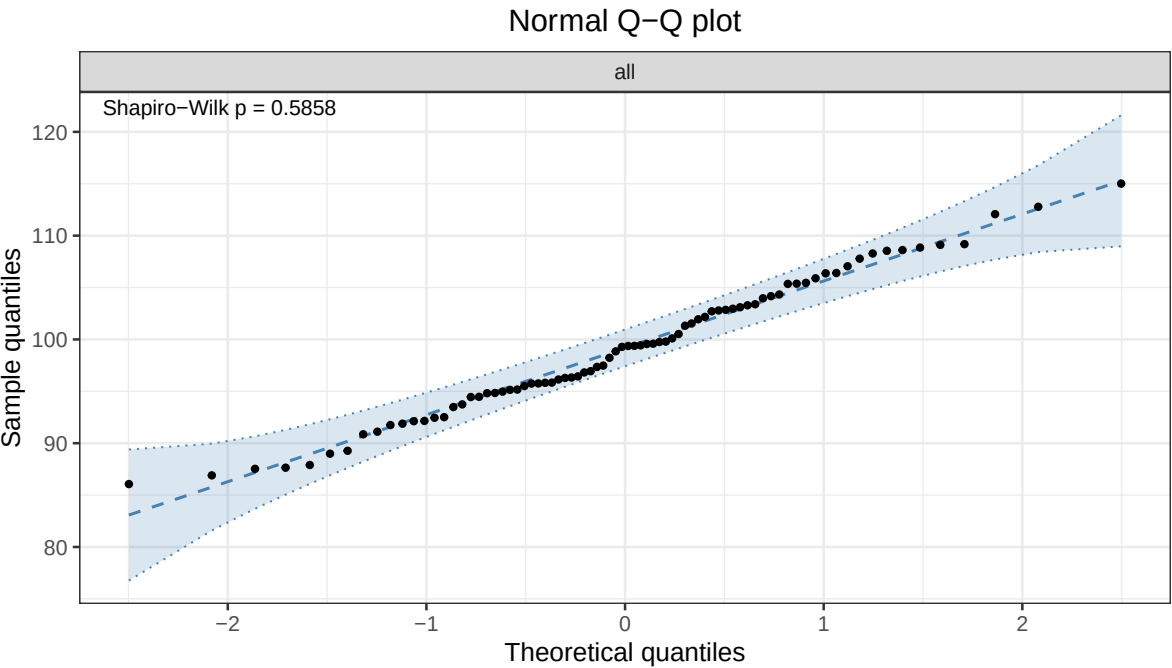
5.3.5 Graphics

A scatter plot can be used to quickly identify test anomalies and the distribution of measurements:

```
plot(p, type = "dot")
```



```
plot(p, type = "qq")
```



Graphic dependencies: `plot(type="dot")` is a scatter plot by run order (no prior analysis needed), `type="his"` is a standard histogram; `type="var"` requires `variance()` to have been run, `type="qq"` requires `normal()`, and `type="profile"` requires `profile()`. Confirm that the corresponding analysis has been run before plotting.

```
summary(p)
```

Precision analysis -- summary

Precision analysis object
Data class : data.frame
Total rows : 80
Formula : value ~ day/run
Samples : 1

Factor levels
day : 1 (4), 2 (4), 3 (4), 4 (4), 5 (4), 6 (4), 7 (4), 8 (4), 9 (4), 10 (4), 11 (4), 12
run : 1 (40), 2 (40)
rep : 1 (40), 2 (40)

Analysis status

- [x] outlier
- [x] normal
- [x] variance
- [x] ci
- [] profile

Outlier detection -- Grubbs (alpha = 0.05)
No outliers detected.

Normality test -- Shapiro-Wilk

Sample	n	W	p-value
all	80	0.9868	0.5858

Variance components					
sample component		VC	%Total	SD	CV[%]
all	day	18.8358	42.6	4.3400	4.3788
all	day:run	17.4343	39.4	4.1754	4.2128
all	error	7.9482	18.0	2.8193	2.8445

Confidence intervals (SD)				
sample component		estimate	lower	upper
all	total	6.6497	5.4288	8.5842
all	day	4.3400	0.0000	6.2255
all	day:run	4.1754	2.0129	5.5513
all	error	2.8193	2.3147	3.6073

Confidence intervals (%CV)				
sample component		estimate	lower	upper
all	total	6.7091	5.4773	8.6609
all	day	4.3788	0.0000	6.2812
all	day:run	4.2128	2.0309	5.6009
all	error	2.8445	2.3353	3.6395

5.4 Example 2: Multi-Sample 3×5 and Sadler Profile

5.4.1 Reading Data

When multiple concentration levels need to be evaluated, use `by` to specify the sample grouping column, and estimate the variance components separately for each sample:

```
profile_data <- as.data.frame(read_csv(
  "./data/precision-profile.csv",
  show_col_types = FALSE
))
profile_data$sample <- factor(profile_data$sample)
profile_data$day <- factor(profile_data$day)
profile_data$rep <- factor(profile_data$rep)
```

5.4.2 Per-Sample Variance Components

```
p2 <- precision(profile_data, value ~ day, by = "sample")
p2 <- variance(p2)
```

Variance components					
sample component		VC	%Total	SD	CV[%]
L1	day	0.6376	63.0	0.7985	3.9446
L1	error	0.3750	37.0	0.6124	3.0250
L2	day	5.7139	75.1	2.3904	4.7584
L2	error	1.8937	24.9	1.3761	2.7393
L3	day	3.3913	15.7	1.8415	1.8088
L3	error	18.1466	84.3	4.2599	4.1840
L4	day	30.5549	54.6	5.5276	2.7529
L4	error	25.4525	45.4	5.0451	2.5126
L5	day	0.0000	0.0	0.0000	0.0000
L5	error	171.7986	100.0	13.1072	3.3003

```
p2 <- ci(p2)
```

Confidence intervals (SD)

sample	component	estimate	lower	upper

L1	total	1.0063	0.6007	2.9368
L1	day	0.7985	0.0000	1.4267
L1	error	0.6124	0.4391	1.0108
L2	total	2.7582	1.5714	9.9889
L2	day	2.3904	0.0000	4.2023
L2	error	1.3761	0.9868	2.2716
L3	total	4.6409	3.2872	7.8819
L3	day	1.8415	0.0000	4.1779
L3	error	4.2599	3.0547	7.0319
L4	total	7.4838	4.6166	19.1652
L4	day	5.5276	0.0000	10.0268
L4	error	5.0451	3.6177	8.3280
L5	total	13.1072	9.5629	20.8255
L5	day	0.0000	NA	NA
L5	error	13.1072	9.3990	21.6365

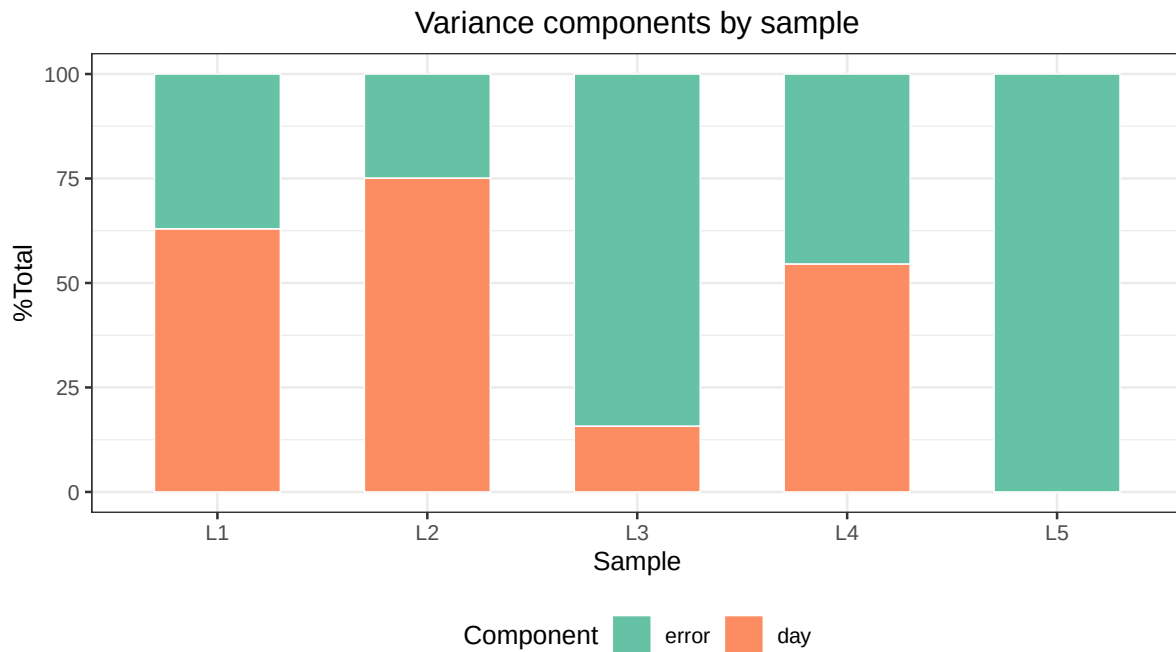
Confidence intervals (%CV)

sample	component	estimate	lower	upper

L1	total	4.9710	2.9673	14.5079
L1	day	3.9446	0.0000	7.0480
L1	error	3.0250	2.1692	4.9936
L2	total	5.4905	3.1281	19.8843
L2	day	4.7584	0.0000	8.3652
L2	error	2.7393	1.9643	4.5219
L3	total	4.5583	3.2287	7.7416
L3	day	1.8088	0.0000	4.1035
L3	error	4.1840	3.0003	6.9067
L4	total	3.7271	2.2992	9.5447
L4	day	2.7529	0.0000	4.9936
L4	error	2.5126	1.8017	4.1476
L5	total	3.3003	2.4078	5.2436
L5	day	0.0000	NA	NA
L5	error	3.3003	2.3666	5.4478

Comparing the proportion of variability across samples can be used for product optimization and testing scheme design:

```
plot(p2, type = "var")
```



5.4.3 Precision Profile

`profile()` fits the Sadler model family to each variance component and selects the best model by AIC:

```
list_sadler()
```

Sadler Precision Profile Models

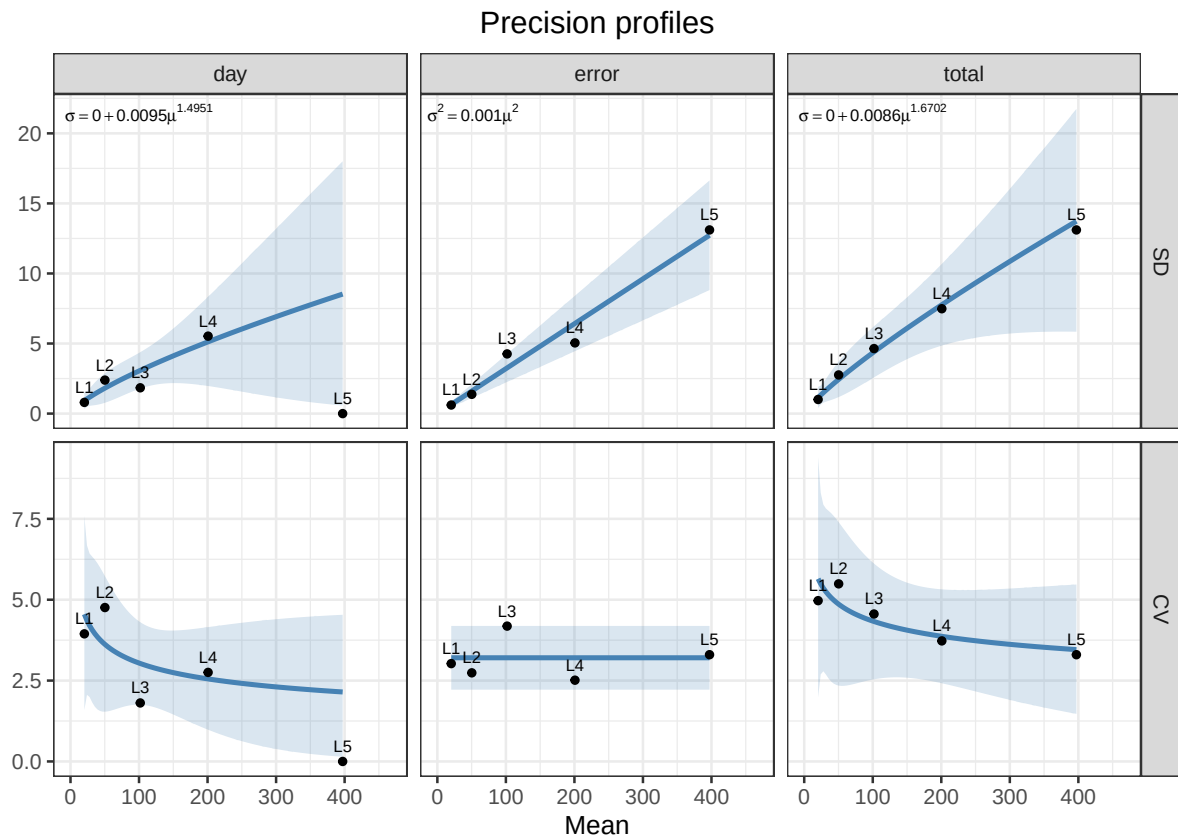
1	Constant SD	$\sigma^2 = b_1$
2	Constant CV	$\sigma^2 = b_1 * \mu^2$
3	Linear (variance)	$\sigma^2 = b_1 + b_2 * \mu$
4	Power (fixed K)	$\sigma^2 = b_1 * \mu^K$
5	Power (fixed K, +intercept)	$\sigma^2 = b_1 + b_2 * \mu^K$
6	Linear (SD)	$\sigma = b_1 + b_2 * \mu$
7	Power (SD)	$\sigma = b_1 + b_2 * \mu^{b_3}$
8	Power (variance)	$\sigma^2 = b_1 + b_2 * \mu^{b_3}$
9	Exponential (log-log)	$\log(\sigma^2) = b_1 + b_2 * \log(\mu)$
10	Power (full)	$\sigma^2 = b_1 * \mu^{b_2}$

```
p2 <- profile(p2, model.no = 1:10)
```

Precision profiles -- Sadler

```
total: sigma = 0.0000 + 0.0086 * mu^1.6702 (AIC = 56.45, R^2 = 0.9847)
day: sigma = 0.0000 + 0.0095 * mu^1.4951 (AIC = 43.46, R^2 = 0.8919)
error: sigma^2 = 0.0010 * mu^2 (AIC = 55.96, R^2 = 0.9806)
```

```
plot(p2, type = "profile")
```

Parameter notes: `profile(model.no = 1:10)` specifies the candidate model numbers; ... can be passed to the internal Sadler fit (for example K). Each variance component requires at least 3 samples with a finite SD to be fitted, so a multi-sample design should include at least 3 concentration levels. The profile can be used, for example, in functional-sensitivity analyses to predict the SD/CV at any concentration (see Functional Sensitivity).

In this example, the best models for the total and day components are power-function type (model 7), and the day:rep component is a constant-CV type (model 2), consistent with the common setting where “SD increases with concentration while CV is relatively stable”.

5.5 Example 3: Multi-Sample Multi-Site (Site) Precision

5.5.1 Reading Data

The data contain 2 sites, 3 samples, and per site per sample 3 days $\times$ 3 runs $\times$ 5 replicates:

```
sites_data <- as.data.frame(read_csv(
  "./data/precision-sites.csv",
  show_col_types = FALSE
))
sites_data[] <- lapply(sites_data, function(x)
  if (is.numeric(x) && all(x == round(x))) factor(x) else x)
str(sites_data)
```

```
'data.frame': 270 obs. of 6 variables:
 $ site : chr "A" "A" "A" "A" ...
 $ sample: chr "S1" "S1" "S1" "S1" ...
 $ day : Factor w/ 3 levels "1","2","3": 1 1 1 1 1 1 1 1 1 1 ...
 $ run : Factor w/ 3 levels "1","2","3": 1 1 1 1 1 2 2 2 2 2 ...
 $ rep : Factor w/ 5 levels "1","2","3","4",...: 1 2 3 4 5 1 2 3 4 5 ...
 $ value : num 49.2 52.3 48.8 49.4 51.5 ...
```

5.5.2 Multi-Column Grouping

by can accept multiple column names and groups internally by the interaction, obtaining all site × sample combinations at once:

```
p3 <- precision(sites_data, value ~ day/run, by = c("sample", "site"))
p3 <- variance(p3)
```

Variance components

sample	component	VC	%Total	SD	CV[%]
S1.A	day	0.0000	0.0	0.0000	0.0000
S1.A	day:run	0.1646	6.8	0.4057	0.8231
S1.A	error	2.2677	93.2	1.5059	3.0553
S2.A	day	0.3241	4.3	0.5693	0.5602
S2.A	day:run	0.1523	2.0	0.3903	0.3840
S2.A	error	7.0483	93.7	2.6549	2.6124
S3.A	day	4.4871	15.6	2.1183	1.0420
S3.A	day:run	1.7006	5.9	1.3041	0.6415
S3.A	error	22.5797	78.5	4.7518	2.3375
S1.B	day	0.0000	0.0	0.0000	0.0000
S1.B	day:run	0.5459	23.5	0.7389	1.4758
S1.B	error	1.7784	76.5	1.3335	2.6637
S2.B	day	0.0000	0.0	0.0000	0.0000
S2.B	day:run	1.6690	21.5	1.2919	1.2891
S2.B	error	6.0856	78.5	2.4669	2.4616
S3.B	day	0.7633	2.3	0.8737	0.4448
S3.B	day:run	0.0000	0.0	0.0000	0.0000
S3.B	error	31.8033	97.7	5.6394	2.8709

```
names(p3$results)
```

```
[1] "S1.A" "S2.A" "S3.A" "S1.B" "S2.B" "S3.B"
```

5.5.3 Splitting by Site for Separate Analysis

An equivalent approach is to first `split()` by site and then run the same analysis parameters on each site subset. When the variance-component structure is expected to differ between sites, or separate reports are needed, splitting and analyzing separately is clearer:

```
for (s in c("A", "B")) {
  sub <- sites_data[sites_data$site == s, ]
  ps <- precision(sub, value ~ day/run, by = "sample")
  ps <- variance(ps)
  ps <- ci(ps)
}
```

Note: Multi-center studies should unify the analysis plan, variance-component model, and reporting convention; when “reporting per center first and then combining”, be sure to record whether each center’s raw data were included, and whether the exclusion criteria and factor definitions are consistent.

5.6 Example 4: Custom More Complex Models

5.6.1 Reading Data

Besides the standard day/run/replicate nesting, the `precision()` formula can be any VCA nested structure, for example adding an operator level: `value ~ operator/day/run`, i.e., nested layer by

layer as operator -> day -> run:

```
custom_data <- as.data.frame(read_csv(
  "./data/precision-custom.csv",
  show_col_types = FALSE
))
custom_data[] <- lapply(custom_data, function(x)
  if (is.numeric(x) && all(x == round(x))) factor(x) else x)
```

5.6.2 Fitting and Interpretation

```
p4 <- precision(custom_data, value ~ operator/day/run)
p4 <- variance(p4)
```

Variance components

sample	component	VC	%Total	SD	CV[%]
all	operator	2.2914	11.8	1.5137	1.4898
all	operator:day	6.3159	32.4	2.5132	2.4734
all	operator:day:run	3.2463	16.7	1.8017	1.7733
all	error	7.6322	39.2	2.7627	2.7190

5.7 Key Points for Interpreting Results

1. **Design match:** First clarify the nesting structure of the study design; the `precision()` formula must faithfully reflect the experimental hierarchy.
2. **Data format:** After `read_csv`, convert to `data.frame` and explicitly convert factor columns to factor to avoid VCA errors or ambiguity.
3. **Call order:** outlier → normal → variance → ci → profile, reassign after each analysis; plotting depends on prior analyses.
4. **Negative components:** When a factor has too few levels or the data happen to be nearly identical, a variance component may be set to zero; interpret carefully and specify the handling approach in the protocol.
5. **CI applicability:** The Satterthwaite approximation becomes less accurate with very few variance components or unbalanced data; note the method in the report.
6. **Terminology table:** Rearrange the numeric quantities in `$results` (`sd_comp/cv_comp/vc`) into an EP05 terminology table; do not directly use the display-oriented string `$vc`.
7. **Acceptability judgment:** Statistical estimates do not automatically constitute a pass/fail determination; they should be compared with pre-specified precision limits and reviewed by professionals.

Chapter 6

Linearity Analysis

6.1 Analysis Objectives

Linearity evaluates whether there is an acceptable linear relationship between the measured result and the analyte concentration (or target value) throughout the entire working range of a measurement system, usually using dilution data and analyzing bias per CLSI EP06. This document demonstrates how to use the `fit_equation()` function in the `ivdtools` package to complete linear fitting, weighted fitting, evaluation of bias and residuals at each level, and EP06-A2-style polynomial analysis to determine whether higher-order terms are needed.

The example data are deterministic teaching data used only to demonstrate the workflow; formal studies should use the dilution scheme, concentration levels, replicate counts, and acceptance criteria pre-specified in the protocol.

```
library(ivdtools)
library(readr)
```

6.2 Overview of Functions

Function	Main purpose	Key input or output
<code>list_equation()</code>	View the built-in equation registry	<code>category</code> , <code>engine</code> filters
<code>replicate_to_mean()</code>	Summarize replicates and optionally compute weights	"n", "1/sd", "1/sd^2"
<code>fit_equation()</code>	Fit built-in or custom equations	<code>eq</code> , <code>weights</code> , <code>lower/upper</code> , <code>constraints</code>
<code>compare_equation()</code>	Fit multiple candidate equations and rank by AIC	<code>eqs</code> , <code>deltaAIC</code>
<code>coef()</code> / <code>residuals()</code>	Extract parameters and residuals	
<code>predict()</code>	Forward prediction or inverse back-calculation of concentration	<code>inverse=TRUE</code> , <code>interval</code>
<code>plot()</code>	Plot fitted curve and intervals	<code>interval</code> , <code>frame</code>

6.3 Example 1: Linearity Bias of a Serial Dilution

6.3.1 Reading and Validating Data

The dilution series contains 8 concentration levels with 3 replicates each, for a total of 24 rows:

```
dilution <- read_csv("./data/linearity-dilution.csv", show_col_types = FALSE)
dilution <- as.data.frame(dilution)
head(dilution, 6)
```

```
  conc replicate signal
1   50          1  48.6
2   50          2  49.2
3   50          3  50.0
4  100          1 101.3
5  100          2  99.7
6  100          3 101.4
```

```
dim(dilution)
```

```
[1] 24  3
```

```
summary(dilution)
```

```
      conc      replicate      signal
Min.   : 50   Min.   :1   Min.   : 48.6
1st Qu.: 175   1st Qu.:1   1st Qu.: 173.6
Median : 600   Median :2   Median : 595.6
Mean   :1594   Mean   :2   Mean   :1593.0
3rd Qu.:2000   3rd Qu.:3   3rd Qu.:2004.5
Max.   :6400   Max.   :3   Max.   :6406.9
```

6.3.2 Viewing Available Response Equations

```
list_equation(category = "Response")
```

ID	Name	Formula	Params	nP	Engine
E01	Linear	$a + b \cdot x$	a, b	2	lm
E02	Quadratic	$a + b \cdot x + c \cdot x^2$	a, b, c	3	lm
E03	ExpoGrowth	$a \cdot \exp(b \cdot x)$	a, b	2	nls
E04	ExpoDecay	$a \cdot \exp(-b \cdot x)$	a, b	2	nls
E05	Power	$a \cdot x^b$	a, b	2	nls
E06	Log	$a + b \cdot \log(x)$	a, b	2	lm
E07	4PLC	$A + (B - A) / (1 + (x / C)^D)$	A, B, C, D	4	nls
E08	5PLC	$A + (B - A) / (1 + (x / C)^D)^E$	A, B, C, D, E	5	nls
E09	Gaussian	$a \cdot \exp(-(x - b)^2 / (2 \cdot c^2))$	a, b, c	3	nls
E10	Michaelis	$V_{\max} \cdot x / (K_m + x)$	Vmax, Km	2	nls
E11	Sinusoidal	$a \cdot \sin(b \cdot x + c) + d$	a, b, c, d	4	nls
E12	Cubic	$a + b \cdot x + c \cdot x^2 + d \cdot x^3$	a, b, c, d	4	lm

The linear equation corresponds to registry ID E01 (the name "Linear" is also accepted).

6.3.3 Summarizing Replicate Measurements

First use `replicate_to_mean()` to obtain the mean, SD, and replicate count at each level.

```
rep_sum <- replicate_to_mean(dilution, x = "conc", y = "signal")
rep_sum
```

```
  x    y_mean    y_sd y_n
1  50  49.26667  0.7023769  3
2 100 100.80000  0.9539392  3
```

```

3 200 202.66667 4.4523402 3
4 400 397.83333 7.3961702 3
5 800 798.86667 10.2617412 3
6 1600 1596.90000 13.7535450 3
7 3200 3207.80000 20.9408691 3
8 6400 6389.56667 19.9234368 3

```

6.3.4 Linear Fitting and Bias

```

fit_lin <- fit_equation("E01", data = rep_sum, x = "x", y = "y_mean")
print(fit_lin)

```

E01 (Linear)

Formula: a + b*x

Call:

```
stats::lm(formula = frm, data = d, weights = weights)
```

Residuals:

Min	1Q	Median	3Q	Max
-4.330	-2.404	-1.439	0.430	10.364

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
a	0.975290	2.205878722	0.44213	0.67388
b	0.998894	0.000844269	1183.14720	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.944 on 6 degrees of freedom

Multiple R-squared: 1.0000, Adjusted R-squared: 1.0000

F-statistic: 1.39984e+06 on 1 and 6 DF, p-value: <2e-16

Compute the original residual (the linearity bias, i.e., observed mean minus predicted) and the recovery bias (observed mean minus theoretical value):

```

predict <- predict(fit_lin)

```

x	y
50	50.91999
100	100.86468
200	200.75408
400	400.53286
800	800.09044
1600	1599.20559
3200	3197.43588
6400	6393.89648

```

lin_diag <- data.frame(
  conc = rep_sum$x,
  signal = rep_sum$y_mean,
  predict = predict$y,
  Residual = residuals(fit_lin),
  Recover_bias = rep_sum$y_mean - rep_sum$x
)
print(lin_diag)

```

conc	signal	predict	Residual	Recover_bias
------	--------	---------	----------	--------------

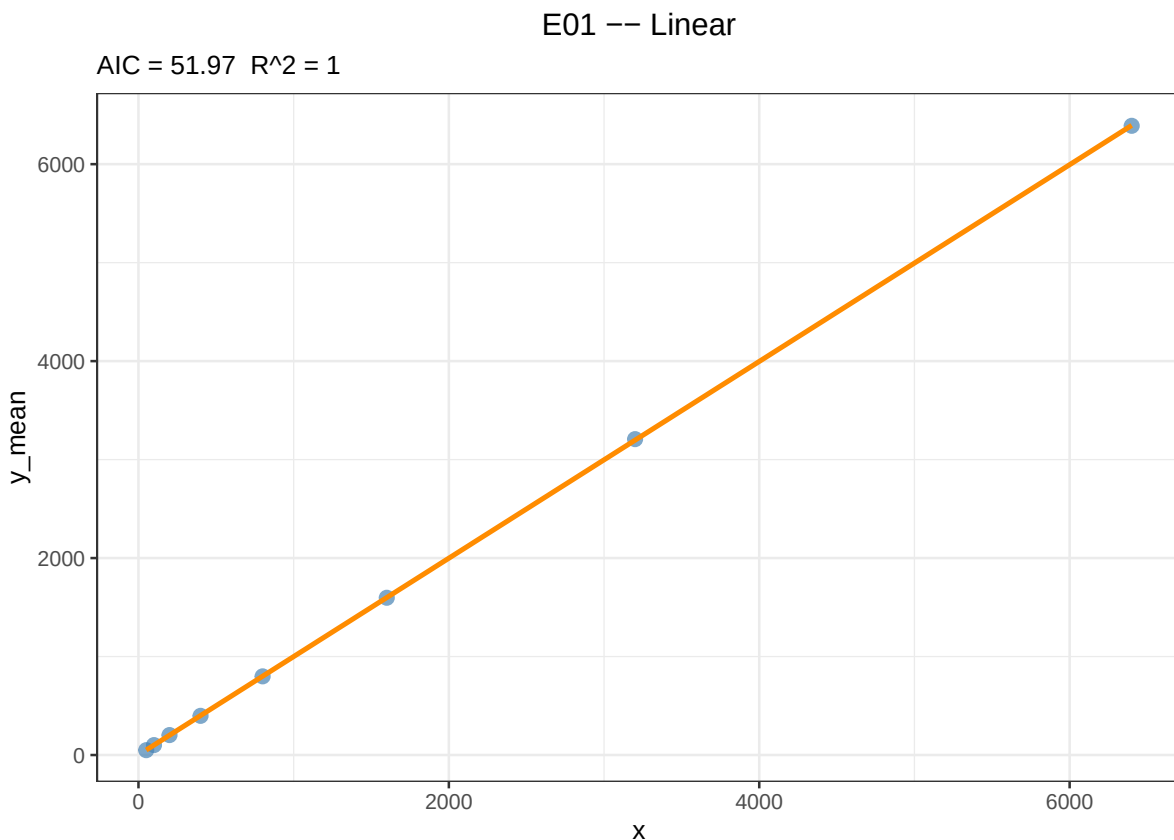
1	50	49.26667	50.91999	-1.65332058	-0.7333333
2	100	100.80000	100.86468	-0.06468401	0.8000000
3	200	202.66667	200.75408	1.91258912	2.6666667
4	400	397.83333	400.53286	-2.69953130	-2.1666667
5	800	798.86667	800.09044	-1.22377212	-1.1333333
6	1600	1596.90000	1599.20559	-2.30558710	-3.1000000
7	3200	3207.80000	3197.43588	10.36411628	7.8000000
8	6400	6389.56667	6393.89648	-4.32981030	-10.4333333

Whether the bias is acceptable must be compared with the linearity acceptance limits pre-specified in the protocol.

Parameter notes: The weights of `fit_equation()` accepts a numeric vector or the built-in schemes "equal", "1/y", "1/y^2", "1/x", "1/x^2", "inverse", "inverse2". Weighted and equal-weight fits should both be compared comprehensively in terms of residual structure, back-calculation bias, and pre-specified acceptance criteria; do not rely on a single fit metric.

6.3.5 Graphics and Prediction

```
plot(fit_lin, interval = "confidence", level = 0.95)
```



```
predict(
  fit_lin,
  newdata = data.frame(x = c(0.5, 2, 8)),
  interval = "confidence"
)
```

x	y	y_ci_lwr	y_ci_upr
0.5	1.474737	-3.922223	6.871698
2.0	2.973078	-2.421993	8.368150


```
8.0 8.966442 3.578916 14.353968
```

Back-calculate concentration from response:

```
predict(
  fit_lin,
  newdata = data.frame(y_mean = c(650, 3300, 6500)),
  inverse = TRUE
)
```

```
      x      y
649.7434 650
3302.6777 3300
6506.2210 6500
```

Prediction boundaries: Inverse back-calculation should avoid extrapolating beyond the calibration concentration range. Linear back-calculation within range is stable, but in plateau or nonlinear regions, inverse prediction is very sensitive to response error.

6.4 Example 2: Polynomial Analysis of a Twofold Dilution

6.4.1 Reading and Validating Data

The data are 5 concentration levels $\times$ 3 replicates:

```
poly_data <- read_csv("./data/linearity-polynomial.csv", show_col_types = FALSE)
poly_data <- as.data.frame(poly_data)
head(poly_data, 6)
```

```
      conc replicate signal
1  75.00         1    75.0
2  75.00         2    75.0
3  75.00         3    74.4
4 483.75         1   454.1
5 483.75         2   455.3
6 483.75         3   456.0
```

6.4.2 Compute replicate means and introduce weights:

```
rep_sum_w <- replicate_to_mean(
  poly_data,
  x = "conc",
  y = "signal",
  weights = "1/sd^2"
)
rep_sum_w
```

```
      x  y_mean  y_sd y_n  weight
1  75.00  74.8000 0.3464102 3 8.33333333
2 483.75 455.1333 0.9609024 3 1.08303249
3 892.50 865.9667 8.0257918 3 0.01552474
4 1301.25 1328.4667 24.4933324 3 0.00166688
5 1710.00 1710.4333 8.1193185 3 0.01516914
```

Parameter notes: The `weights` of `replicate_to_mean()` can be `NULL` (no weight column generated), `"n"`, `"1/sd"`, or `"1/sd^2"` (inverse-variance weighting). Weights should be determined based on the actual variance structure of the replicates or the protocol.

6.4.3 Candidate Equation Comparison

EP06-A2 typically compares linear, quadratic, and cubic models to determine whether higher-order terms are needed and to compare the residual standard errors:

```
fit_poly_lin <- fit_equation("E01", data = rep_sum_w, x = "x", y = "y_mean", weights = rep_sum_w$w)
print(fit_poly_lin)
```

E01 (Linear)

Formula: a + b*x

Call:

```
stats::lm(formula = frm, data = d, weights = weights)
```

Residuals:

Min	1Q	Median	3Q	Max
-20.993	-17.367	-5.447	16.760	27.047

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
a	-18.00771	20.0633790	-0.89754	0.43557
b	1.01397	0.0188681	53.73983	1.4192e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 24.3886 on 3 degrees of freedom

Multiple R-squared: 0.9990, Adjusted R-squared: 0.9986

F-statistic: 2887.97 on 1 and 3 DF, p-value: 1.42e-05

```
fit_poly_qua <- fit_equation("E02", data = rep_sum_w, x = "x", y = "y_mean", weights = rep_sum_w$w)
print(fit_poly_qua)
```

E02 (Quadratic)

Formula: a + b*x + c*x^2

Call:

```
stats::lm(formula = frm, data = d, weights = weights)
```

Residuals:

Min	1Q	Median	3Q	Max
-13.443	-13.294	-13.146	8.912	30.970

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
a	-7.14812e+00	3.09846e+01	-0.2307	0.8389994
b	9.72049e-01	8.28550e-02	11.7319	0.0071872 **
c	2.34851e-05	4.48020e-05	0.5242	0.6524439

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 28.0077 on 2 degrees of freedom

Multiple R-squared: 0.9991, Adjusted R-squared: 0.9982

F-statistic: 1095.05 on 2 and 2 DF, p-value: 0.000912

```
fit_poly_cub <- fit_equation("E12", data = rep_sum_w, x = "x", y = "y_mean", weights = rep_sum_w$w)
print(fit_poly_cub)
```

E12 (Cubic)

Formula: $a + b \cdot x + c \cdot x^2 + d \cdot x^3$

Call:

```
stats::lm(formula = frm, data = d, weights = weights)
```

Residuals:

Min	1Q	Median	3Q	Max
-13.146	-2.191	-2.191	8.764	8.764

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
a	2.04822e+01	2.48863e+01	0.82303	0.56161
b	7.25244e-01	1.39796e-01	5.18788	0.12123
c	3.86253e-04	1.91646e-04	2.01544	0.29321
d	-1.35487e-07	7.07339e-08	-1.91545	0.30631

Residual standard error: 18.3308 on 1 degrees of freedom

Multiple R-squared: 0.9998, Adjusted R-squared: 0.9992

F-statistic: 1705.48 on 3 and 1 DF, p-value: 0.0178

6.5 Key Points for Interpreting Results

1. **Separate fit from model judgment:** A model being successfully fitted does not mean it is appropriate; judge in light of residuals, bias, acceptance criteria, and the protocol.
2. **Residuals on two scales:** Examine both original and relative residuals; for data spanning orders of magnitude, focus on relative residuals.
3. **Weight rationale:** Prefer the variance structure of the replicates or protocol requirements.
4. **Range and extrapolation:** Conclusions are valid only within the validated concentration range; avoid inverse prediction in plateau regions or beyond range.
5. **Reproducibility:** Save the raw data, code, model parameters, convergence information, package version, and complete session information.

Chapter 7

Analytical Sensitivity

7.1 Analysis Objectives

Analytical sensitivity describes the ability of a measurement system to distinguish “signal from noise” at the low-concentration end, and consists of three progressively increasing limits:

- **LoB (Limit of Blank)**: The highest measurement that blank samples may produce (95th percentile).
- **LoD (Limit of Detection)**: The **lowest concentration** that can be distinguished from blank at a 95% confidence level.
- **LoQ (Limit of Quantitation)**: The **lowest concentration** at which the measurement result reaches a pre-specified precision target (for example $CV = 20\%$).

This document demonstrates how to compute analytical sensitivity with the `lob_lod_loq()` function of the `ivdtools` package according to **CLSI EP17-A2**:

- **LoB** by the 5.3.3.1 parametric method: $LoB = M_B + cp \cdot SD_B$, pooling all blank results into one pool, with $cp = 1.645/\sqrt{(1 - 1/(4(B-K)))}$ the small-sample-adjusted 95% one-sided quantile (B is the total number of blank results, K the number of blank samples).
- **LoD** by 5.4.3: first fit a Sadler precision profile model (variance as a function of concentration) to the non-blank low-concentration samples, then perform **fixed-point iteration** from LoB, $X = LoB + cp \cdot SD_{WL}(X)$, with $cp = 1.645/\sqrt{(1 - 1/(4(N_{TOT}-K)))}$.
- **LoQ** by Appendix D1 Example 1 (functional sensitivity): find the intersection of the precision profile with the target CV level line, $\sqrt{Var(X)}/X = target_cv$. This LoQ **reflects only precision and does not include bias**.

The example data are deterministic teaching data used only to demonstrate the workflow; formal studies must use the experimental design, target concentration, and acceptance criteria pre-specified in the protocol or standard.

```
library(ivdtools)
library(readr)
```

7.2 Overview of Functions

Function	Main purpose	Key input or output
<code>lob_lod_loq()</code>	Compute LoB / LoD / LoQ in one call	Summarized data, blank sample names, target CV
<code>replicate_to_mean()</code>	Summarize replicate measurements	Mean, SD, replicate count n
<code>fit_equation()</code>	Fit the Sadler precision profile (called internally)	<code>eq="sadler"</code> , <code>df_col</code>

Function	Main purpose	Key input or output
<code>print()</code> / <code>plot()</code>	Result display	Tabular summary, SD/CV dual panels

The input to `lob_lod_loq()` is an **already summarized** data.frame: one row per sample, containing a sample-name column, a mean column (i.e., concentration), an SD column, and a replicate-count `n` column; the column names can be freely specified. It internally and automatically completes the optimal Sadler model fitting, so no manual fitting is required.

7.3 Example: LoB / LoD / LoQ Analysis

7.3.1 Reading and Validating Data

The data contain 2 blank samples (blank_A, blank_B) and 6 low-concentration samples (L1–L6), each with replicate measurements (40 for each blank, 30 for each low concentration), for a total of 260 rows. The SD increases with concentration, consistent with the common low-end situation of “variance increasing with the mean”:

```
sens <- read_csv("./data/sensitivity.csv", show_col_types = FALSE)
sens <- as.data.frame(sens)
str(sens)
```

```
'data.frame': 260 obs. of 2 variables:
 $ sample: chr "blank_A" "blank_A" "blank_A" "blank_A" ...
 $ value : num 0.024 -0.006 0.055 -0.072 0.057 -0.023 -0.05 0.003 0.051 0.029 ...
```

```
dim(sens)
```

```
[1] 260 2
```

```
table(sens$sample)
```

```
blank_A blank_B    L1    L2    L3    L4    L5    L6
      40      40    30    30    30    30    30    30
```

7.3.2 Per-Sample Summarization

Use `replicate_to_mean()` to group by sample name and compute the mean, SD, and replicate count (when the grouping variable is character, the original values are retained, so each blank occupies one row):

```
rep <- replicate_to_mean(data.frame(x = sens$sample, y = sens$value),
                             x = "x", y = "y")
rep$sample <- rep$x
rep$mean <- rep$y_mean
rep$sd <- rep$y_sd
rep$n <- rep$y_n
rep[c("sample", "mean", "sd", "n")]
```

```
  sample      mean      sd  n
1 blank_A 0.0080750 0.04561623 40
2 blank_B -0.0036500 0.05287360 40
3      L1 0.5327667 0.11511694 30
4      L2 0.9601000 0.14135803 30
5      L3 1.9882667 0.21709110 30
```

```

6      L4  5.0710667 0.56385703 30
7      L5 10.1809667 0.96523481 30
8      L6 19.8890000 2.08779088 30

```

Note: `lob_lod_loq()` requires the **variance model response to be the variance** and weighted by degrees of freedom, so after summarizing you also need to add the two columns `var = sd^2` and `df = n - 1` to each row before passing them to `lob_lod_loq()`. If your data already contain these two columns, this step is unnecessary.

7.3.3 Computing LoB / LoD / LoQ

The `blank` parameter specifies the blank sample names (multiple allowed), and `target_cv` specifies the precision target for LoQ:

```

summ <- rep[c("sample", "mean", "sd", "n")]
res <- lob_lod_loq(summ, sample_col = "sample", mean_col = "mean",
                  sd_col = "sd", n_col = "n",
                  blank = c("blank_A", "blank_B"), target_cv = 0.20)

res

```

Limit of Blank / Detection / Quantitation (EP17-A2)

Precision profile: Sadler; best model = Model_3

Equation : $\sigma^2 = b1 + b2 * \mu^2$

Parameters : $b1 = 0.010196$, $b2 = 0.010388$

Fit : $AIC = -479.4$; $RSS = 0.08315$; $GoF\ p = 0.9437$

LoB (parametric, $B = 80$, $K = 2$) : 0.08357

LoD (fixed-point) : 0.2552

LoQ (CV = 20%) : 0.5868

Note: LoQ is based on precision ONLY; bias is not included.

Interpreting the results:

- **LoB = 0.0836:** the 95th percentile of blank measurements (parametric method), pooling $B = 80$ blank results and $K = 2$ blank samples.
- **LoD = 0.26:** the fixed point of $X = LoB + cp \cdot SD(X)$ on the precision profile, i.e., measurements above this concentration can be judged “non-blank”.
- **LoQ = 0.59 (CV = 20%):** the lowest concentration needed to reach the 20% precision target, based only on precision.

7.3.4 Returned Object Structure

`lob_lod_loq()` returns a `sensitivity` object, with each limit and parameter stored as a named component:

```
str(res, max.level = 1)
```

List of 13

```

$ call      : language lob_lod_loq(data = summ, sample_col = "sample", mean_col = "mean", sd_col = 
$ lob       : num 0.0836
$ lod       : num 0.255
$ loq       : num 0.587
$ n_blank   : int 80
$ B         : int 80
$ K         : int 2
$ target_cv : num 0.2
$ cp_lob    : num 1.65
$ cp_lod    : num 1.65
$ fit       :List of 12
..- attr(*, "class")= chr "fit_equation"
$ model     : chr "Model_3"

```

```
$ notes      : chr(0)
- attr(*, "class")= chr "sensitivity"
```

```
res$lob
```

```
[1] 0.0835705
```

```
res$lod
```

```
[1] 0.2552205
```

```
res$loq
```

```
[1] 0.5867594
```

```
res$B
```

```
[1] 80
```

```
res$K
```

```
[1] 2
```

```
res$cp_lob
```

```
[1] 1.647643
```

```
res$cp_lod
```

```
[1] 1.646183
```

```
res$model
```

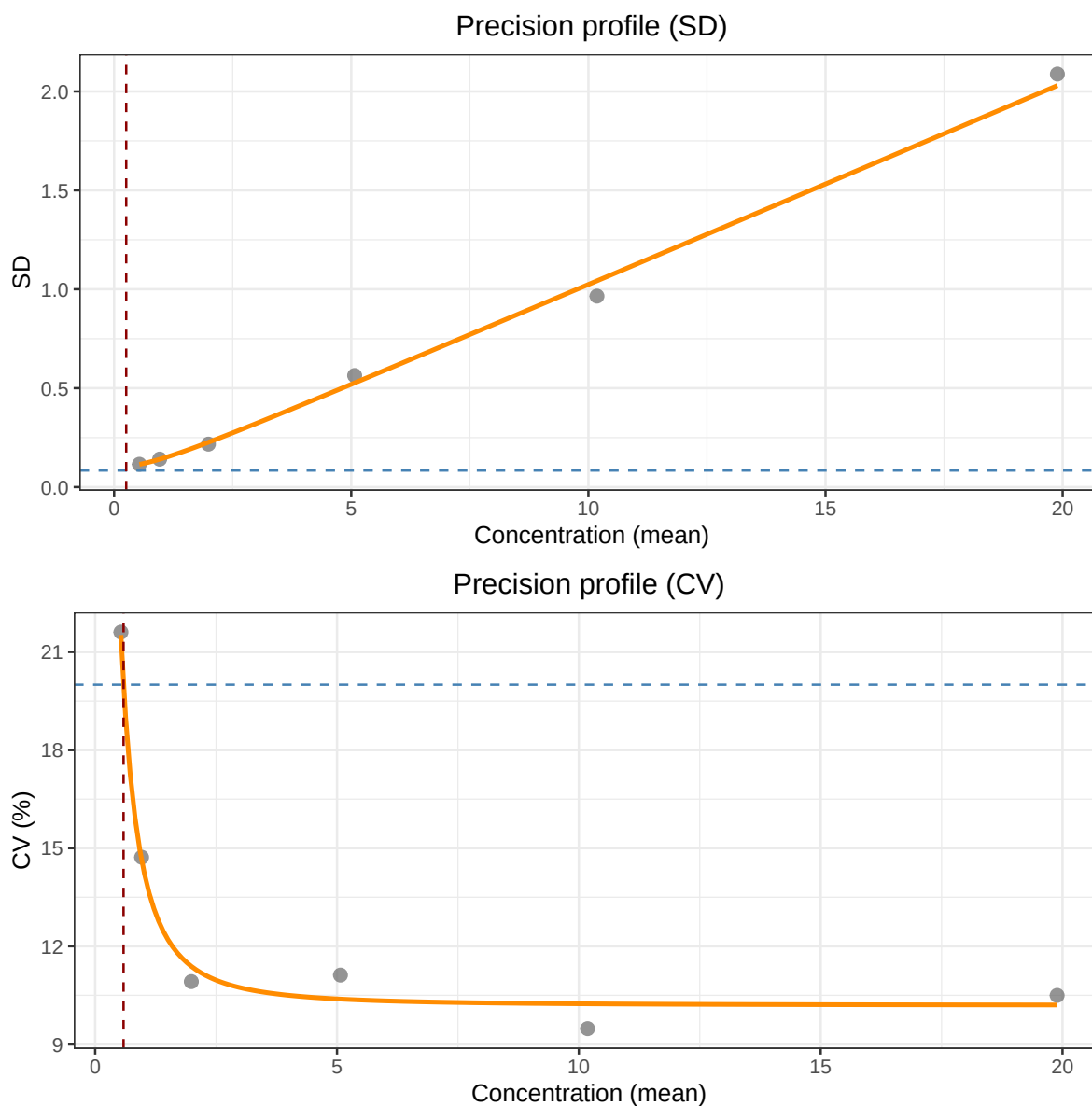
```
[1] "Model_3"
```

`cp_lob` and `cp_lod` are the two small-sample correction factors of EP17; `model` is the optimal Sadler model selected by AIC (in this example, model 3, the mixed-variance type).

7.3.5 Graphics

`plot()` draws the two precision-profile panels for SD and CV, overlays the LoB / LoD / LoQ reference lines, and visually shows the geometric meaning of each limit:

```
plot(res)
```

7.4 Other Input Scenarios

7.4.1 Scenario 1: Blank Only (LoB Only)

If there are no low-concentration levels, only LoB is computed, and LoD / LoQ are NA:

```
res_b <- lob_lod_loq(summ[summ$sample %in% c("blank_A", "blank_B"), ],
  sample_col = "sample", mean_col = "mean",
  sd_col = "sd", n_col = "n",
  blank = c("blank_A", "blank_B"))

res_b
```

Limit of Blank / Detection / Quantitation (EP17-A2)

LoB (parametric, B = 80, K = 2) : 0.08357

LoD (fixed-point) : --

LoQ (CV = 20%) : --

Note: LoQ is based on precision ONLY; bias is not included.

7.4.2 Scenario 2: No Blank (LoQ Only)

If there are no blank samples (`blank = NULL`), only LoQ is computed, and LoB / LoD are NA:

```
res_q <- lob_lod_loq(summ[!summ$sample %in% c("blank_A", "blank_B"), ],
                    sample_col = "sample", mean_col = "mean",
                    sd_col = "sd", n_col = "n")

res_q
```

Limit of Blank / Detection / Quantitation (EP17-A2)

Precision profile: Sadler; best model = Model_3

```
Equation      : sigma^2 = b1 + b2 * mu^2
Parameters    : b1 = 0.010196, b2 = 0.010388
Fit           : AIC = -479.4; RSS = 0.08315; GoF p = 0.9437
LoB (parametric, B = 0, K = 0) : --
LoD (fixed-point)              : --
LoQ (CV = 20%)                 : 0.5868
```

Note: LoQ is based on precision ONLY; bias is not included.

7.4.3 Scenario 3: LoQ < LoD

When the LoQ computed from the target CV is below the LoD (for example, large blank noise but good precision at low concentrations), LoQ takes $\max(\text{LoQ}, \text{LoD})$ and issues a warning, ensuring **LoQ** **LoD**:

```
# Construct data with "large blank noise but good low-concentration precision"
d_noisy <- data.frame(
  sample = c("blank", paste0("L", 1:6)),
  mean   = c(0, 0.5, 1, 2, 5, 10, 20),
  sd     = c(5, sqrt(0.01 + 0.02 * c(0.5, 1, 2, 5, 10, 20)^2)),
  n      = c(30L, rep(5L, 6))
)
res_n <- lob_lod_loq(d_noisy, sample_col = "sample", mean_col = "mean",
                    sd_col = "sd", n_col = "n", blank = "blank")

res_n
```

Limit of Blank / Detection / Quantitation (EP17-A2)

Precision profile: Sadler; best model = Model_3

```
Equation      : sigma^2 = b1 + b2 * mu^2
Parameters    : b1 = 0.01, b2 = 0.02
Fit           : AIC = -469.2; RSS = 2.924e-29; GoF p = 1
LoB (parametric, B = 30, K = 1) : 8.261
LoD (fixed-point)              : 10.79
LoQ (CV = 20%)                 : 10.79
```

Note: LoQ is based on precision ONLY; bias is not included.

Note: LoQ (precision) < LoD; LoQ reported as LoD = 10.79.

7.5 Key Points for Interpreting Results

1. **Target first:** Before analysis, determine the target CV (20% in this document is only a teaching example), confidence level, and acceptance criteria.
2. **Blank pooled pool:** LoB pools all blank results for computation; `cp` includes a B-K degrees-of-freedom correction that cannot be ignored when the sample size is small.
3. **LoQ contains only precision:** With this workflow, LoQ does not include bias and cannot replace a total-error (TE)-based limit-of-quantitation claim.
4. **Model selection:** Different Sadler models extrapolate very differently at the low-concentration end; the report should note the AIC-optimal model and its ranking.

5. **LoQ LoD:** The program guarantees LoQ is not below LoD; if the CV-based LoQ is below the LoD, it warns and takes `max`.
6. **Data format:** The input to `lob_lod_loq()` is an already-summarized `data.frame`; the original replicate data must first be processed by `replicate_to_mean()`.

Chapter 8

Bottle-to-Bottle Variability Analysis

8.1 Analysis Objectives

Bottle-to-bottle variability evaluates the differences in measurement results among different bottles (or different lots) of the same product. The analysis is usually done in two steps: first, use `bottle_anova()` to test whether the means of the bottles (or lots) differ statistically, and then use variance-component analysis to separate the between-bottle variability from the within-bottle variability, judging whether the magnitude of the difference meets the pre-specified precision limits. This document demonstrates both scenarios: the bottle-to-bottle difference of 15 bottles $\times$ 5 measurements, and the between-lot difference when different reagent lots measure the same quality-control material.

The example data are deterministic teaching data; statistical significance is not equivalent to product unacceptability; judgments can only be made against the acceptance limits pre-specified in the protocol.

```
library(ivdtools)
library(readr)
```

8.2 Overview of Functions

Function	Main purpose	Key input or output
<code>bottle_anova()</code>	One-way or nested ANOVA	<code>value ~ batch</code> or <code>value ~ batch/vial</code>
<code>tukey()</code>	Tukey HSD pairwise comparison and compact letters	<code>term</code> specifies the effect
<code>precision()</code> / <code>variance()</code>	Split between-bottle/within-bottle variance components	<code>value ~ bottle</code> one-way

8.3 Example 1: Bottle-to-Bottle Difference of 15 Bottles $\times$ 5 Measurements

8.3.1 Reading and Validating Data

```
bottle <- read_csv("../data/bottle-difference.csv", show_col_types = FALSE)
bottle <- as.data.frame(bottle)
bottle$bottle <- factor(bottle$bottle)
str(bottle)
```

```
'data.frame': 75 obs. of 3 variables:
 $ bottle : Factor w/ 15 levels "B01","B02","B03",...: 1 1 1 1 1 2 2 2 2 2 ...
 $ replicate: num 1 2 3 4 5 1 2 3 4 5 ...
 $ value : num 97.5 100.5 98.8 97.5 97.8 ...
```

```
dim(bottle)
```

```
[1] 75 3
```

8.3.2 Step 1: Test for Significant Difference

The one-way model `value ~ bottle` tests whether the means of the 15 bottles are equal:

```
ba <- bottle_anova(bottle, value ~ bottle)
print(ba)
```

Bottle ANOVA Analysis

```
Response:      value
Group variable: bottle (15 groups)
Complete cases: 75 / 75
Confidence level: 0.95
```

Descriptive Statistics:

bottle	n	Mean	SD
B01	5	98.4520	1.2739
B02	5	101.7140	0.9360
B03	5	99.8060	0.8153
B04	5	102.7980	0.4992
B05	5	98.7520	0.8662
B06	5	100.0960	1.4971
B07	5	100.7180	0.8466
B08	5	100.5000	1.3179
B09	5	99.8160	1.0246
B10	5	101.6400	0.8034
B11	5	99.3300	0.9360
B12	5	98.4700	1.0884
B13	5	99.7440	1.1004
B14	5	100.9040	0.4803
B15	5	99.7920	1.4477

ANOVA Table:

Term	Df	Sum Sq	Mean Sq	F-value	p-value	Sig
------	----	--------	---------	---------	---------	-----

```

bottle      14  107.7233      7.6945      7.1460      0.0000 ***
Residuals   60   64.6057      1.0768

```

```
-----
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Assumption Checks:

Bartlett test: K-squared = 9.9325, df = 14, p-value = 0.7671 (OK)

Shapiro-Wilk: W = 0.9828, p-value = 0.4011 (OK)

Post-hoc: use `tukey()` for pairwise comparisons

The ANOVA table gives the between-bottle F test; the `assumptions` section reports the Bartlett variance-homogeneity and Shapiro-Wilk normality tests.

8.3.3 Pairwise Comparison

```

posthoc <- tukey(ba)
posthoc

```

Tukey HSD Post-hoc Test

Response: value

Complete cases: 75 / 75

Confidence level: 0.95

Term: bottle

Comparison	Diff	lwr	upr	p-value	Sig
B02-B01	3.2620	0.9412	5.5828	0.0005	***
B03-B01	1.3540	-0.9668	3.6748	0.7526	
B04-B01	4.3460	2.0252	6.6668	0.0000	***
B05-B01	0.3000	-2.0208	2.6208	1.0000	
B06-B01	1.6440	-0.6768	3.9648	0.4526	
B07-B01	2.2660	-0.0548	4.5868	0.0624	
B08-B01	2.0480	-0.2728	4.3688	0.1408	
B09-B01	1.3640	-0.9568	3.6848	0.7431	
B10-B01	3.1880	0.8672	5.5088	0.0008	***
B11-B01	0.8780	-1.4428	3.1988	0.9896	
B12-B01	0.0180	-2.3028	2.3388	1.0000	
B13-B01	1.2920	-1.0288	3.6128	0.8078	
B14-B01	2.4520	0.1312	4.7728	0.0287	*
B15-B01	1.3400	-0.9808	3.6608	0.7656	
B03-B02	-1.9080	-4.2288	0.4128	0.2229	
B04-B02	1.0840	-1.2368	3.4048	0.9391	
B05-B02	-2.9620	-5.2828	-0.6412	0.0026	**
B06-B02	-1.6180	-3.9388	0.7028	0.4794	
B07-B02	-0.9960	-3.3168	1.3248	0.9686	
B08-B02	-1.2140	-3.5348	1.1068	0.8674	
B09-B02	-1.8980	-4.2188	0.4228	0.2298	
B10-B02	-0.0740	-2.3948	2.2468	1.0000	
B11-B02	-2.3840	-4.7048	-0.0632	0.0384	*
B12-B02	-3.2440	-5.5648	-0.9232	0.0006	***
B13-B02	-1.9700	-4.2908	0.3508	0.1830	
B14-B02	-0.8100	-3.1308	1.5108	0.9952	
B15-B02	-1.9220	-4.2428	0.3988	0.2134	
B04-B03	2.9920	0.6712	5.3128	0.0022	**
B05-B03	-1.0540	-3.3748	1.2668	0.9507	

B06-B03	0.2900	-2.0308	2.6108	1.0000	
B07-B03	0.9120	-1.4088	3.2328	0.9852	
B08-B03	0.6940	-1.6268	3.0148	0.9990	
B09-B03	0.0100	-2.3108	2.3308	1.0000	
B10-B03	1.8340	-0.4868	4.1548	0.2778	
B11-B03	-0.4760	-2.7968	1.8448	1.0000	
B12-B03	-1.3360	-3.6568	0.9848	0.7693	
B13-B03	-0.0620	-2.3828	2.2588	1.0000	
B14-B03	1.0980	-1.2228	3.4188	0.9330	
B15-B03	-0.0140	-2.3348	2.3068	1.0000	
B05-B04	-4.0460	-6.3668	-1.7252	0.0000	***
B06-B04	-2.7020	-5.0228	-0.3812	0.0092	**
B07-B04	-2.0800	-4.4008	0.2408	0.1258	
B08-B04	-2.2980	-4.6188	0.0228	0.0549	
B09-B04	-2.9820	-5.3028	-0.6612	0.0023	**
B10-B04	-1.1580	-3.4788	1.1628	0.9026	
B11-B04	-3.4680	-5.7888	-1.1472	0.0002	***
B12-B04	-4.3280	-6.6488	-2.0072	0.0000	***
B13-B04	-3.0540	-5.3748	-0.7332	0.0016	**
B14-B04	-1.8940	-4.2148	0.4268	0.2326	
B15-B04	-3.0060	-5.3268	-0.6852	0.0020	**
B06-B05	1.3440	-0.9768	3.6648	0.7619	
B07-B05	1.9660	-0.3548	4.2868	0.1854	
B08-B05	1.7480	-0.5728	4.0688	0.3515	
B09-B05	1.0640	-1.2568	3.3848	0.9470	
B10-B05	2.8880	0.5672	5.2088	0.0037	**
B11-B05	0.5780	-1.7428	2.8988	0.9999	
B12-B05	-0.2820	-2.6028	2.0388	1.0000	
B13-B05	0.9920	-1.3288	3.3128	0.9696	
B14-B05	2.1520	-0.1688	4.4728	0.0968	
B15-B05	1.0400	-1.2808	3.3608	0.9556	
B07-B06	0.6220	-1.6988	2.9428	0.9997	
B08-B06	0.4040	-1.9168	2.7248	1.0000	
B09-B06	-0.2800	-2.6008	2.0408	1.0000	
B10-B06	1.5440	-0.7768	3.8648	0.5576	
B11-B06	-0.7660	-3.0868	1.5548	0.9972	
B12-B06	-1.6260	-3.9468	0.6948	0.4711	
B13-B06	-0.3520	-2.6728	1.9688	1.0000	
B14-B06	0.8080	-1.5128	3.1288	0.9953	
B15-B06	-0.3040	-2.6248	2.0168	1.0000	
B08-B07	-0.2180	-2.5388	2.1028	1.0000	
B09-B07	-0.9020	-3.2228	1.4188	0.9866	
B10-B07	0.9220	-1.3988	3.2428	0.9837	
B11-B07	-1.3880	-3.7088	0.9328	0.7199	
B12-B07	-2.2480	-4.5688	0.0728	0.0670	
B13-B07	-0.9740	-3.2948	1.3468	0.9739	
B14-B07	0.1860	-2.1348	2.5068	1.0000	
B15-B07	-0.9260	-3.2468	1.3948	0.9831	
B09-B08	-0.6840	-3.0048	1.6368	0.9992	
B10-B08	1.1400	-1.1808	3.4608	0.9125	
B11-B08	-1.1700	-3.4908	1.1508	0.8956	
B12-B08	-2.0300	-4.3508	0.2908	0.1498	
B13-B08	-0.7560	-3.0768	1.5648	0.9976	
B14-B08	0.4040	-1.9168	2.7248	1.0000	
B15-B08	-0.7080	-3.0288	1.6128	0.9988	
B10-B09	1.8240	-0.4968	4.1448	0.2859	
B11-B09	-0.4860	-2.8068	1.8348	1.0000	
B12-B09	-1.3460	-3.6668	0.9748	0.7601	

B13-B09	-0.0720	-2.3928	2.2488	1.0000	
B14-B09	1.0880	-1.2328	3.4088	0.9374	
B15-B09	-0.0240	-2.3448	2.2968	1.0000	
B11-B10	-2.3100	-4.6308	0.0108	0.0523	
B12-B10	-3.1700	-5.4908	-0.8492	0.0009	***
B13-B10	-1.8960	-4.2168	0.4248	0.2312	
B14-B10	-0.7360	-3.0568	1.5848	0.9982	
B15-B10	-1.8480	-4.1688	0.4728	0.2668	
B12-B11	-0.8600	-3.1808	1.4608	0.9914	
B13-B11	0.4140	-1.9068	2.7348	1.0000	
B14-B11	1.5740	-0.7468	3.8948	0.5257	
B15-B11	0.4620	-1.8588	2.7828	1.0000	
B13-B12	1.2740	-1.0468	3.5948	0.8226	
B14-B12	2.4340	0.1132	4.7548	0.0311	*
B15-B12	1.3220	-0.9988	3.6428	0.7819	
B14-B13	1.1600	-1.1608	3.4808	0.9015	
B15-B13	0.0480	-2.2728	2.3688	1.0000	
B15-B14	-1.1120	-3.4328	1.2088	0.9266	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Compact Letter Display:

B04: a
B02: a
B10: a
B14: a
B07: ab
B08: ab
B06: b
B09: b
B03: b
B15: b
B13: b
B11: b
B05: b
B12: b
B01: b

Tukey HSD gives all pairwise comparisons and compact letter displays; bottles with different letters differ significantly in mean. In this example, there are many bottles, so some significant pairwise differences are normal; the key question is whether the **magnitude of the difference** is acceptable.

8.3.4 Step 2: Whether the Difference Magnitude Meets Expectations

Use `precision(value ~ bottle)` to split the total variability into the between-bottle component (bottle) and the within-bottle component (error):

```
bp <- precision(bottle, value ~ bottle)
bp <- variance(bp)
```

Variance components				
sample	component	VC	%Total	SD CV[%]
all	bottle	1.3236	55.1	1.1505 1.1485
all	error	1.0768	44.9	1.0377 1.0359

The between-bottle SD is 1.15, CV 1.15%, and the within-bottle SD is 1.04, CV 1.04%. Assuming the protocol specifies a total precision limit of 2% (CV), the between-bottle difference is within the

limit. **Note:** The “limit” here is only an example; formal studies must use the limits specified in the protocol or standard, and should compare the total CV (including between- and within-bottle) and each component separately.

```
bp <- ci(bp)
```

Confidence intervals (SD)

sample	component	estimate	lower	upper
all	total	1.5493	1.2449	2.0519
all	bottle	1.1505	0.4254	1.5704
all	error	1.0377	0.8807	1.2633

Confidence intervals (%CV)

sample	component	estimate	lower	upper
all	total	1.5467	1.2428	2.0485
all	bottle	1.1485	0.4246	1.5678
all	error	1.0359	0.8792	1.2612

8.4 Example 2: Different Reagent Lots Measuring the Same Quality-Control Material

8.4.1 Reading Data

Three reagent lots (L1, L2, L3) each measure the same quality-control material 15 times:

```
lot_data <- read_csv("../data/reagent-lot.csv", show_col_types = FALSE)
lot_data <- as.data.frame(lot_data)
lot_data$lot <- factor(lot_data$lot)
str(lot_data)
```

```
'data.frame': 45 obs. of 3 variables:
 $ lot      : Factor w/ 3 levels "L1","L2","L3": 1 1 1 1 1 1 1 1 1 1 ...
 $ replicate: num 1 2 3 4 5 6 7 8 9 10 ...
 $ value    : num 100 102 101 102 101 ...
```

```
lot_summary <- aggregate(value ~ lot, lot_data, function(x) c(n = length(x), mean = mean(x), sd =
lot_summary <- data.frame(lot = lot_summary$lot, lot_summary$value)
lot_summary
```

	lot	n	mean	sd
1	L1	15	100.8173	1.1626353
2	L2	15	101.2760	1.4271040
3	L3	15	100.5673	0.7829201

8.4.2 Testing the Between-Lot Difference

```
ba_lot <- bottle_anova(lot_data, value ~ lot)
print(ba_lot)
```

Bottle ANOVA Analysis

```
Response:      value
Group variable: lot (3 groups)
Complete cases: 45 / 45
```

8.4. EXAMPLE 2: DIFFERENT REAGENT LOTS MEASURING THE SAME QUALITY-CONTROL MATERIAL

Confidence level: 0.95

Descriptive Statistics:

lot	n	Mean	SD
L1	15	100.8173	1.1626
L2	15	101.2760	1.4271
L3	15	100.5673	0.7829

ANOVA Table:

Term	Df	Sum Sq	Mean Sq	F-value	p-value	Sig
lot	2	3.8754	1.9377	1.4528	0.2454	
Residuals	42	56.0183	1.3338			

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Assumption Checks:

Bartlett test: K-squared = 4.6245, df = 2, p-value = 0.0990 (OK)

Shapiro-Wilk: W = 0.9817, p-value = 0.6870 (OK)

Post-hoc: use `tukey()` for pairwise comparisons

```
tukey(ba_lot)
```

Tukey HSD Post-hoc Test

Response: value
Complete cases: 45 / 45
Confidence level: 0.95

Term: lot

Comparison	Diff	lwr	upr	p-value	Sig
L2-L1	0.4587	-0.5659	1.4832	0.5269	
L3-L1	-0.2500	-1.2745	0.7745	0.8247	
L3-L2	-0.7087	-1.7332	0.3159	0.2245	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Compact Letter Display:

```
L2: a
L1: a
L3: a
```

The compact letter display shows between which lots the means differ significantly and in which direction. Even if the between-lot mean difference is statistically significant, it must still be judged against the pre-specified acceptable bias (for example, the allowable target-value range of the quality-control material) to determine whether it is within the acceptable range; do not draw conclusions from the p-value alone.

8.5 Key Points for Interpreting Results

1. **Set acceptance limits first:** Before analysis, specify the allowable difference magnitude between lots/bottles (SD/CV or mean-bias limit).
2. **Significance acceptability:** ANOVA/Tukey test whether there is a difference; whether the magnitude is acceptable is determined by the limits.
3. **Variance decomposition:** Use variance components to separate between-bottle/lot and within-bottle variability, and evaluate the contribution of each component separately.
4. **Pairwise-comparison scale:** With many bottles, the number of pairwise comparisons is large; use the compact letter display to focus on the difference structure.
5. **Consistent conditions:** Between-lot comparisons should keep conditions such as method, instrument, operator, and time constant, with only the lot varying.
6. **Complete records:** Report the mean, SD of each bottle/lot, the ANOVA table, Tukey results, variance components, and limit judgments.

Chapter 9

Reference Intervals

9.1 Analysis Objectives

A reference interval (RI) is defined by the measurement results of a reference sample population and is used to interpret the normal range of an individual's results, corresponding to CLSI EP28-A3c. `ivdtools` provides three methods: the percentile method (nonparametric), the parametric method (normal or lognormal), and the robust method. The three methods differ in their sensitivity to sample size, distribution shape, and outliers; it is recommended to apply multiple methods to the same data simultaneously for comparison, and to perform outlier detection and normality assessment before establishing reference intervals.

This document demonstrates pre-analyses with `outliers_test()` and `normal_test()`, and then calls `reference_interval(method = "all")` to output the reference intervals of the three methods at once. The example data are deterministic teaching data (n=120, mildly right-skewed and containing outliers).

```
library(ivdtools)
library(readr)
```

9.2 Overview of Functions

Function	Main purpose	Key input or output
<code>outliers_test()</code>	Outlier detection	Grubbs / ESD / Dixon / IQR
<code>normal_test()</code>	Normality test and graphics	<code>method="auto"</code> selects the test by sample size
<code>reference_interval()</code>	Reference intervals by three methods	<code>method="all"</code> , <code>interval</code> , <code>ci</code> , <code>id</code>
<code>plot.reference_interval()</code>	Plot intervals by method	jitter + upper/lower limits

9.3 Example: Reference Intervals Using Three Methods Together

9.3.1 Reading and Validating Data

```
ri_data <- read_csv("./data/reference-interval.csv", show_col_types = FALSE)
ri_data <- as.data.frame(ri_data)
str(ri_data)
```

```
'data.frame': 120 obs. of 2 variables:
 $ id : chr "S001" "S002" "S003" "S004" ...
 $ value: num 131.1 100.8 106 74.3 116.9 ...
```

9.3.2 Pre-Analysis 1: Outlier Detection

EP28 recommends reviewing outliers before establishing reference intervals. With sample size $n=120$, Dixon is not suitable (Dixon applies only to $n \leq 30$), so use Grubbs and the generalized ESD:

```
outliers_test(ri_data, col = "value", method = "grubbs")
```

Grubbs Outliers Test

```
Data: ri_data
Column: value (n = 120, missing = 0)
Parameters: alpha = 0.05
Outliers: 1
```

Index	Value	G stat	Critical
88	183.0000	4.0104	3.4451

```
outliers_test(ri_data, col = "value", method = "esd", r = 3)
```

Generalized ESD Outliers Test

```
Data: ri_data
Column: value (n = 120, missing = 0)
Parameters: alpha = 0.05, r = 3
Outliers: 2
```

Index	Value	R stat	Critical
88	183.0000	4.0104	3.4451
57	178.0000	4.0576	3.4424

Parameter notes: The method of `outliers_test()` can be "grubbs", "esd", "dixon", or "iqr"; ESD specifies the maximum number of detections via `r`, Dixon specifies the tail via `type` ("both"/"min"/"max"), and IQR specifies the multiplier via `coef` (default 1.5). **Detecting an outlier does not mean it should be deleted** — go back to the original records, experimental procedure, and protocol requirements, and perform an include/exclude sensitivity analysis when there is sufficient justification.

9.3.3 Pre-Analysis 2: Normality Test

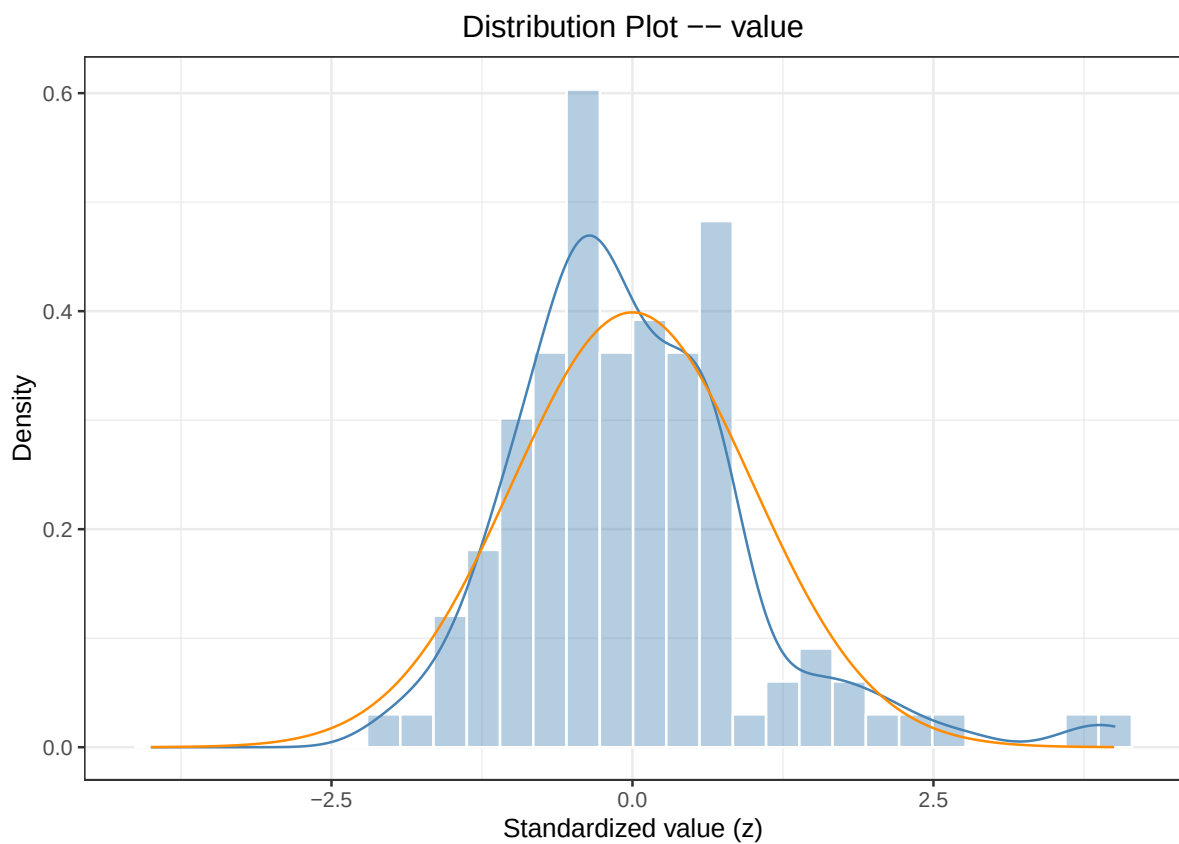
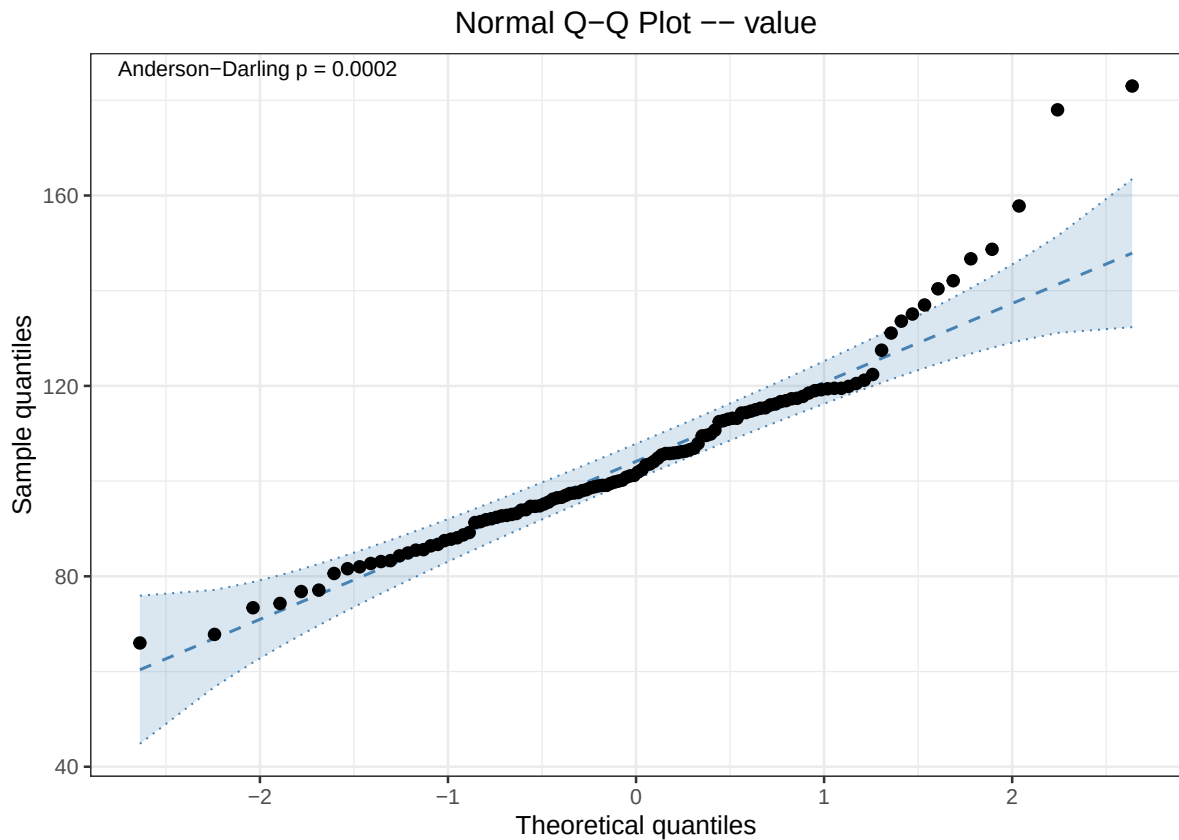
```
nt <- normal_test(ri_data, col = "value", method = "auto")
nt
```

Normality Test

```
Data: ri_data
Column: value (n = 120, missing = 0)
Method: Anderson-Darling
```

```
A = 1.7577, p = 0.0002
```

```
plot(nt, type = c("qq", "his"))
```



`method="auto"` automatically selects the Anderson-Darling test for $n=120$; the p-value is very small, indicating that the data do not follow a normal distribution (the data are right-skewed and contain outliers). The QQ plot and histogram can visually confirm the skewness.

9.3.4 Reference Intervals Using Three Methods Together

```
ri <- reference_interval(  
  ri_data,  
  col = "value",  
  interval = 0.95,  
  ci = 0.95,  
  method = "all",  
  id = "id"  
)  
print(ri)
```

Reference Interval Analysis

Column: value
Reference interval: 95% (alpha = 0.025)
Confidence level: 0.95
ID variable: id
Complete cases: 120 / 120

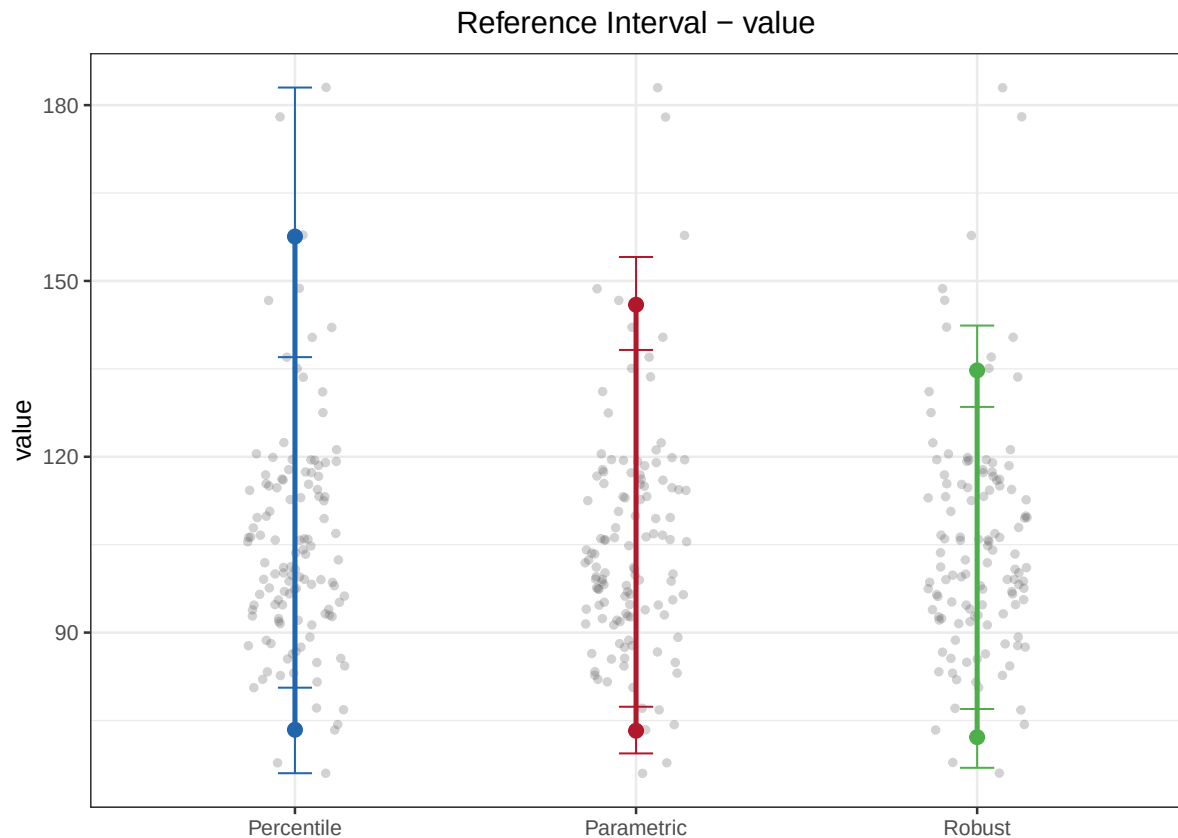
No missing values.
No duplicate IDs found.

Percentile (quantile type 6, CLSI EP28-A3c)
Lower = 73.4225 [66.0000, 80.6000]
Upper = 157.5725 [137.0000, 183.0000]

Parametric (log-transformed, Shapiro-Wilk p = 0.0661)
Lower = 73.2539 [69.3807, 77.3434]
Upper = 145.9407 [138.2241, 154.0880]

Robust (Huber M-estimation (k=1.5, 16 iter) + MAD)
Lower = 72.1573 [66.9270, 76.9617]
Upper = 134.7365 [128.5036, 142.3884]

```
plot(ri)
```

Parameter notes: In `reference_interval()`, `interval` is the coverage of the reference interval (default 0.95, corresponding to 2.5%–97.5%), while `ci` is the confidence level of the interval endpoints; the two have different meanings. `method` can be "percentile", "parametric", "robust", "all", or any combination of these. `id` is used for duplicate-sample detection.

Comparison of the three methods in this example:

- **Percentile method** (73.4–157.6): nonparametric and makes no distributional assumption, but the upper limit is raised by two large outliers;
- **Parametric method** (73.3–145.9): automatically detects non-normality and applies a log transformation (`transformed=TRUE`) before back-transforming;
- **Robust method** (72.2–134.7): based on the Huber M-estimator and MAD, most robust to outliers.

The upper limit of the percentile method is clearly higher than that of the robust method, which is precisely the effect of the outliers. **The reporting method should be selected under the guidance of the sample source, sample size, distribution characteristics, and study protocol**, rather than simply picking the one whose result “looks best”.

9.4 Key Points for Interpreting Results

1. **Protocol first:** Specify the reference population, inclusion/exclusion criteria, sample size (EP28 recommends 120 for the percentile method), and partitioning strategy.
2. **Check before compute:** Perform outlier detection and normality assessment first, and record the outlier-handling decision (whether to exclude, and the basis).
3. **Methods comparable:** Outputting the three methods together helps reveal the effect of outliers or distributional assumptions on the interval; state the chosen method and its rationale in the report.
4. **Distinguish parameters:** Distinguish `interval` (coverage) from `ci` (endpoint confidence); do not mix them up.
5. **Transparent transformation:** When the parametric method automatically applies a log transformation, report the transformation type and the back-transformation method.

6. **Non-unique conclusion:** A reference interval is not the same as a clinical decision limit, nor does it automatically constitute an acceptability determination.

Chapter 10

Quality Control

10.1 Analysis Objectives

Quality control (QC) determines whether a measurement system is in control by periodically measuring quality-control materials and monitoring whether the results deviate from target values. `ivdtools` provides Levey–Jennings charts with Westgard multi-rule evaluation (`qc_chart()`), grouped monitoring in lot-change/group-change scenarios, and the Youden plot for two-level QC (`youden_plot()`).

This document demonstrates: obtaining and understanding the Westgard rules, whether daily QC results exceed the limits, how to evaluate grouping correctly when a reagent lot changes, and the Youden plot for two-level QC. The example data are deterministic teaching data; **the Westgard rule combination and allowable limits should be pre-determined according to the laboratory’s risk-management strategy and protocol.**

```
library(ivdtools)
library(readr)
```

10.2 Overview of Functions

Function	Main purpose	Key input or output
<code>list_westgard()</code>	View Westgard rule names and meanings	<code>brief=TRUE</code> returns only the rule vector
<code>qc_chart()</code>	Levey–Jennings chart + rule judgment	<code>rules</code> , <code>mean/sd</code> , <code>group</code> , <code>run</code>
<code>youden_plot()</code>	Youden plot for two-level QC	two sample columns, target means/SDs

10.3 Example 1: Obtaining Rules and Daily QC

10.3.1 Viewing the Westgard Rules

```
list_westgard()
```

Westgard QC Rules

```
-----
1-2s    1 point exceeds +/-2s (warning)
1-3s    1 point exceeds +/-3s (out of control)
2-2s    2 consecutive points exceed +/-2s (same side)
```

```

R-4s    2 consecutive points differ by at least 4s
4-1s    4 consecutive points exceed +/-1s (same side)
8x       8 consecutive points on same side of mean
10x     10 consecutive points on same side of mean
-----

```

10.3.2 Reading Daily QC Data

```

qc_daily <- read_csv("../data/qc-daily.csv", show_col_types = FALSE)
qc_daily <- as.data.frame(qc_daily)
head(qc_daily)

```

```

run  value
1    1 103.24
2    2 103.06
3    3  94.52
4    4  99.51
5    5  99.14
6    6  95.62

```

The data are daily QC results with target mean = 100, SD = 3, for a total of 30 run points.

10.3.3 Running Quality Judgments

```

qc <- qc_chart(
  qc_daily,
  value = "value",
  rules = "1-2s,1-3s,2-2s,R-4s,4-1s,10x",
  mean = 100,
  sd = 3,
  run = "run"
)
print(qc)

```

```

QC Control Chart
Value column:    value
Mean (target):   100.0000
SD (target):     3.0000
N (total):       30 | Complete: 30 | Missing: 0
Rules applied:   1-2s, 1-3s, 2-2s, R-4s, 4-1s, 10x

```

Violations:

```

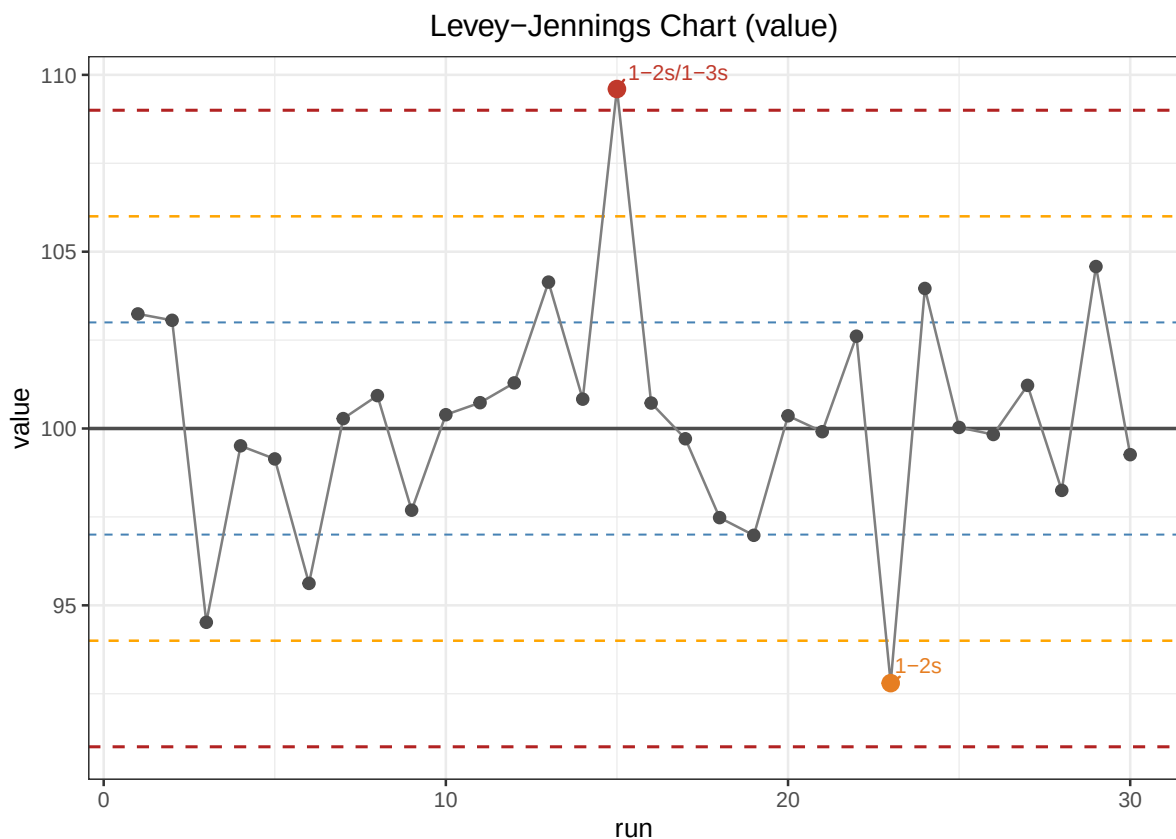
-----
1-2s  : points 15, 23
1-3s  : points 15
-> 2 violation(s): 1 warning(s), 1 out of control

```

```

plot(qc)

```



`qc_chart()` records the points violating each rule in `$violations`:

```
qc$violations
```

```
$`1-2s`  
[1] 15 23
```

```
$`1-3s`  
[1] 15
```

```
$`2-2s`  
integer(0)
```

```
$`R-4s`  
integer(0)
```

```
$`4-1s`  
integer(0)
```

```
$`10x`  
integer(0)
```

In this example, point 15 exceeds $+3s$ (triggering `1-3s` and `1-2s`, judged out of control), and point 23 falls below $-2s$ (triggering `1-2s`, judged a warning). `n_warn` and `n_oc` give the warning and out-of-control point counts, respectively.

Parameter notes: The rules of `qc_chart()` is a comma-separated rule string; `mean/sd` are the target values (when not given, they are estimated from the data, but monitored QC should use target values from the instructions). `run` specifies the run-order column, and `group` specifies the grouping column (see Example 2).

10.4 Example 2: Grouped Evaluation on Reagent Lot Change

10.4.1 Reading Data

In the data, the first 10 points are lot L1, the middle 10 are lot L2 (with an overall shift), and the last 10 are lot L3:

```
qc_lot <- read_csv("./data/qc-lot-change.csv", show_col_types = FALSE)
qc_lot <- as.data.frame(qc_lot)
table(qc_lot$lot)
```

```
L1 L2 L3
10 10 10
```

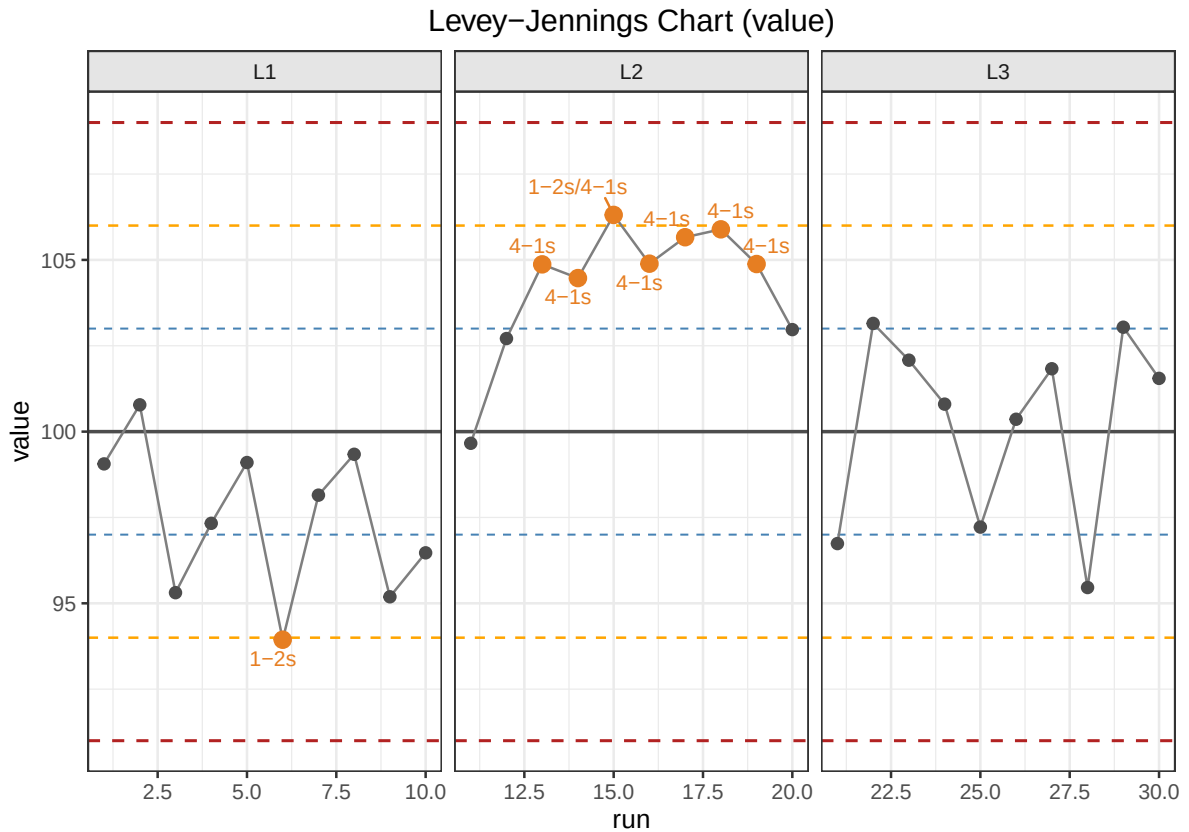
10.4.2 Grouping the Whole Sequence

Use `group = "lot"` for plotting; the Westgard rules are still judged on the **entire sequence**:

```
qc_lot_all <- qc_chart(
  qc_lot,
  value = "value",
  mean = 100,
  sd = 3,
  run = "run",
  group = "lot"
)
qc_lot_all$n_oc
```

```
[1] 0
```

```
plot(qc_lot_all)
```



Although the plot is faceted by lot, the rules are computed on the entire sequence, so observe the systematic shift caused by the lot change.

10.5 Example 3: Youden Plot for Two-Level QC

10.5.1 Reading Data

The Youden plot uses the results of two quality-control materials at different concentrations (levels) in the same run (wide-table format, one column per level):

```
youden_dat <- read_csv("../data/qc-youden.csv", show_col_types = FALSE)
youden_dat <- as.data.frame(youden_dat)
head(youden_dat)
```

```
run  low  high
1    1  77.86 209.36
2    2  82.19 207.10
3    3  75.50 189.00
4    4  80.40 195.36
5    5  80.64 198.23
6    6  78.22 200.14
```

10.5.2 Drawing the Youden Plot

```
yd <- youden_plot(
  youden_dat,
  sample1 = "low",
  sample2 = "high",
  mean1 = 80,
  mean2 = 200,
```

```

sd1 = 2,
sd2 = 5
)
print(yd)

```

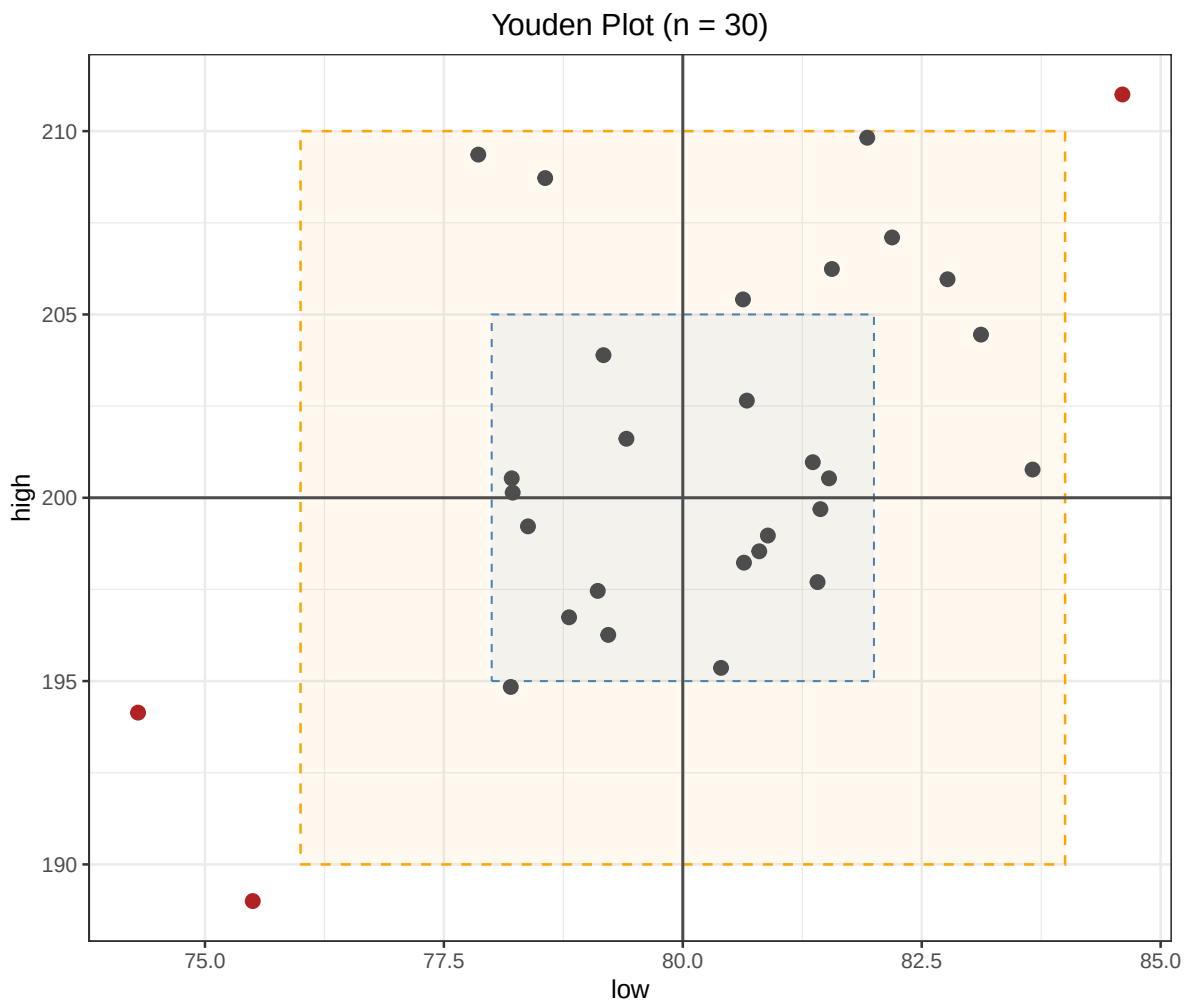
Youden Plot

```

Sample 1 (low):
  Mean:  80.0000
  SD:    2.0000
Sample 2 (high):
  Mean:  200.0000
  SD:    5.0000
Correlation: 0.5428
N (total):  30 | Complete: 30 | Missing: 0
Points outside +/-2SD: 3 (rows: 3, 17, 19)

```

```
plot(yd)
```



```
yd$outside_rows
```

```
[1]  3 17 19
```

`youden_plot()` uses a $\pm 2s$ rectangle to mark points outside the range. In this example, 3 run points fall outside the rectangle (including an artificially set out-of-control point), suggesting that these runs

may be affected by a systematic error at the same time. A shared shift at both levels (points moving outward along the diagonal) suggests a systematic error, while a large shift in a single direction is more likely to come from random error or a single-level problem.

Parameter notes: The `sample1/sample2` of `youden_plot()` are the numeric columns of the two levels; `mean1/mean2/sd1/sd2` are the target values (when not given, they are estimated from the data). `$n_outside` and `$outside_rows` give the points outside the rectangle.

10.6 Key Points for Interpreting Results

1. **Target-value source:** The target mean and SD of the QC chart should come from the instructions or validation data, not be estimated on the fly from the data.
2. **Rule combination:** The Westgard rule combination and the disposition process for judging out of control should be pre-specified in the protocol and re-evaluated with the risk.
3. **Two-level complementarity:** The Youden plot looks at the joint performance of both levels simultaneously, identifying signs of systematic and random error.
4. **Out-of-control disposition:** After detecting an out-of-control point, stop reporting, investigate the cause, re-validate and recall if necessary, and record the complete disposition chain.
5. **Statistics release:** QC judgment is one basis for the release decision; the final decision should comply with the laboratory's quality-system requirements.

Chapter 11

Sample Size Calculation

11.1 Analysis Objectives

Sample size calculation determines the required sample size or the achievable test power during the research-design and performance-evaluation planning phase. `ivdtools` provides three standalone functions corresponding to three common problem types:

- `sample_size_bland_altman()`: sample size/power for Bland–Altman agreement evaluation (Lu et al. 2016);
- `sample_size_proportion_ci()`: sample size required for a single-proportion confidence interval to reach a specified half-width;
- `sample_size_proportion()`: sample size/power/significance for a one-arm target-value test (a single-sample proportion against a known target).

All three functions are parameter-driven and do not require raw data files. They return scalars with attributes; `print()` gives a friendly explanation, and `as.numeric()` extracts the numeric result. The parameters in the examples (power, confidence level, allowable half-width, etc.) are teaching settings; formal studies must use the parameters specified in the protocol.

```
library(ivdtools)
```

11.2 Overview of Functions

Function	Main purpose	Solving direction
<code>sample_size_bland_altman()</code>	BA agreement evaluation	from power find n , or from n find power
<code>sample_size_proportion_ci()</code>	Single-proportion confidence interval	from d find n , or from n find d
<code>sample_size_proportion()</code>	One-arm target-value test	given any two of n/alpha/power , find the third

11.3 Example 1: Sample Size for Bland–Altman Agreement

11.3.1 Finding Sample Size from Target Power

```
n_ba <- sample_size_bland_altman(  
  power = 0.80,  
  mu = 0.2,  
  sd = 1,  
  delta = 2.5
```

```
)
print(n_ba)
```

Sample size for Bland-Altman agreement assessment

```
Mode           : Compute n from power
Mean difference : 0.2000
SD of differences: 1.0000
Clinical limit  : 2.5000
Confidence level : 0.9500
Agreement level : 0.9500
Sample size (n) : 201
Power          : 0.800300
```

```
as.numeric(n_ba)
```

```
[1] 201
```

Under the settings of mean difference $\mu = 0.2$, difference SD $sd = 1$, and clinical agreement limit $\delta = 2.5$, about 201 pairs are needed to achieve 80% power.

Parameter notes: μ is the mean difference, sd is the difference standard deviation (>0), and δ is the clinically acceptable agreement limit (>0). `conf.level` is the confidence level of the agreement-limit confidence interval, while `agree.level` is the coverage of the LoA; the two are at different levels — the former affects the reliability of the estimate, the latter determines the width of the LoA itself.

11.3.2 Finding Power from Sample Size

Given an existing sample size, look up the achievable power:

```
sample_size_bland_altman(n = 201, mu = 0.2, sd = 1, delta = 2.5)
```

Sample size for Bland-Altman agreement assessment

```
Mode           : Compute power from n
Mean difference : 0.2000
SD of differences: 1.0000
Clinical limit  : 2.5000
Confidence level : 0.9500
Agreement level : 0.9500
Sample size (n) : 201
Power          : 0.800300
```

This mutually verifies the above; $n=201$ exactly corresponds to power 0.80.

11.4 Example 2: Sample Size for a Single-Proportion Confidence Interval

11.4.1 Finding Sample Size from Target Half-Width

```
n_pci <- sample_size_proportion_ci(p = 0.3, d = 0.05)
print(n_pci)
```

Single proportion confidence interval

```
Mode           : Compute n from p and d
Method          : wilson
Target proportion: 0.3000
Confidence level : 0.9500
```

```
Sample size (n) : 320
Half-width (d) : 0.050000
```

For an expected proportion $p = 0.3$, to keep the half-width of the 95% confidence interval no larger than 0.05, about 320 cases are needed.

11.4.2 Finding Half-Width from Sample Size

```
sample_size_proportion_ci(p = 0.3, n = 320)
```

Single proportion confidence interval

```
Mode          : Compute d from p and n
Method        : wilson
Target proportion: 0.3000
Confidence level : 0.9500
Sample size (n) : 320
Half-width (d)  : 0.049967
```

At $n=320$ the half-width is about 0.05, mutually verifying the above.

Parameter notes: p is the expected proportion (must be given, $0 < p < 1$) and d is the desired half-width ($0 < d < 0.5$). `method` accepts 7 confidence-interval methods (default "wilson", also "wald", "wald-cc", "agresti-coull", "jeffreys", "wilson-cc", "clopper-pearson"); the method choice affects the required sample size and should be stated in the protocol.

11.5 Example 3: Sample Size for a One-Arm Target-Value Test

11.5.1 Finding Sample Size

```
n_sp <- sample_size_proportion(
  p0 = 0.2,
  p1 = 0.35,
  alpha = 0.05,
  power = 0.8,
  alternative = "greater",
  method = "wilson"
)
print(n_sp)
```

Sample size for a single-arm target value test

```
Mode          : Compute n from alpha and power
Method        : wilson
H0: p <= 0.2000
H1: p > 0.2000
Expected p    : 0.3500
Sample size (n) : 47
Alpha        : 0.050000
Power        : 0.815687
Critical x    : 14
```

Under the settings of $p_0 = 0.2$ (target/null-hypothesis proportion), $p_1 = 0.35$ (expected proportion), one-sided $\alpha = 0.05$, and power 0.80, about 47 cases are needed.

Parameter notes: `alternative` can be "greater" (in which case $p_1 > p_0$ is required), "less", or "two.sided". This method is based on confidence-interval inversion: given any two of $n/\alpha/\text{power}$, find the third; when all three are given, it serves as a consistency check.

11.5.2 Finding Power

```
sample_size_proportion(
  p0 = 0.2,
  p1 = 0.35,
  n = 47,
  alpha = 0.05,
  alternative = "greater"
)
```

Sample size for a single-arm target value test

```
Mode          : Compute power from n and alpha
Method        : wilson
H0: p <= 0.2000
H1: p > 0.2000
Expected p    : 0.3500
Sample size (n) : 47
Alpha        : 0.050000
Power        : 0.815687
Critical x    : 14
```

11.5.3 Finding the Significance Level (alpha)

```
sample_size_proportion(
  p0 = 0.2,
  p1 = 0.35,
  n = 47,
  power = 0.8,
  alternative = "greater"
)
```

Sample size for a single-arm target value test

```
Mode          : Compute alpha from n and power
Method        : wilson
H0: p <= 0.2000
H1: p > 0.2000
Expected p    : 0.3500
Sample size (n) : 47
Alpha        : 0.046728
Power        : 0.815687
Critical x    : 14
```

The three solving directions verify one another, and the results are consistent (n=47, power=0.8, alpha 0.05).

11.6 Key Points for Interpreting Results

1. **Parameters first:** Before computing sample size, determine power, confidence level, allowable error/half-width, expected proportion, and the one- or two-sided direction.
2. **Results are conditional:** The sample size is valid only under the set parameters; inaccurate parameter estimates lead to inaccurate sample sizes.
3. **Consistent methods:** For proportion-type problems, determine the CI method (Wilson, etc.) in advance; do not choose after the fact.
4. **Direction constraint:** For the one-arm test, note the consistency between `alternative` and the relative magnitude of `p0/p1`.
5. **Record in protocol:** Write the parameters, function calls, and outputs together into the study protocol for easy review and update.

Chapter 12

Method Comparison

12.1 Analysis Objectives

Method comparison evaluates the agreement between the results measured on the same samples by a candidate method and a reference method, corresponding to CLSI EP09. The `mcr()` function in `ivdtools` provides a progressive S3 workflow: descriptive statistics → correlation analysis → regression (OLS, WLS, Deming, weighted Deming, Passing–Bablok) → Bland–Altman → outliers → bias at medical decision levels, and provides plotting, prediction, and summarization. This document demonstrates the full workflow with a complete example containing different lots and sample types, and demonstrates subgroup comparison after splitting by lot/sample type.

The example data are deterministic teaching data, into which an outlier has been deliberately added to demonstrate outlier detection; statistical results do not automatically constitute a pass/fail determination; acceptance criteria should be defined before analysis.

```
library(ivdtools)
library(readr)
```

12.2 Overview of Functions

Function	Main purpose	Key input or output
<code>mcr()</code>	Create a method-comparison object	<code>id</code> , <code>candidate</code> , <code>reference</code> , <code>weights</code>
<code>describe()</code>	Descriptive statistics	<code>candidate/reference/difference</code> and extra columns
<code>correlation()</code>	Correlation analysis	<code>pearson/spearman/kendall</code>
<code>regression()</code>	Regression fitting	<code>ols/wls/deming/wdeming/pb</code> , <code>lambda</code> , <code>weights</code>
<code>bland_altman()</code>	Agreement limits	<code>difference/ratio/percent</code> , <code>x_axis</code>
<code>outlier()</code>	Difference outlier	<code>grubbs/esd/dixon/iqr</code> , <code>type</code>
<code>bias()</code>	Bias at medical decision levels	<code>mdl</code> , <code>interval</code>
<code>predict()</code>	Forward/inverse prediction	requires regression first
<code>plot()</code> / <code>summary()</code>	Graphics and summary	<code>type</code> selects the graphic

12.3 Example: A Complete Single-Sample Analysis

12.3.1 Reading and Validating Data

The data contain 40 cases, each with an ID `id`, candidate method result `candidate`, reference method result `reference`, as well as a lot `batch` (B1/B2) and a sample type `sample_type`


```
B2                20    50.0%
```

```
sample_type
Level          Count  Percent
-----
plasma         13    32.5%
serum          14    35.0%
urine          13    32.5%
```

```
mc
```

Method Comparison

```
Data class:      data.frame
Total rows:      40
Complete pairs:  40
ID variable:     id
Test method (X): candidate
Reference (Y):   reference
Duplicate IDs:   none
```

Analysis slots:

```
[x] Describe
[ ] Correlation
[ ] Regression
[ ] Outlier
[ ] Bland-Altman
[ ] Bias
```

The `cols` of `describe()` specifies the categorical/numeric extra columns to include in the description, and supports exclusion with a `-` prefix.

12.3.3 Correlation Analysis

```
mc <- correlation(mc, method = "pearson")
```

Correlation

```
Method: pearson
r = 0.9985, 95% CI [0.9971, 0.9992]
p-value: < 2.2e-16
n = 40
```

Parameter notes: The method of `correlation()` can be "pearson", "spearman", or "kendall". Correlation reflects the degree to which the two variables co-vary; it is **not** the same as agreement and cannot replace Bland–Altman or regression bias evaluation.

12.3.4 Regression Analysis

Compare OLS, Deming, Passing–Bablok, and WLS in turn. Because `regression()` updates the regression result within the object, the multiple methods are stored in separate objects for comparison:

```
mc_ols <- regression(mc, method = "ols")
```

Regression

```
Method: OLS
Intercept = -2.9723 (SE = 1.0971)
Slope      = 0.9598 (SE = 0.0087)
```

```

Intercept 95% CI: [-5.1933, -0.7513]
Slope      95% CI: [0.9423, 0.9773]
  H0: slope = 1 -> t = -4.6494, p = 3.944e-05
    slope significantly different from 1 (CI excludes 1)
R-squared = 0.9969
Sigma      = 3.2015
n = 40

```

```
mc_dem <- regression(mc, method = "deming")
```

Regression

```

Method: Deming
Intercept = -3.1323 (SE = 1.4133)
Slope      = 0.9612 (SE = 0.0092)
Intercept 95% CI: [-5.9934, -0.2712]
Slope      95% CI: [0.9426, 0.9798]
  H0: slope = 1 -> t = -4.2313, p = 0.0001413
    slope significantly different from 1 (CI excludes 1)
Sigma      = 3.2026
n = 40

```

```
mc_pb <- regression(mc, method = "pb")
```

Regression

```

Method: Passing-Bablok
Intercept = -2.1674 (SE = NA)
Slope      = 0.9556 (SE = NA)
Intercept 95% CI: [-3.1487, -0.9828]
Slope      95% CI: [0.9444, 0.9669]
Sigma      = 3.2299
n = 40

```

```
mc_wls <- regression(mc, method = "wls", weights = "1/y")
```

Regression

```

Method: WLS
Weights: 1/y
Intercept = -2.2924 (SE = 0.9153)
Slope      = 0.9518 (SE = 0.0108)
Intercept 95% CI: [-4.1452, -0.4396]
Slope      95% CI: [0.9299, 0.9736]
  H0: slope = 1 -> t = -4.4712, p = 6.818e-05
    slope significantly different from 1 (CI excludes 1)
R-squared = 0.9951
Sigma      = 0.4819
n = 40

```

```

knitr::kable(
  data.frame(
    Method = c("OLS", "Deming", "Passing-Bablok", "WLS(1/y)"),
    Intercept = c(mc_ols$regression$intercept, mc_dem$regression$intercept,
      mc_pb$regression$intercept, mc_wls$regression$intercept),
    Slope = c(mc_ols$regression$slope, mc_dem$regression$slope,
      mc_pb$regression$slope, mc_wls$regression$slope)
  )
)

```

```

),
digits = 4,
caption = "Intercept and slope comparison across the four regression methods"
)

```

Table 12.3: Intercept and slope comparison across the four regression methods

Method	Intercept	Slope
OLS	-2.9723	0.9598
Deming	-3.1323	0.9612
Passing-Bablok	-2.1674	0.9556
WLS(1/y)	-2.2924	0.9518

```
print(mc_ols)
```

Method Comparison

```

Data class:      data.frame
Total rows:      40
Complete pairs:  40
ID variable:     id
Test method (X): candidate
Reference (Y):   reference
Duplicate IDs:   none

```

Analysis slots:

```

[x] Describe
[x] Correlation (pearson)
[x] Regression (OLS)
[ ] Outlier
[ ] Bland-Altman
[ ] Bias

```

In this example, the OLS slope is 0.96, and its 95% CI does not contain 1, indicating that the candidate method has about a 4% proportional bias (the true bias in the data is 5%).

Parameter notes: The method of `regression()` can be "ols", "wls", "deming", "wdeming", or "pb". Deming uses `lambda` to specify the variance ratio (ref/cand, default 1); wls/wdeming must provide `weights` (a built-in scheme or a data column name); `conf.level` controls the parameter confidence interval.

12.3.5 Bias at Medical Decision Levels

Use `bias()` to evaluate the predicted bias at medical decision levels (MDL):

```
mc_ols <- bias(mc_ols, mdl = c(30, 80, 150), interval = "both")
```

Bias

```

Method: OLS
Interval: both (95%)
MDL points: 3
Bias range: [4.1791, 9.0064]

```

```

mdl      bias ci_lower ci_upper pi_lower pi_upper
-----
30 4.179112 2.407658 5.950566 -2.539698 10.89792

```

```
80 6.190463 5.018240 7.362685 -0.395770 12.77670
150 9.006354 7.789153 10.223554 2.411967 15.60074
```

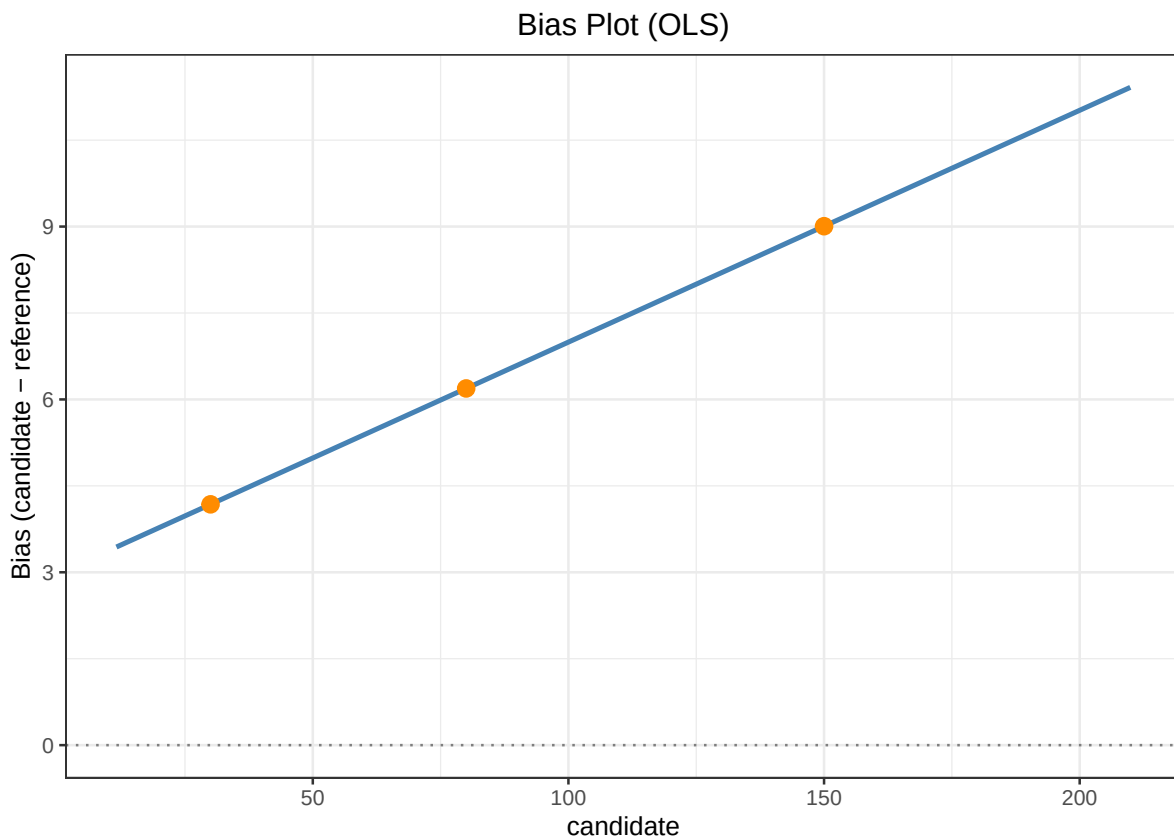
```
mc_ols$bias
```

Bias

```
Method: OLS
Interval: both (95%)
MDL points: 3
Bias range: [4.1791, 9.0064]
```

mdl	bias	ci_lower	ci_upper	pi_lower	pi_upper
30	4.179112	2.407658	5.950566	-2.539698	10.89792
80	6.190463	5.018240	7.362685	-0.395770	12.77670
150	9.006354	7.789153	10.223554	2.411967	15.60074

```
plot(mc_ols, type = "bias", mdl = c(30, 80, 150))
```



Parameter notes: In `bias(mdl, level, interval)`, `mdl` is a vector of medical decision levels; `interval` can be `"`, `"confidence"`, `"prediction"`, or `"both"`. Bias is defined as the difference between the fitted value of the candidate method at `mdl` and `mdl`; interpret it together with the regression direction and the allowable bias specified in the protocol.

12.3.6 Bland-Altman Analysis

```
mc_ols <- bland_altman(mc_ols, type = "difference")
```

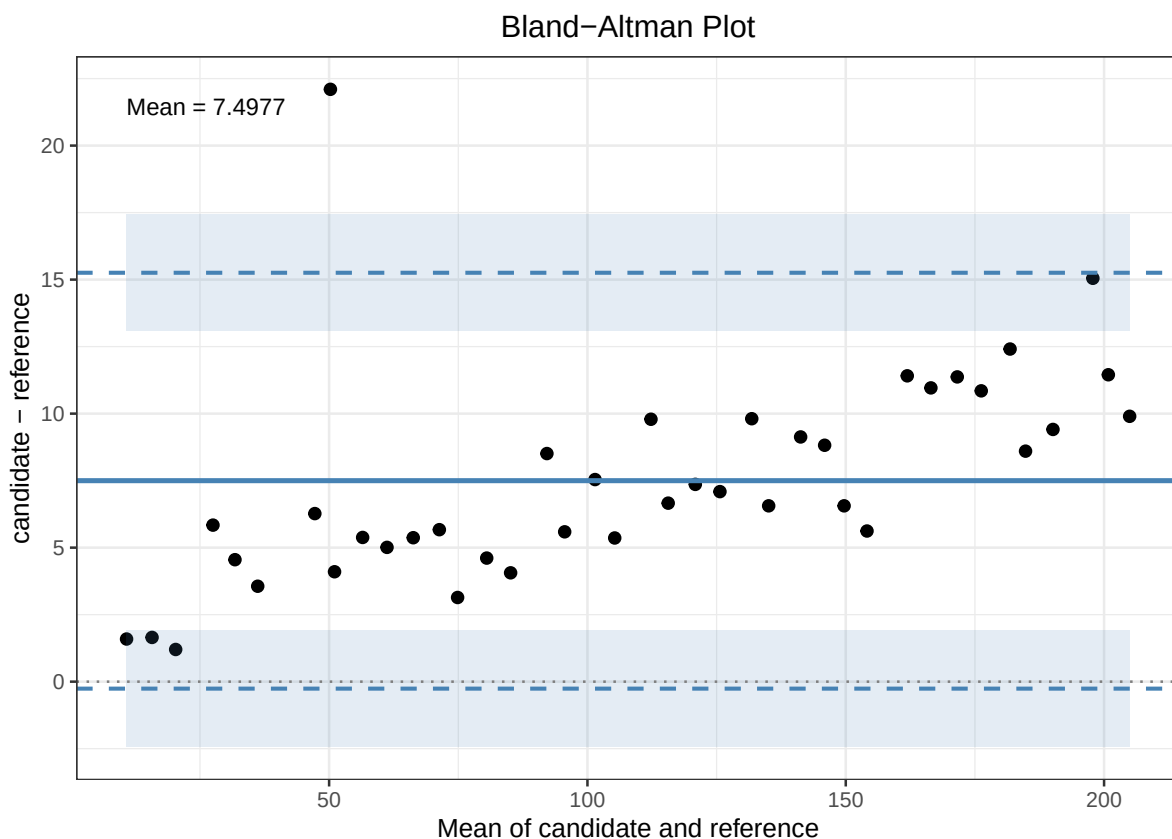
Bland-Altman

```
Type:      difference
Y-axis:    candidate - reference
X-axis:    Mean of candidate and reference
n = 40
```

```
Mean diff:      7.4977
SD diff:       3.9583
95% LoA: [-0.2603, 15.2558]
```

```
95% CI for LoA:
  Lower LoA: [-2.4419, 1.9213]
  Upper LoA: [13.0742, 17.4374]
```

```
plot(mc_ols, type = "bland_altman")
```



```
mc_ols$bland_altman[c("mean_diff", "sd_diff", "loa")]
```

```
$mean_diff
[1] 7.49775
```

```
$sd_diff
[1] 3.958259
```

```
$loa
      lower      upper
-0.2602947 15.2557947
```

Parameter notes: The type of `bland_altman()` can be "difference", "ratio", or "percent" (the Y-axis metric), and `x_axis` can be "mean", "candidate", or "reference" (the X-axis); `agree.level` controls the LoA width, and `conf.level` controls the LoA confidence interval.

12.3.7 Outlier Detection

Detect outliers in the paired differences:

```
mc_ols <- outlier(mc_ols, method = "grubbs", type = "difference")
```

Outlier

```
Method: grubbs
Data:   Difference (candidate - reference)
Alpha:  0.05
```

1 outlier(s) found:

Row	Value	Statistic	Critical
7	22.1000	3.6891	3.0361

```
mc_ols$outlier
```

Outlier

```
Method: grubbs
Data:   Difference (candidate - reference)
Alpha:  0.05
```

1 outlier(s) found:

Row	Value	Statistic	Critical
7	22.1000	3.6891	3.0361

```
mc_ols$outlier$indices
```

```
[1] 7
```

The difference for sample 7 is detected as clearly too large. **Detecting an outlier does not mean it should be deleted** — go back to the original records and experimental procedure, and perform an include/exclude sensitivity analysis when there is sufficient justification.

12.3.8 Prediction

```
predict(mc_ols, candidate = c(40, 100))
```

```
candidate ref_pred
1         40 35.41862
2        100 93.00500
```

```
predict(mc_ols, reference = c(50, 120), inverse = TRUE)
```

```
reference cand_pred
1         50 55.19253
2        120 128.12645
```

12.3.9 Subgroup Comparison

The method performance may differ across lots or sample types. First split by lot, then re-run regression and Bland–Altman on each subgroup:

```

for (b in c("B1", "B2")) {
  sub <- mc_dat[mc_dat$batch == b, ]
  mcs <- mcr(sub, id = "id", candidate = "candidate", reference = "reference")
  mcs <- regression(mcs, method = "ols")
  mcs <- bland_altman(mcs, type = "difference")
  cat(b, ": slope =", round(mcs$regression$slope, 4),
      ", mean diff =", round(mcs$bland_altman$mean_diff, 3), "\n")
}

```

Regression

```

Method: OLS
Intercept = -2.9704 (SE = 2.2333)
Slope      = 0.9582 (SE = 0.0327)
Intercept 95% CI: [-7.6623, 1.7215]
Slope     95% CI: [0.8895, 1.0269]
H0: slope = 1 -> t = -1.2784, p = 0.2173
slope not significantly different from 1 (CI contains 1)
R-squared = 0.9795
Sigma     = 4.2427
n = 20

```

Bland-Altman

```

Type:      difference
Y-axis:    candidate - reference
X-axis:    Mean of candidate and reference
n = 20

```

```

Mean diff:      5.5550
SD diff:        4.3129
95% LoA: [-2.8981, 14.0081]

```

```

95% CI for LoA:
  Lower LoA: [-6.4069, 0.6107]
  Upper LoA: [10.4993, 17.5169]

```

B1 : slope = 0.9582 , mean diff = 5.555

Regression

```

Method: OLS
Intercept = -1.3159 (SE = 2.3598)
Slope      = 0.9502 (SE = 0.0142)
Intercept 95% CI: [-6.2736, 3.6419]
Slope     95% CI: [0.9203, 0.9801]
H0: slope = 1 -> t = -3.4988, p = 0.002564
slope significantly different from 1 (CI excludes 1)
R-squared = 0.9960
Sigma     = 1.8780
n = 20

```

Bland-Altman

```

Type:      difference
Y-axis:    candidate - reference
X-axis:    Mean of candidate and reference
n = 20

```

```

Mean diff:      9.4405
SD diff:        2.3693
95% LoA: [4.7968, 14.0842]

```

95% CI for LoA:

Lower LoA: [2.8693, 6.7244]

Upper LoA: [12.1566, 16.0117]

B2 : slope = 0.9502 , mean diff = 9.44

Similarly, `split()` by sample type and analyze separately. Subgroup-comparison conclusions are used to judge whether method performance is affected by lot/sample type; only pre-defined acceptance criteria can support an acceptability conclusion.

12.3.10 Summary

```
summary(mc_ols)
```

Method Comparison Regression (mcr)

Method Comparison

```
Data class:      data.frame
Total rows:      40
Complete pairs:  40
ID variable:     id
Test method (X): candidate
Reference (Y):   reference
Duplicate IDs:   none
```

Analysis slots:

```
[x] Describe
[x] Correlation (pearson)
[x] Regression (OLS)
[x] Outlier (grubbs)
[x] Bland-Altman (difference)
[x] Bias
```

Descriptive Statistics

Variable	n	Mean	SD	Min	Q1	Median	Q3	Max	NA
candidate (X)	40	112.4977	59.2513	11.5900	63.1075	112.5750	159.5925	209.9000	
reference (Y)	40	105.0000	56.9555	10.0000	57.4750	105.0000	152.5250	200.0000	
Diff (X-Y)	40	7.4977	3.9583	1.2000	5.2725	6.6100	9.7950	22.1000	

batch

Level	Count	Percent
B1	20	50.0%
B2	20	50.0%

sample_type

Level	Count	Percent
plasma	13	32.5%
serum	14	35.0%
urine	13	32.5%

Correlation

Method: pearson

r = 0.9985, 95% CI [0.9971, 0.9992]

p-value: < 2.2e-16
n = 40

Regression

Method: OLS
Intercept = -2.9723 (SE = 1.0971)
Slope = 0.9598 (SE = 0.0087)
Intercept 95% CI: [-5.1933, -0.7513]
Slope 95% CI: [0.9423, 0.9773]
H0: slope = 1 -> t = -4.6494, p = 3.944e-05
slope significantly different from 1 (CI excludes 1)
R-squared = 0.9969
Sigma = 3.2015
n = 40

Outlier

Method: grubbs
Data: Difference (candidate - reference)
Alpha: 0.05

1 outlier(s) found:

Row	Value	Statistic	Critical
7	22.1000	3.6891	3.0361

Bland-Altman

Type: difference
Y-axis: candidate - reference
X-axis: Mean of candidate and reference
n = 40

Mean diff: 7.4977
SD diff: 3.9583
95% LoA: [-0.2603, 15.2558]

95% CI for LoA:

Lower LoA: [-2.4419, 1.9213]
Upper LoA: [13.0742, 17.4374]

Bias

Method: OLS
Interval: both (95%)
MDL points: 3
Bias range: [4.1791, 9.0064]

mdl	bias	ci_lower	ci_upper	pi_lower	pi_upper
30	4.179112	2.407658	5.950566	-2.539698	10.89792
80	6.190463	5.018240	7.362685	-0.395770	12.77670
150	9.006354	7.789153	10.223554	2.411967	15.60074

12.4 Key Points for Interpreting Results

1. **Direction check:** For both regression and Bland–Altman, confirm the direction and bias sign of the candidate relative to the reference.
2. **Correlation agreement:** A high correlation coefficient does not mean the methods agree; focus on the regression slope and the Bland–Altman LoA.
3. **Method selection:** OLS assumes the reference has no error; use Deming/PB when the reference

has error; consider WLS/WDeming when variances are unequal.

4. **Outlier handling:** After detecting an outlier, investigate and perform a sensitivity analysis; do not delete automatically.
5. **Subgroup interpretation:** Judge lot/sample-type splits separately to avoid the mean masking subgroup differences.
6. **Acceptance criteria:** Bias, LoA, and MDL bias must all be compared with the pre-specified allowable limits before drawing an acceptability conclusion.

Chapter 13

Qualitative Analysis

13.1 Analysis Objectives

Qualitative analysis evaluates the degree of agreement between the candidate method and the reference method in positive/negative determinations, corresponding to CLSI EP12. `ivdtools` provides `raw_to_table()` to build a four-fold table from raw paired data, `counts_to_table()` to build one directly from existing TP/FP/TN/FN counts, and three accompanying analysis methods: `diagnostics()` (sensitivity, specificity, PPV, NPV, likelihood ratios, etc.), `kappa()` (Kappa and PABAK), and `mcnemar()` (paired-difference test). Quantitative results can be converted to qualitative with `continuous_to_binary()`.

This document uses three examples to demonstrate the complete analysis for the three data sources. The example data are deterministic teaching data; the positive direction and threshold should be determined according to the protocol.

```
library(ivdtools)
library(readr)
```

13.2 Overview of Functions

Function	Main purpose	Key input or output
<code>raw_to_table()</code>	Build a four-fold table from raw paired data	<code>candidate</code> , <code>reference</code> , <code>positive</code> , <code>na.rm</code>
<code>counts_to_table()</code>	Build a four-fold table from TP/FP/TN/FN	<code>candidate/reference</code> naming
<code>continuous_to_binary()</code>	Convert quantitative to qualitative	<code>cutoff</code> (<code>cutoff</code> is 1)
<code>describe()</code>	Description of raw data	only available for <code>raw_to_table()</code> sources
<code>diagnostics()</code>	Diagnostic accuracy metrics	<code>ci.method</code> , <code>prevalence</code>
<code>kappa()</code>	Kappa / PABAK	<code>prevalence</code> switch
<code>mcnemar()</code>	Paired-difference test	uses exact test when discordant pairs <25

13.3 Example 1: Complete Analysis of Raw Paired Data

13.3.1 Reading and Validating Data

```
qual <- read_csv("./data/qualitative-raw.csv", show_col_types = FALSE)
qual <- as.data.frame(qual)
str(qual)
```

```
'data.frame': 60 obs. of 5 variables:
 $ id      : chr  "S001" "S002" "S003" "S004" ...
 $ new     : chr  "positive" "negative" "negative" "negative" ...
 $ gold    : chr  "positive" "negative" "negative" "negative" ...
 $ age     : num  46.1 44.7 65.7 60 64.6 32.7 47 40.9 38.4 48.1 ...
 $ gender  : chr  "M" "F" "M" "F" ...
```

```
table(qual$gold)
```

```
negative positive
      36       24
```

```
table(qual$new)
```

```
negative positive
      36       24
```

13.3.2 Building the Four-Fold Table

```
qa <- raw_to_table(
  qual,
  candidate = "new",
  reference = "gold",
  id = "id",
  positive = "positive"
)
qa
```

Fourfold Table

```
Source      : raw data
Total n     : 60
Duplicate IDs : none
Candidate   : new
Reference    : gold
```

2x2 Contingency Table

		gold		
		positive	negative	Sum
new	positive	23	1	24
	negative	1	35	36
	sum	24	36	60

Parameter notes: `raw_to_table()` requires that both the candidate and reference

columns have exactly 2 levels; `positive` specifies the positive level (used for level mapping of factor/character columns); `na.rm` controls whether missing rows are deleted. `id` is optional and used for duplicate-sample detection.

13.3.3 Describing the Raw Data

```
qa <- describe(qa)
```

Data Summary

```
Rows:      60
Columns:   5
ID:        60 unique, no duplicates

new
  negative      36 ( 60.0%)
  positive      24 ( 40.0%)
gold
  negative      36 ( 60.0%)
  positive      24 ( 40.0%)
age
  Mean (SD):     51.05 (13.45)
  Median (Q1-Q3): 48.90 (42.15 - 57.28)
  Range:         24.30 - 90.10
gender
  F              32 ( 53.3%)
  M              28 ( 46.7%)
```

`describe()` is only available for `raw_to_table()` sources; it outputs the row count, ID duplicates, per-column summaries, and missingness.

13.3.4 Diagnostic Accuracy Metrics

```
qa <- diagnostics(qa, ci.method = "wilson")
```

Diagnostic Performance Summary

```
Sensitivity      0.958 (0.798, 0.993)
Specificity      0.972 (0.858, 0.995)
PPV              0.958 (0.798, 0.993)
NPV              0.972 (0.858, 0.995)
Accuracy         0.967 (0.886, 0.991)
Prevalence       0.400 (0.286, 0.526)
LR+              34.500 (4.986, 238.724)
LR-              0.043 (0.006, 0.292)
Odds Ratio       805.000 (47.921, 13522.837)
```

```
# PPV & NPV are based on data prevalence of 0.400.
# Confidence intervals for proportions use 'wilson' method.
```

```
qa$diagnostics[c("sensitivity", "specificity", "ppv", "npv", "accuracy")]
```

\$sensitivity

```
      est      lower      upper
0.9583333 0.7975819 0.9926065
```

\$specificity

```
      est      lower      upper
0.9722222 0.8583028 0.9950796
```

```
$ppv
```

```
      est      lower      upper
0.9583333 0.7975819 0.9926065
```

```
$npv
```

```
      est      lower      upper
0.9722222 0.8583028 0.9950796
```

```
$accuracy
```

```
      est      lower      upper
0.9666667 0.8863623 0.9908107
```

In this example, the sensitivity is 0.958 and the specificity is 0.972. If the prevalence in the target population is known, use `prevalence` to recompute PPV/NPV:

```
qa <- diagnostics(qa, prevalence = 0.4)
```

```
Diagnostic Performance Summary
```

```
Sensitivity      0.958 (0.798, 0.993)
Specificity      0.972 (0.858, 0.995)
PPV              0.958 (0.790, 0.993)
NPV              0.972 (0.864, 0.995)
Accuracy         0.967 (0.886, 0.991)
Prevalence       0.400
LR+              34.500 (4.986, 238.724)
LR-              0.043 (0.006, 0.292)
Odds Ratio       805.000 (47.921, 13522.837)
```

```
# PPV & NPV are based on user-specified prevalence of 0.400.
# Confidence intervals for proportions use 'wilson' method.
```

```
qa$diagnostics[c("ppv", "npv")]
```

```
$ppv
```

```
      est      lower      upper
0.9583333 0.7895852 0.9926193
```

```
$npv
```

```
      est      lower      upper
0.9722222 0.8641373 0.9950711
```

Parameter notes: The `ci.method` of `diagnostics()` accepts 7 proportion confidence-interval methods (default "wilson"); when `prevalence` is given, PPV/NPV are recomputed by the Bayes formula; otherwise they are based on the observed prevalence.

13.3.5 Kappa Agreement

```
qa <- kappa(qa)
```

```
Cohen's Kappa
```

```
new vs gold
n = 60
```

```

Kappa          0.9306
SE             0.0483
95% CI         (0.8359, 1.0000)
Observed agreement 0.9667
Expected agreement 0.5200

```

```
qa <- kappa(qa, prevalence = 0.5)
```

PABAK

```

new vs gold
n = 60

```

```

Kappa          0.9333
SE             0.0463
95% CI         (0.8425, 1.0000)
Observed agreement 0.9667
Expected agreement 0.5000
# PABAK (prevalence = 0.500): 2 * p_obs - 1, assumes expected agreement = 0.5

```

Parameter notes: `kappa()` gives Cohen's Kappa; when `prevalence` is given, it instead computes PABAK ($2 \cdot p_{\text{obs}} - 1$). The two handle prevalence bias differently; note the method in the report.

13.3.6 McNemar Test

```
qa <- mcnemar(qa)
```

McNemar Test

```

new vs gold
Discordant pairs: b (pos, neg) = 1, c (neg, pos) = 1
n (b + c)          = 2

```

```

Method: Exact binomial test
P-value:          1.0000
Conclusion: Not significant at 95% confidence level

```

The discordant pairs $b+c = 2 < 25$, so the function automatically uses the exact binomial test.

13.3.7 Summary

```
summary(qa)
```

Qualitative Analysis - summary

Fourfold Table

```

Source          : raw data
Total n         : 60
Duplicate IDs    : none
Candidate       : new
Reference       : gold

```

2x2 Contingency Table

```

-----
gold

```

		positive	negative	Sum
new	positive	23	1	24
	negative	1	35	36
	sum	24	36	60

Analyses performed:

[x] Describe
[x] Diagnostics
[x] Kappa
[x] McNemar

Data Summary

Rows: 60
Columns: 5
ID: 60 unique, no duplicates

new
negative 36 (60.0%)
positive 24 (40.0%)
gold
negative 36 (60.0%)
positive 24 (40.0%)
age
Mean (SD): 51.05 (13.45)
Median (Q1-Q3): 48.90 (42.15 - 57.28)
Range: 24.30 - 90.10
gender
F 32 (53.3%)
M 28 (46.7%)

Diagnostic Performance Summary

Sensitivity 0.958 (0.798, 0.993)
Specificity 0.972 (0.858, 0.995)
PPV 0.958 (0.790, 0.993)
NPV 0.972 (0.864, 0.995)
Accuracy 0.967 (0.886, 0.991)
Prevalence 0.400
LR+ 34.500 (4.986, 238.724)
LR- 0.043 (0.006, 0.292)
Odds Ratio 805.000 (47.921, 13522.837)

PPV & NPV are based on user-specified prevalence of 0.400.

Confidence intervals for proportions use 'wilson' method.

PABAK

new vs gold
n = 60

Kappa 0.9333
SE 0.0463
95% CI (0.8425, 1.0000)
Observed agreement 0.9667
Expected agreement 0.5000
PABAK (prevalence = 0.500): 2 * p_obs - 1, assumes expected agreement = 0.5

McNemar Test

new vs gold

Discordant pairs: b (pos, neg) = 1, c (neg, pos) = 1

n (b + c) = 2

Method: Exact binomial test

P-value: 1.0000

Conclusion: Not significant at 95% confidence level

13.4 Example 2: Building Directly from TP/FP/TN/FN

When counts are already available, no raw data are needed:

```
tab <- counts_to_table(
  tp = 80, fp = 5, tn = 120, fn = 3,
  candidate = "new method",
  reference = "gold standard"
)
tab
```

Fourfold Table

```
Source      : counts
Total n     : 208
Candidate   : new method
Reference    : gold standard
```

2x2 Contingency Table

		gold standard		
		positive	negative	Sum
new method	positive	80	5	85
	negative	3	120	123
	sum	83	125	208

```
tab <- diagnostics(tab)
```

Diagnostic Performance Summary

```
Sensitivity    0.964 (0.899, 0.988)
Specificity    0.960 (0.910, 0.983)
PPV            0.941 (0.870, 0.975)
NPV            0.976 (0.931, 0.992)
Accuracy       0.962 (0.926, 0.980)
Prevalence     0.399 (0.335, 0.467)
LR+            24.096 (10.198, 56.934)
LR-            0.038 (0.012, 0.114)
Odds Ratio     640.000 (148.773, 2753.180)
```

```
# PPV & NPV are based on data prevalence of 0.399.
# Confidence intervals for proportions use 'wilson' method.
```

```
tab <- kappa(tab)
```

Cohen's Kappa

```
new method vs gold standard
n = 208
```

```
Kappa          0.9201
SE              0.0277
95% CI         (0.8659, 0.9744)
Observed agreement 0.9615
Expected agreement 0.5184
```

```
tab <- mcnemar(tab)
```

McNemar Test

```
new method vs gold standard
Discordant pairs: b (pos, neg) = 5, c (neg, pos) = 3
n (b + c)          = 8
```

```
Method: Exact binomial test
P-value:          0.7266
Conclusion: Not significant at 95% confidence level
```

```
summary(tab)
```

Qualitative Analysis - summary

Fourfold Table

```
Source      : counts
Total n     : 208
Candidate   : new method
Reference   : gold standard
```

2x2 Contingency Table

		gold standard		
		positive	negative	Sum
new method	positive	80	5	85
	negative	3	120	123
	sum	83	125	208

Analyses performed:

```
[ ] Describe
[x] Diagnostics
[x] Kappa
[x] McNemar
```

Diagnostic Performance Summary

```
Sensitivity 0.964 (0.899, 0.988)
Specificity 0.960 (0.910, 0.983)
PPV         0.941 (0.870, 0.975)
NPV         0.976 (0.931, 0.992)
Accuracy    0.962 (0.926, 0.980)
Prevalence  0.399 (0.335, 0.467)
```

```

LR+          24.096 (10.198, 56.934)
LR-          0.038 (0.012, 0.114)
Odds Ratio   640.000 (148.773, 2753.180)

```

```

# PPV & NPV are based on data prevalence of 0.399.
# Confidence intervals for proportions use 'wilson' method.

```

Cohen's Kappa

```

new method vs gold standard
n = 208

```

```

Kappa          0.9201
SE             0.0277
95% CI         (0.8659, 0.9744)
Observed agreement 0.9615
Expected agreement 0.5184

```

McNemar Test

```

new method vs gold standard
Discordant pairs: b (pos, neg) = 5, c (neg, pos) = 3
n (b + c)         = 8

```

```

Method: Exact binomial test
P-value:          0.7266
Conclusion: Not significant at 95% confidence level

```

```

describe(tab) # counts sources have no raw data; describe() is unavailable

```

Objects built by `counts_to_table()` have no raw data, so `describe()` is unavailable; `diagnostics()`, `kappa()`, and `mcnemar()` all work normally.

13.5 Example 3: Converting Quantitative Data to Qualitative

13.5.1 Reading Data

The candidate method is a quantitative result `result` and the reference is qualitative `gold`:

```

cont <- read_csv("./data/qualitative-continuous.csv", show_col_types = FALSE)
cont <- as.data.frame(cont)
str(cont)

```

```

'data.frame':  50 obs. of  3 variables:
 $ id      : chr  "S001" "S002" "S003" "S004" ...
 $ result: num  10.11 13.06 1.53 7.78 4.98 ...
 $ gold   : chr  "positive" "positive" "negative" "negative" ...

```

13.5.2 Converting to Binary

Convert to binary at cutoff = 8:

```

cont_bin <- continuous_to_binary(cont, cols = "result", cutoff = 8)
head(cont_bin)

```

```

   id   result   gold result_binary
1 S001 10.111087 positive           1
2 S002 13.057209 positive           1
3 S003  1.534680 negative           0
4 S004  7.782787 negative           0

```

```
5 S005  4.982424 negative          0
6 S006  9.443399 negative          1
```

`continuous_to_binary()` generates a `result_binary` column (cutoff is 1, otherwise 0). To satisfy the positive mapping requirement of `raw_to_table()`, first relabel it with labels consistent with the reference:

```
cont_bin$result_binary_lab <- ifelse(cont_bin$result_binary == 1, "positive", "negative")
```

13.5.3 Building the Four-Fold Table and Analyzing

```
qa3 <- raw_to_table(
  cont_bin,
  candidate = "result_binary_lab",
  reference = "gold",
  id = "id",
  positive = "positive"
)
qa3 <- diagnostics(qa3)
```

Diagnostic Performance Summary

Sensitivity	0.952	(0.773, 0.992)
Specificity	0.897	(0.736, 0.964)
PPV	0.870	(0.679, 0.955)
NPV	0.963	(0.817, 0.993)
Accuracy	0.920	(0.812, 0.968)
Prevalence	0.420	(0.294, 0.558)
LR+	9.206	(3.140, 26.994)
LR-	0.053	(0.008, 0.361)
Odds Ratio	173.333	(16.746, 1794.100)

```
# PPV & NPV are based on data prevalence of 0.420.
# Confidence intervals for proportions use 'wilson' method.
```

```
qa3$diagnostics[c("sensitivity", "specificity", "accuracy")]
```

```
$sensitivity
      est      lower      upper
0.9523810 0.7733064 0.9915440
```

```
$specificity
      est      lower      upper
0.8965517 0.7361492 0.9641851
```

```
$accuracy
      est      lower      upper
0.9200000 0.8116175 0.9684505
```

Note: The level specified by `positive` must exist in both columns, so first relabel the binary column as `positive/negative`. If `positive` is not specified, the function may reverse the positive/negative direction according to the sorted levels, flipping TP/FP; be sure to check.

13.6 Key Points for Interpreting Results

1. **Direction consistent:** Confirm that the positive-direction definitions of the candidate and reference are consistent, to avoid flipping the whole four-fold table.
2. **Levels unique:** Both the candidate and reference columns must have exactly 2 levels; multi-level or continuous columns must first be converted to binary.
3. **Prevalence:** PPV/NPV depend on prevalence; report the observed prevalence or specify the target-population prevalence.
4. **Method labeling:** Kappa and PABAK have different meanings; McNemar uses an exact test for small numbers of discordant pairs; report the method.
5. **Threshold recorded:** Converting quantitative to qualitative must record the cutoff and positive direction; the results hold only under that threshold.

Chapter 14

ROC Curve and Cutoff Determination

14.1 Analysis Objectives

The receiver operating characteristic (ROC) curve evaluates the ability of a quantitative marker to distinguish positive from negative outcomes, and is commonly used to establish a diagnostic cutoff. The `roc()` function in `ivdtools` provides a progressive workflow: `roc()` creates the object → `describe()` → `auc()` (AUC and confidence interval) → `cutoff()` (Youden and closest-corner optimal cutoffs) → `plot()` and `summary()`.

This document demonstrates the complete ROC analysis workflow with a complete single-marker example. The reference variable must be binary; if the original reference is a continuous variable, first convert it to binary (see `continuous_to_binary()` in `Qualitative Analysis`) and record the threshold and positive direction.

```
library(ivdtools)
library(readr)
```

14.2 Overview of Functions

Function	Main purpose	Key input or output
<code>roc()</code> <code>describe()</code>	Create a ROC object Marker descriptive statistics	<code>cols</code> , <code>reference</code> , <code>id</code> , <code>positive</code> reference forced to be treated as categorical
<code>auc()</code> <code>cutoff()</code>	AUC and confidence interval Optimal cutoff	per marker Youden and closest corner; requires <code>auc()</code> first
<code>plot()</code>	ROC curve	<code>cutpoint</code> marks the optimal point
<code>summary()</code>	Summary	

14.3 Example: Single-Marker ROC Analysis

14.3.1 Reading and Validating Data

```
roc_dat <- read_csv("../data/roc-example.csv", show_col_types = FALSE)
roc_dat <- as.data.frame(roc_dat)
str(roc_dat)
```

```
'data.frame': 100 obs. of 3 variables:
 $ sid: num 1 2 3 4 5 6 7 8 9 10 ...
 $ ref: num 0 0 0 0 0 0 0 0 0 0 ...
 $ x1 : num 7.59 8.31 5.89 5.59 9.57 ...
```

```
table(roc_dat$ref)
```

```
0 1
50 50
```

The reference column `ref` is binary 0/1, with 50 positive and 50 negative cases.

14.3.2 Creating the Object and Description

```
ro <- roc(roc_dat, cols = "x1", reference = "ref", id = "sid")
ro
```

ROC Analysis

```
Data class:      data.frame
Total rows:      100
Complete cases:  100
Reference:       ref (binary)
Positive / Neg:  50 / 50
Evaluation cols: 1 (x1)
Duplicate IDs:   none
```

Analysis slots:

```
[ ] Describe
[ ] AUC
[ ] Cutoff
[ ] MLR
```

```
ro <- describe(ro)
```

Descriptive Statistics

Variable	n	Mean	SD	Min	Q1	Median	Q3	Max	NA
x1	100	9.2576	4.4744	-0.3894	5.8224	8.5472	12.9591	18.7295	

ref	
Level	Count Percent
0	50 50.0%
1	50 50.0%

Parameter notes: The `cols` of `roc()` are the quantitative marker columns; `reference` is the binary reference column (either numeric 0/1, a two-level factor, or a two-value character

is accepted); **positive** specifies the positive level (when not given, the last sorted level is taken by default). The direction of each marker (**geq/leq**) is determined automatically from the mean of the positive group.

14.3.3 AUC

```
ro <- auc(ro)
```

AUC Table

Column	AUC	95% CI	Direction
x1	0.9420	[0.8940, 0.9900]	geq

```
ro$auc_table
```

AUC Table

Column	AUC	95% CI	Direction
x1	0.9420	[0.8940, 0.9900]	geq

The AUC of marker x1 is 0.942 (95% CI [0.894, 0.990]), indicating strong discrimination.

14.3.4 Optimal Cutoff

```
ro <- cutoff(ro)
```

Optimal Cutoff Analysis

Column: x1

Youden / Corner: cutoff = 8.7617 (sens = 0.9000, spec = 0.9200, Youden = 0.8200)

Full cutoff table:

Cutoff	Sens	Spec	TP	FP	TN	FN	Youden
-Inf	1.0000	0.0000	50	50	0	0	0.0000
18.7295	0.0200	1.0000	1	0	50	49	0.0200
18.5675	0.0400	1.0000	2	0	50	48	0.0400
18.2248	0.0600	1.0000	3	0	50	47	0.0600
17.7837	0.0800	1.0000	4	0	50	46	0.0800
16.4453	0.1000	1.0000	5	0	50	45	0.1000
15.8893	0.1200	1.0000	6	0	50	44	0.1200
15.8087	0.1400	1.0000	7	0	50	43	0.1400
15.6058	0.1600	1.0000	8	0	50	42	0.1600
15.5003	0.1800	1.0000	9	0	50	41	0.1800
15.2855	0.2000	1.0000	10	0	50	40	0.2000
15.0168	0.2200	1.0000	11	0	50	39	0.2200
14.8639	0.2400	1.0000	12	0	50	38	0.2400
14.6980	0.2600	1.0000	13	0	50	37	0.2600
14.4757	0.2800	1.0000	14	0	50	36	0.2800
14.4469	0.3000	1.0000	15	0	50	35	0.3000
14.4042	0.3200	1.0000	16	0	50	34	0.3200
14.1952	0.3400	1.0000	17	0	50	33	0.3400
14.1938	0.3600	1.0000	18	0	50	32	0.3600
13.9382	0.3800	1.0000	19	0	50	31	0.3800
13.7845	0.4000	1.0000	20	0	50	30	0.4000
13.3718	0.4200	1.0000	21	0	50	29	0.4200

13.0699	0.4400	1.0000	22	0	50	28	0.4400
12.9970	0.4600	1.0000	23	0	50	27	0.4600
12.9854	0.4800	1.0000	24	0	50	26	0.4800
12.9659	0.5000	1.0000	25	0	50	25	0.5000
12.9569	0.5200	1.0000	26	0	50	24	0.5200
12.7812	0.5400	1.0000	27	0	50	23	0.5400
12.7534	0.5600	1.0000	28	0	50	22	0.5600
12.5227	0.5800	1.0000	29	0	50	21	0.5800
12.3440	0.6000	1.0000	30	0	50	20	0.6000
12.2962	0.6200	1.0000	31	0	50	19	0.6200
12.1394	0.6400	1.0000	32	0	50	18	0.6400
12.1354	0.6600	1.0000	33	0	50	17	0.6600
12.0393	0.6800	1.0000	34	0	50	16	0.6800
11.7418	0.6800	0.9800	34	1	49	16	0.6600
11.6642	0.7000	0.9800	35	1	49	15	0.6800
11.5082	0.7000	0.9600	35	2	48	15	0.6600
11.4038	0.7200	0.9600	36	2	48	14	0.6800
11.2133	0.7400	0.9600	37	2	48	13	0.7000
11.0696	0.7600	0.9600	38	2	48	12	0.7200
10.3902	0.7800	0.9600	39	2	48	11	0.7400
10.1719	0.8000	0.9600	40	2	48	10	0.7600
9.8836	0.8200	0.9600	41	2	48	9	0.7800
9.5685	0.8200	0.9400	41	3	47	9	0.7600
9.3526	0.8400	0.9400	42	3	47	8	0.7800
9.1944	0.8600	0.9400	43	3	47	7	0.8000
9.1032	0.8800	0.9400	44	3	47	6	0.8200
8.8558	0.8800	0.9200	44	4	46	6	0.8000
8.7617	0.9000	0.9200	45	4	46	5	0.8200
8.5538	0.9000	0.9000	45	5	45	5	0.8000
8.5406	0.9000	0.8800	45	6	44	5	0.7800
8.5296	0.9200	0.8800	46	6	44	4	0.8000
8.5259	0.9200	0.8600	46	7	43	4	0.7800
8.3125	0.9200	0.8400	46	8	42	4	0.7600
8.2985	0.9200	0.8200	46	9	41	4	0.7400
7.9078	0.9200	0.8000	46	10	40	4	0.7200
7.8493	0.9200	0.7800	46	11	39	4	0.7000
7.6187	0.9200	0.7600	46	12	38	4	0.6800
7.5941	0.9200	0.7400	46	13	37	4	0.6600
7.5556	0.9200	0.7200	46	14	36	4	0.6400
7.5150	0.9200	0.7000	46	15	35	4	0.6200
7.4355	0.9200	0.6800	46	16	34	4	0.6000
7.1837	0.9200	0.6600	46	17	33	4	0.5800
6.8235	0.9400	0.6600	47	17	33	3	0.6000
6.8029	0.9400	0.6400	47	18	32	3	0.5800
6.7765	0.9400	0.6200	47	19	31	3	0.5600
6.6770	0.9400	0.6000	47	20	30	3	0.5400
6.5446	0.9400	0.5800	47	21	29	3	0.5200
6.5440	0.9600	0.5800	48	21	29	2	0.5400
6.1603	0.9600	0.5600	48	22	28	2	0.5200
6.0665	0.9600	0.5400	48	23	27	2	0.5000
6.0554	0.9600	0.5200	48	24	26	2	0.4800
6.0361	0.9600	0.5000	48	25	25	2	0.4600
5.9000	0.9600	0.4800	48	26	24	2	0.4400
5.8857	0.9600	0.4600	48	27	23	2	0.4200
5.6326	0.9600	0.4400	48	28	22	2	0.4000
5.6281	0.9600	0.4200	48	29	21	2	0.3800
5.5851	0.9600	0.4000	48	30	20	2	0.3600
5.5802	0.9600	0.3800	48	31	19	2	0.3400

5.4514	0.9600	0.3600	48	32	18	2	0.3200
5.3207	0.9600	0.3400	48	33	17	2	0.3000
5.1582	0.9600	0.3200	48	34	16	2	0.2800
5.0599	0.9600	0.3000	48	35	15	2	0.2600
5.0167	0.9600	0.2800	48	36	14	2	0.2400
4.5654	0.9600	0.2600	48	37	13	2	0.2200
3.9706	0.9800	0.2600	49	37	13	1	0.2400
3.9552	0.9800	0.2400	49	38	12	1	0.2200
3.9103	1.0000	0.2400	50	38	12	0	0.2400
3.7719	1.0000	0.2200	50	39	11	0	0.2200
3.5881	1.0000	0.2000	50	40	10	0	0.2000
3.4583	1.0000	0.1800	50	41	9	0	0.1800
3.4422	1.0000	0.1600	50	42	8	0	0.1600
2.9673	1.0000	0.1400	50	43	7	0	0.1400
2.9111	1.0000	0.1200	50	44	6	0	0.1200
2.7698	1.0000	0.1000	50	45	5	0	0.1000
2.5203	1.0000	0.0800	50	46	4	0	0.0800
2.3646	1.0000	0.0600	50	47	3	0	0.0600
2.2596	1.0000	0.0400	50	48	2	0	0.0400
0.4733	1.0000	0.0200	50	49	1	0	0.0200
-0.3894	1.0000	0.0000	50	50	0	0	0.0000
Inf	0.0000	1.0000	0	0	50	50	0.0000

```
print(ro$cutoff_table)
```

Optimal Cutoff Analysis

Column: x1

Youden / Corner: cutoff = 8.7617 (sens = 0.9000, spec = 0.9200, Youden = 0.8200)

Full cutoff table:

Cutoff	Sens	Spec	TP	FP	TN	FN	Youden
-Inf	1.0000	0.0000	50	50	0	0	0.0000
18.7295	0.0200	1.0000	1	0	50	49	0.0200
18.5675	0.0400	1.0000	2	0	50	48	0.0400
18.2248	0.0600	1.0000	3	0	50	47	0.0600
17.7837	0.0800	1.0000	4	0	50	46	0.0800
16.4453	0.1000	1.0000	5	0	50	45	0.1000
15.8893	0.1200	1.0000	6	0	50	44	0.1200
15.8087	0.1400	1.0000	7	0	50	43	0.1400
15.6058	0.1600	1.0000	8	0	50	42	0.1600
15.5003	0.1800	1.0000	9	0	50	41	0.1800
15.2855	0.2000	1.0000	10	0	50	40	0.2000
15.0168	0.2200	1.0000	11	0	50	39	0.2200
14.8639	0.2400	1.0000	12	0	50	38	0.2400
14.6980	0.2600	1.0000	13	0	50	37	0.2600
14.4757	0.2800	1.0000	14	0	50	36	0.2800
14.4469	0.3000	1.0000	15	0	50	35	0.3000
14.4042	0.3200	1.0000	16	0	50	34	0.3200
14.1952	0.3400	1.0000	17	0	50	33	0.3400
14.1938	0.3600	1.0000	18	0	50	32	0.3600
13.9382	0.3800	1.0000	19	0	50	31	0.3800
13.7845	0.4000	1.0000	20	0	50	30	0.4000
13.3718	0.4200	1.0000	21	0	50	29	0.4200
13.0699	0.4400	1.0000	22	0	50	28	0.4400
12.9970	0.4600	1.0000	23	0	50	27	0.4600
12.9854	0.4800	1.0000	24	0	50	26	0.4800

12.9659	0.5000	1.0000	25	0	50	25	0.5000
12.9569	0.5200	1.0000	26	0	50	24	0.5200
12.7812	0.5400	1.0000	27	0	50	23	0.5400
12.7534	0.5600	1.0000	28	0	50	22	0.5600
12.5227	0.5800	1.0000	29	0	50	21	0.5800
12.3440	0.6000	1.0000	30	0	50	20	0.6000
12.2962	0.6200	1.0000	31	0	50	19	0.6200
12.1394	0.6400	1.0000	32	0	50	18	0.6400
12.1354	0.6600	1.0000	33	0	50	17	0.6600
12.0393	0.6800	1.0000	34	0	50	16	0.6800
11.7418	0.6800	0.9800	34	1	49	16	0.6600
11.6642	0.7000	0.9800	35	1	49	15	0.6800
11.5082	0.7000	0.9600	35	2	48	15	0.6600
11.4038	0.7200	0.9600	36	2	48	14	0.6800
11.2133	0.7400	0.9600	37	2	48	13	0.7000
11.0696	0.7600	0.9600	38	2	48	12	0.7200
10.3902	0.7800	0.9600	39	2	48	11	0.7400
10.1719	0.8000	0.9600	40	2	48	10	0.7600
9.8836	0.8200	0.9600	41	2	48	9	0.7800
9.5685	0.8200	0.9400	41	3	47	9	0.7600
9.3526	0.8400	0.9400	42	3	47	8	0.7800
9.1944	0.8600	0.9400	43	3	47	7	0.8000
9.1032	0.8800	0.9400	44	3	47	6	0.8200
8.8558	0.8800	0.9200	44	4	46	6	0.8000
8.7617	0.9000	0.9200	45	4	46	5	0.8200
8.5538	0.9000	0.9000	45	5	45	5	0.8000
8.5406	0.9000	0.8800	45	6	44	5	0.7800
8.5296	0.9200	0.8800	46	6	44	4	0.8000
8.5259	0.9200	0.8600	46	7	43	4	0.7800
8.3125	0.9200	0.8400	46	8	42	4	0.7600
8.2985	0.9200	0.8200	46	9	41	4	0.7400
7.9078	0.9200	0.8000	46	10	40	4	0.7200
7.8493	0.9200	0.7800	46	11	39	4	0.7000
7.6187	0.9200	0.7600	46	12	38	4	0.6800
7.5941	0.9200	0.7400	46	13	37	4	0.6600
7.5556	0.9200	0.7200	46	14	36	4	0.6400
7.5150	0.9200	0.7000	46	15	35	4	0.6200
7.4355	0.9200	0.6800	46	16	34	4	0.6000
7.1837	0.9200	0.6600	46	17	33	4	0.5800
6.8235	0.9400	0.6600	47	17	33	3	0.6000
6.8029	0.9400	0.6400	47	18	32	3	0.5800
6.7765	0.9400	0.6200	47	19	31	3	0.5600
6.6770	0.9400	0.6000	47	20	30	3	0.5400
6.5446	0.9400	0.5800	47	21	29	3	0.5200
6.5440	0.9600	0.5800	48	21	29	2	0.5400
6.1603	0.9600	0.5600	48	22	28	2	0.5200
6.0665	0.9600	0.5400	48	23	27	2	0.5000
6.0554	0.9600	0.5200	48	24	26	2	0.4800
6.0361	0.9600	0.5000	48	25	25	2	0.4600
5.9000	0.9600	0.4800	48	26	24	2	0.4400
5.8857	0.9600	0.4600	48	27	23	2	0.4200
5.6326	0.9600	0.4400	48	28	22	2	0.4000
5.6281	0.9600	0.4200	48	29	21	2	0.3800
5.5851	0.9600	0.4000	48	30	20	2	0.3600
5.5802	0.9600	0.3800	48	31	19	2	0.3400
5.4514	0.9600	0.3600	48	32	18	2	0.3200
5.3207	0.9600	0.3400	48	33	17	2	0.3000
5.1582	0.9600	0.3200	48	34	16	2	0.2800

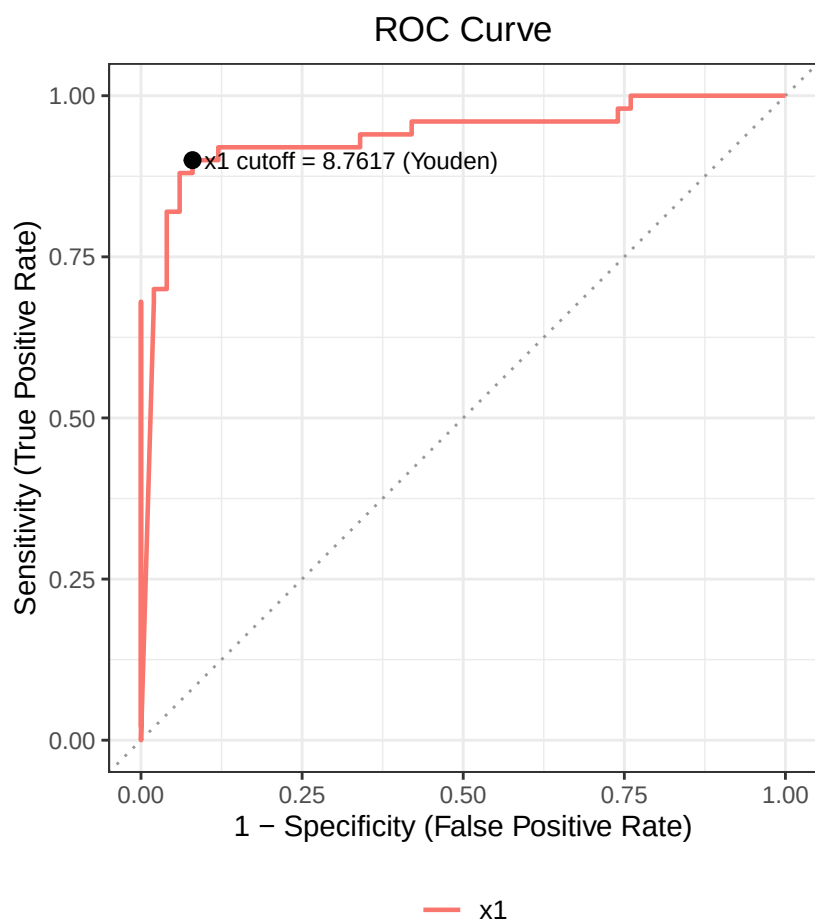
5.0599	0.9600	0.3000	48	35	15	2	0.2600
5.0167	0.9600	0.2800	48	36	14	2	0.2400
4.5654	0.9600	0.2600	48	37	13	2	0.2200
3.9706	0.9800	0.2600	49	37	13	1	0.2400
3.9552	0.9800	0.2400	49	38	12	1	0.2200
3.9103	1.0000	0.2400	50	38	12	0	0.2400
3.7719	1.0000	0.2200	50	39	11	0	0.2200
3.5881	1.0000	0.2000	50	40	10	0	0.2000
3.4583	1.0000	0.1800	50	41	9	0	0.1800
3.4422	1.0000	0.1600	50	42	8	0	0.1600
2.9673	1.0000	0.1400	50	43	7	0	0.1400
2.9111	1.0000	0.1200	50	44	6	0	0.1200
2.7698	1.0000	0.1000	50	45	5	0	0.1000
2.5203	1.0000	0.0800	50	46	4	0	0.0800
2.3646	1.0000	0.0600	50	47	3	0	0.0600
2.2596	1.0000	0.0400	50	48	2	0	0.0400
0.4733	1.0000	0.0200	50	49	1	0	0.0200
-0.3894	1.0000	0.0000	50	50	0	0	0.0000
Inf	0.0000	1.0000	0	0	50	50	0.0000

`cutoff()` gives all candidate cutoffs with their corresponding sensitivity, specificity, and Youden index, and marks the optimal one with two criteria: **Youden** (maximizing $Se + Sp - 1$) and **closest corner** (minimizing the distance to (0,1)). In this example, the Youden/Corner optimal cutoff is 8.76 (sensitivity 0.90, specificity 0.92, Youden 0.82).

Parameter notes: `cutoff()` must be executed after `auc()`; `plot(cutpoint=...)` requires `cutoff()` to have been run. `cutpoint` can be "Youden", "Corner", "all", or NULL.

14.3.5 Graphics and Summary

```
plot(ro, cutpoint = "Youden")
```



```
summary(ro)
```

ROC Analysis -- Summary

ROC Analysis

```
Data class:      dataframe
Total rows:      100
Complete cases:  100
Reference:        ref (binary)
Positive / Neg:   50 / 50
Evaluation cols:  1 (x1)
Duplicate IDs:    none
```

Analysis slots:

```
[x] Describe
[x] AUC
[ ] Cutoff
[ ] MLR
```

Descriptive Statistics

Variable	n	Mean	SD	Min	Q1	Median	Q3	Max	NA
x1	100	9.2576	4.4744	-0.3894	5.8224	8.5472	12.9591	18.7295	

ref	
Level	Count Percent

```

-----
0          50    50.0%
1          50    50.0%

```

AUC Table

```

-----
Column          AUC          95% CI  Direction
-----
x1              0.9420  [0.8940, 0.9900]      geq

```

Optimal Cutoff Analysis

Column: x1

Youden / Corner: cutoff = 8.7617 (sens = 0.9000, spec = 0.9200, Youden = 0.8200)

Full cutoff table:

Cutoff	Sens	Spec	TP	FP	TN	FN	Youden
-Inf	1.0000	0.0000	50	50	0	0	0.0000
18.7295	0.0200	1.0000	1	0	50	49	0.0200
18.5675	0.0400	1.0000	2	0	50	48	0.0400
18.2248	0.0600	1.0000	3	0	50	47	0.0600
17.7837	0.0800	1.0000	4	0	50	46	0.0800
16.4453	0.1000	1.0000	5	0	50	45	0.1000
15.8893	0.1200	1.0000	6	0	50	44	0.1200
15.8087	0.1400	1.0000	7	0	50	43	0.1400
15.6058	0.1600	1.0000	8	0	50	42	0.1600
15.5003	0.1800	1.0000	9	0	50	41	0.1800
15.2855	0.2000	1.0000	10	0	50	40	0.2000
15.0168	0.2200	1.0000	11	0	50	39	0.2200
14.8639	0.2400	1.0000	12	0	50	38	0.2400
14.6980	0.2600	1.0000	13	0	50	37	0.2600
14.4757	0.2800	1.0000	14	0	50	36	0.2800
14.4469	0.3000	1.0000	15	0	50	35	0.3000
14.4042	0.3200	1.0000	16	0	50	34	0.3200
14.1952	0.3400	1.0000	17	0	50	33	0.3400
14.1938	0.3600	1.0000	18	0	50	32	0.3600
13.9382	0.3800	1.0000	19	0	50	31	0.3800
13.7845	0.4000	1.0000	20	0	50	30	0.4000
13.3718	0.4200	1.0000	21	0	50	29	0.4200
13.0699	0.4400	1.0000	22	0	50	28	0.4400
12.9970	0.4600	1.0000	23	0	50	27	0.4600
12.9854	0.4800	1.0000	24	0	50	26	0.4800
12.9659	0.5000	1.0000	25	0	50	25	0.5000
12.9569	0.5200	1.0000	26	0	50	24	0.5200
12.7812	0.5400	1.0000	27	0	50	23	0.5400
12.7534	0.5600	1.0000	28	0	50	22	0.5600
12.5227	0.5800	1.0000	29	0	50	21	0.5800
12.3440	0.6000	1.0000	30	0	50	20	0.6000
12.2962	0.6200	1.0000	31	0	50	19	0.6200
12.1394	0.6400	1.0000	32	0	50	18	0.6400
12.1354	0.6600	1.0000	33	0	50	17	0.6600
12.0393	0.6800	1.0000	34	0	50	16	0.6800
11.7418	0.6800	0.9800	34	1	49	16	0.6600
11.6642	0.7000	0.9800	35	1	49	15	0.6800
11.5082	0.7000	0.9600	35	2	48	15	0.6600
11.4038	0.7200	0.9600	36	2	48	14	0.6800
11.2133	0.7400	0.9600	37	2	48	13	0.7000

11.0696	0.7600	0.9600	38	2	48	12	0.7200
10.3902	0.7800	0.9600	39	2	48	11	0.7400
10.1719	0.8000	0.9600	40	2	48	10	0.7600
9.8836	0.8200	0.9600	41	2	48	9	0.7800
9.5685	0.8200	0.9400	41	3	47	9	0.7600
9.3526	0.8400	0.9400	42	3	47	8	0.7800
9.1944	0.8600	0.9400	43	3	47	7	0.8000
9.1032	0.8800	0.9400	44	3	47	6	0.8200
8.8558	0.8800	0.9200	44	4	46	6	0.8000
8.7617	0.9000	0.9200	45	4	46	5	0.8200
8.5538	0.9000	0.9000	45	5	45	5	0.8000
8.5406	0.9000	0.8800	45	6	44	5	0.7800
8.5296	0.9200	0.8800	46	6	44	4	0.8000
8.5259	0.9200	0.8600	46	7	43	4	0.7800
8.3125	0.9200	0.8400	46	8	42	4	0.7600
8.2985	0.9200	0.8200	46	9	41	4	0.7400
7.9078	0.9200	0.8000	46	10	40	4	0.7200
7.8493	0.9200	0.7800	46	11	39	4	0.7000
7.6187	0.9200	0.7600	46	12	38	4	0.6800
7.5941	0.9200	0.7400	46	13	37	4	0.6600
7.5556	0.9200	0.7200	46	14	36	4	0.6400
7.5150	0.9200	0.7000	46	15	35	4	0.6200
7.4355	0.9200	0.6800	46	16	34	4	0.6000
7.1837	0.9200	0.6600	46	17	33	4	0.5800
6.8235	0.9400	0.6600	47	17	33	3	0.6000
6.8029	0.9400	0.6400	47	18	32	3	0.5800
6.7765	0.9400	0.6200	47	19	31	3	0.5600
6.6770	0.9400	0.6000	47	20	30	3	0.5400
6.5446	0.9400	0.5800	47	21	29	3	0.5200
6.5440	0.9600	0.5800	48	21	29	2	0.5400
6.1603	0.9600	0.5600	48	22	28	2	0.5200
6.0665	0.9600	0.5400	48	23	27	2	0.5000
6.0554	0.9600	0.5200	48	24	26	2	0.4800
6.0361	0.9600	0.5000	48	25	25	2	0.4600
5.9000	0.9600	0.4800	48	26	24	2	0.4400
5.8857	0.9600	0.4600	48	27	23	2	0.4200
5.6326	0.9600	0.4400	48	28	22	2	0.4000
5.6281	0.9600	0.4200	48	29	21	2	0.3800
5.5851	0.9600	0.4000	48	30	20	2	0.3600
5.5802	0.9600	0.3800	48	31	19	2	0.3400
5.4514	0.9600	0.3600	48	32	18	2	0.3200
5.3207	0.9600	0.3400	48	33	17	2	0.3000
5.1582	0.9600	0.3200	48	34	16	2	0.2800
5.0599	0.9600	0.3000	48	35	15	2	0.2600
5.0167	0.9600	0.2800	48	36	14	2	0.2400
4.5654	0.9600	0.2600	48	37	13	2	0.2200
3.9706	0.9800	0.2600	49	37	13	1	0.2400
3.9552	0.9800	0.2400	49	38	12	1	0.2200
3.9103	1.0000	0.2400	50	38	12	0	0.2400
3.7719	1.0000	0.2200	50	39	11	0	0.2200
3.5881	1.0000	0.2000	50	40	10	0	0.2000
3.4583	1.0000	0.1800	50	41	9	0	0.1800
3.4422	1.0000	0.1600	50	42	8	0	0.1600
2.9673	1.0000	0.1400	50	43	7	0	0.1400
2.9111	1.0000	0.1200	50	44	6	0	0.1200
2.7698	1.0000	0.1000	50	45	5	0	0.1000
2.5203	1.0000	0.0800	50	46	4	0	0.0800
2.3646	1.0000	0.0600	50	47	3	0	0.0600

2.2596	1.0000	0.0400	50	48	2	0	0.0400
0.4733	1.0000	0.0200	50	49	1	0	0.0200
-0.3894	1.0000	0.0000	50	50	0	0	0.0000
Inf	0.0000	1.0000	0	0	50	50	0.0000

14.4 Key Points for Interpreting Results

1. **Reference reliable:** ROC conclusions depend on the reference standard; errors in the reference propagate to the AUC and cutoff.
2. **Direction confirmed:** Confirm that the marker's judgment direction (geq/leq) is clinically meaningful.
3. **Cutoff selection:** Youden and closest corner are two criteria; the final cutoff should be determined in light of clinical costs (false-negative/false-positive weights) and the protocol, not only statistical optimality.
4. **Sample size:** The AUC confidence interval varies with sample size and event count; interpret cautiously with small samples or too few events.
5. **Method limitations:** This workflow is an empirical ROC/trapezoidal AUC; paired multi-curve comparison, partial AUC, etc., require separate designs.
6. **Independent validation:** The AUC on the training data is the apparent performance; it should be validated on independent data before generalization.

Chapter 15

ROC Curve and Multivariate Regression

15.1 Analysis Objectives

The diagnostic ability of a single marker is often limited; combining multiple markers can improve the overall discrimination. Besides single-marker ROC, the `roc()` function in `ivdtools` also supports combining multiple markers into a joint score with multivariate logistic regression (`mlr()`), computing its AUC, and predicting the positive probability for new samples.

This document demonstrates the joint analysis of two markers (`x1`, `x2`): first compare the single-marker AUCs, then fit a multivariate logistic regression model, draw the multi-curve ROC, and predict new samples. The AUC of the joint model on the training data is the **apparent performance** and should be validated on independent data before generalization; the reference variable must be binary.

```
library(ivdtools)
library(readr)
```

15.2 Overview of Functions

Function	Main purpose	Key input or output
<code>roc()</code>	Create a ROC object	<code>cols</code> supports multiple markers per column
<code>auc()</code>	Single-marker AUC	
<code>mlr()</code>	Multivariate logistic regression	<code>cols</code> , <code>name</code> ; requires <code>auc()</code> first
<code>plot()</code>	ROC curve	<code>mlr="all"</code> overlays the joint-model curve
<code>predict()</code>	Predict positive probability	<code>newdata</code> , <code>column_map</code>
<code>summary()</code>	Summary	

15.3 Example: Joint Multi-Marker Analysis

15.3.1 Reading and Validating Data

```
mlr_dat <- read_csv("./data/roc-multimarker.csv", show_col_types = FALSE)
mlr_dat <- as.data.frame(mlr_dat)
str(mlr_dat)
```

```
'data.frame': 120 obs. of 4 variables:
 $ sid: num 1 2 3 4 5 6 7 8 9 10 ...
```

```
$ ref: num 0 0 0 0 0 0 0 0 0 0 ...
$ x1 : num 5.51 3.19 8.13 6.16 11.12 ...
$ x2 : num 8.3 12.9 12.4 16.1 11.9 ...
```

```
table(mlr_dat$ref)
```

```
0 1
60 60
```

The data contain 120 cases (60 positive and 60 negative) and two quantitative markers, x1 and x2.

15.3.2 Creating the Object and Single-Marker AUC

```
ro2 <- roc(mlr_dat, cols = c("x1", "x2"), reference = "ref", id = "sid")
ro2 <- auc(ro2)
```

AUC Table

Column	AUC	95% CI	Direction
x1	0.9219	[0.8712, 0.9727]	geq
x2	0.8322	[0.7587, 0.9057]	geq

```
ro2$auc_table
```

AUC Table

Column	AUC	95% CI	Direction
x1	0.9219	[0.8712, 0.9727]	geq
x2	0.8322	[0.7587, 0.9057]	geq

The AUC of x1 is 0.922 and that of x2 is 0.832; among the single markers, x1 discriminates better.

15.3.3 Multivariate Logistic Regression

Model jointly with both markers:

```
ro2 <- mlr(ro2, cols = c("x1", "x2"), name = "combined")
```

Multivariate Logistic Regression (combined)

```
-----
Formula: reference_binary ~ x1 + x2
n = 120 (positive = 60, negative = 60)
AUC = 0.9642 [0.9298, 0.9985]
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-13.6586717	2.7569949	-4.954188	7.263286e-07
x1	0.7638502	0.1549103	4.930919	8.184356e-07
x2	0.3949617	0.1064673	3.709700	2.075049e-04

* Note: AUC is evaluated on the same data used for fitting;
the estimate may be optimistic.

```
ro2$mlr_results$combined$fit_summary
```

```

Call:
glm(formula = formula, family = binomial, data = model_data)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -13.6587      2.7570  -4.954 7.26e-07 ***
x1           0.7639      0.1549   4.931 8.18e-07 ***
x2           0.3950      0.1065   3.710 0.000208 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 166.355  on 119  degrees of freedom
Residual deviance:  58.112  on 117  degrees of freedom
AIC: 64.112

Number of Fisher Scoring iterations: 7

```

```
ro2$mlr_results$combined$auc
```

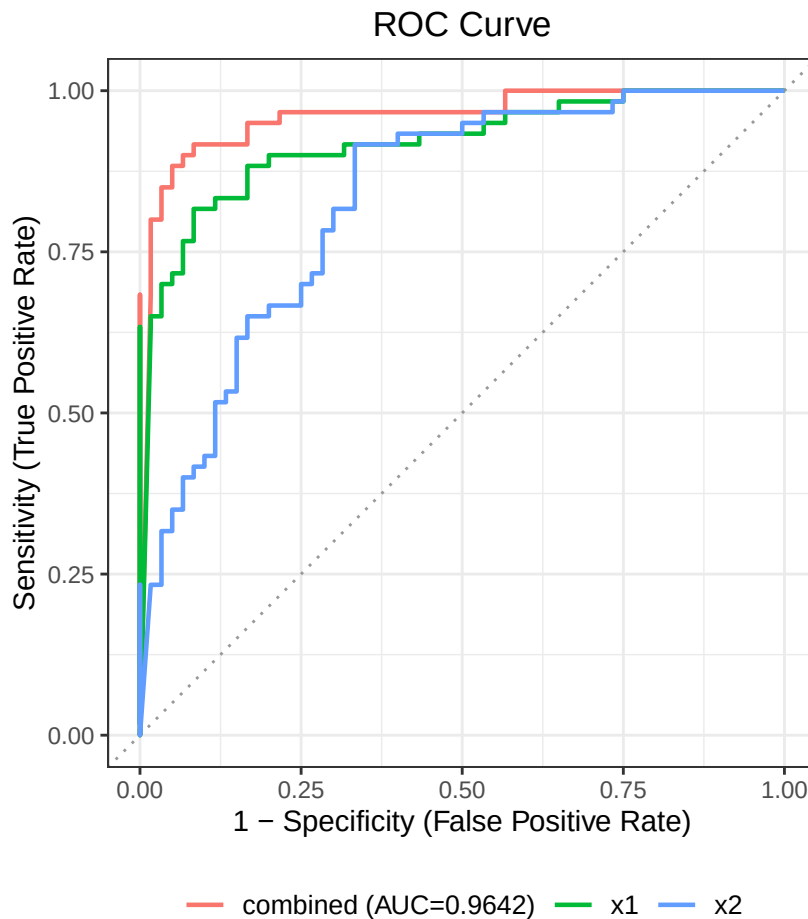
```
[1] 0.9641667
```

The joint model has an AUC of 0.964 (95% CI [0.930, 0.999]), higher than either single marker, indicating that combining the two markers does provide a gain.

Parameter notes: `mlr(cols, name, ...)` fits a binary logistic regression (`glm`) with the selected markers; `name` is the model name (auto-generates `MLR01`, `MLR02`, ...); ... is passed through to `glm`. **`auc()` must be run first** before calling `mlr()`. The coefficients are interpreted as log odds ratios; the joint AUC is computed on the data used for fitting and is an apparent performance.

15.3.4 Multi-Curve ROC Plot

```
plot(ro2, mlr = "all")
```



`plot(ro2, mlr="all")` draws the ROC curves of each single marker and the joint model together for comparison.

15.3.5 Predicting New Samples

```
predict(ro2, newdata = data.frame(x1 = c(7, 12), x2 = c(16, 21)))
```

	x1	x2	pred_prob	pred_se	ci_lower	ci_upper	pred_class
1	7	16	0.1200103	0.05313595	0.04840777	0.2677265	0
2	12	21	0.9781557	0.01751040	0.89984790	0.9955390	1

`predict()` returns each predictor variable, the positive probability (`pred_prob`), the standard error and confidence interval, and the predicted class (`pred_class`) at the default threshold of 0.5. The two cases obtain positive probabilities of 0.12 and 0.98, respectively, consistent with the expected direction.

Parameter notes: When the column names of `newdata` differ from those used in modeling, map them with the named vector `column_map` (for example `column_map = c(new_x1 = "x1")`). If the object contains multiple models, `predict()` must specify the model name with `mlr`.

15.3.6 Summary

```
summary(ro2)
```

ROC Analysis -- Summary

ROC Analysis

```
Data class:      data.frame
Total rows:      120
Complete cases:  120
Reference:       ref (binary)
Positive / Neg:  60 / 60
Evaluation cols: 2  (x1, x2)
Duplicate IDs:   none
```

Analysis slots:

```
[ ] Describe
[x] AUC
[ ] Cutoff
[x] MLR
```

AUC Table

Column	AUC	95% CI	Direction
x1	0.9219	[0.8712, 0.9727]	geq
x2	0.8322	[0.7587, 0.9057]	geq

MLR Results

Multivariate Logistic Regression (combined)

```
Formula: reference_binary ~ x1 + x2
n = 120 (positive = 60, negative = 60)
AUC = 0.9642 [0.9298, 0.9985]
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-13.6586717	2.7569949	-4.954188	7.263286e-07
x1	0.7638502	0.1549103	4.930919	8.184356e-07
x2	0.3949617	0.1064673	3.709700	2.075049e-04

* Note: AUC is evaluated on the same data used for fitting;
the estimate may be optimistic.

15.4 Key Points for Interpreting Results

1. **Joint gain:** A joint AUC higher than each single marker only indicates that the linear combination provides a gain; whether it is a significant improvement should be judged with the confidence interval and clinical decision needs.
2. **Apparent performance:** The AUC on the training data is an optimistic estimate; it must be re-estimated on an independent validation set to avoid overfitting misjudgment.
3. **Collinearity and separation:** When markers are highly correlated or perfectly separated, logistic regression may be unstable or fail to converge.
4. **Event count:** A multivariate model needs a sufficient number of events per parameter; use caution when events are too few.
5. **Cutoff and decision:** The final judgment threshold of the joint score should combine clinical costs and the protocol, not be limited to the default 0.5.
6. **Variable selection:** `mlr()` fits a given set of markers in one call; comparisons of multiple variable sets should be pre-specified in the protocol to avoid repeated trial and error.

Chapter 16

Introduction to Algorithm Principles

16.1 Introduction

This document summarizes the algorithm principles underlying `ivdtools`, to support understanding of the statistical basis of each analysis module and the mapping with the package interface. It covers outliers/distribution, method comparison/qualitative evaluation, ROC, precision/ANOVA/reference intervals, and QC/stability/equation fitting. For ease of cross-reference, method comparison uniformly adopts the convention of candidate X , reference Y , and regression model $Y = a + bX$; the specific parameter names and directions in the package are governed by the registries such as `list_equation()`, `list_sadler()`, and `list_westgard()` in `ivdtools`.

The formulas are used to illustrate the principles; they do not replace textbook derivations, nor do they replace the judgment about method selection in the protocol.

16.2 General Conventions and Notation

Ordered samples $x_{(1)} \leq \dots \leq x_{(n)}$, mean $\bar{x}$, standard deviation s , significance level $\alpha = 1 - \text{level}$. Most tests require approximately independent samples; a detected “outlier” or “failure to follow some distribution” only indicates the strength of conflict with the assumed model, cannot quantify its actual effect on the final performance estimate, and does not automatically trigger deletion.

16.3 Outlier and Distribution Algorithms

16.3.1 Outliers

- **Grubbs**: $G = \max_i |x_i - \bar{x}|/s$, judged against a critical value derived from the t distribution for the most extreme point; approximately normal and applies to one point at a time. Mapping: `outliers_test(method="grubbs", alpha=...)`.
- **Generalized ESD**: successively removes the most extreme point, computes R_i and compares it with λ_i that changes with the remaining n ; the maximum number of outliers is limited by an additional parameter.
- **Dixon**: uses the ratio of the endpoint gap to the sample range; the critical value depends on n and is suitable only for a small-sample range.
- **IQR**: marks values less than $Q_1 - kIQR$ or greater than $Q_3 + kIQR$; the default coefficient is determined by the interface implementation; this is a rule rather than a significance test.

`outliers_test()` first excludes missing values and retains the original row numbers, then returns the statistic, critical value, and outlier flag. A constant column makes statistics with s or the range as the denominator undefined; multiple testing increases false positives.

16.3.2 Normality

- **Shapiro–Wilk**: $W = (\sum a_i x_{(i)})^2 / \sum (x_i - \bar{x})^2$, with weights from the normal order statistics.
- **Anderson–Darling**: the weighted squared distance between the empirical distribution and the fitted normal CDF, with greater weight on the tails.
- **Lilliefors**: a Kolmogorov–Smirnov-type statistic when parameters are estimated from the sample.
- **Cramér–von Mises**: the overall integrated squared distance between the empirical CDF and the theoretical CDF.

`normal_test(method="auto")` selects an available test based on the sample conditions (Shapiro–Wilk for $n \leq 50$, Anderson–Darling for larger samples), and `level` maps to the rejection threshold. Graphics provide Q-Q and histogram; the p-value cannot quantify the actual effect of the deviation on the performance estimate.

16.4 Method Comparison and Qualitative Evaluation Algorithms

16.4.1 Regression, Correlation, and Bias

The unified direction is candidate X , reference Y , model $Y = a + bX$.

- **OLS** minimizes $\sum (y_i - a - bx_i)^2$; assumes the X error is negligible.
- **Deming** minimizes the weighted orthogonal distance to the line; the error variance ratio maps to `lambda`; the direction of $\lambda = \sigma_Y^2 / \sigma_X^2$ must be checked against the source code or protocol.
- **Passing–Bablok** estimates b from the median of all pairwise slopes, then estimates a from the median of $y_i - bx_i$; it is robust to outliers but still requires linearity and independent samples.
- **Pearson** measures linear correlation; **Spearman/Kendall** measure rank association; `correlation(method=...)` answers “correlation” rather than “agreement” and cannot replace bias and agreement-limit evaluation.

The absolute Bland–Altman difference is recorded as $d_i = X_i - Y_i$ (the report must re-check the actual direction in the package object), with mean difference $\bar{d}$ and agreement limits $\bar{d} \pm z_{(1+p)/2} s_d$; relative BA uses percentage differences. `agree.level` controls the coverage proportion and `conf.level` controls the estimation interval. `bias()` maps the regression equation to the specified `level` and outputs the predicted bias and confidence/prediction intervals.

16.4.2 Four-Fold Table

From TP, FP, TN, FN:

$$Se = \frac{TP}{TP + FN}, \quad Sp = \frac{TN}{TN + FP}, \quad PPV = \frac{TP}{TP + FP}.$$

The Wilson interval is inverted from a score test; the exact interval is inverted from binomial tail probabilities. Cohen $\kappa = (P_o - P_e) / (1 - P_e)$, where P_e is obtained from the marginal proportions. McNemar uses only the discordant cells b and c , with the asymptotic statistic approximately $(b - c)^2 / (b + c)$ (with the continuity correction or exact version as recorded in the object). The package interfaces map to `diagnostics()`, `kappa()`, and `mcnemar()`; zero denominators, small cell counts, and extreme prevalence all require explicit explanation.

16.5 ROC Algorithms

For each threshold t , compute $TPR(t) = Se(t)$ and $FPR(t) = 1 - Sp(t)$ according to the marker direction. The empirical ROC treats all observations as candidate thresholds and connects them as a step/broken line; the AUC is summed by the trapezoidal rule over adjacent points:

$$AUC = \sum_i \frac{TPR_i + TPR_{i+1}}{2} (FPR_{i+1} - FPR_i).$$

`auc(roc_obj, cols=...)` saves the per-marker AUC table; the empirical AUC can be interpreted as the probability that a random positive score exceeds a random negative score (after direction adjustment). Sample-correlated multi-curve comparison, partial AUC, or complex sampling are not automatically resolved by this point estimate.

The Youden index $J(t) = Se(t) + Sp(t) - 1$; the closest-corner method minimizes $\sqrt{(1 - Se)^2 + (1 - Sp)^2}$. `cutoff()` outputs both types of optimal points. Tied thresholds, measurement resolution, misclassification costs, and clinical prevalence change the actual choice, so “mathematically optimal” is not an automatic clinical cutoff.

The multi-marker model is

$$\text{logit}\{P(Y = 1)\} = \beta_0 + \sum_j \beta_j x_j,$$

fitted by binomial logistic regression via `mlr(cols, name)`; `predict()` transforms the linear predictor into a probability through the logistic inverse, classifies at the default 0.5, and computes intervals using the model covariance. Perfect separation, collinearity, missingness, and small event counts lead to instability. Re-drawing the ROC on the training set is an apparent performance; development and validation data must be distinguished.

16.6 Precision, ANOVA, and Reference Interval Algorithms

16.6.1 Variance Components

A nested design can be written as $y_{ijkl} = \mu + D_i + R_{j(i)} + \varepsilon_{k(ij)}$, with total variance $\sigma_T^2 = \sigma_D^2 + \sigma_R^2 + \sigma_e^2$. `precision(form = y ~ day/run)` passes the formula to `VCA::anovaVCA()`; `variance()/vc()` summarize the VC, $SD = \sqrt{VC}$, $CV = 100SD/\bar{y}$, and proportions. Satterthwaite uses

$$\nu \approx \frac{(\sum c_j MS_j)^2}{\sum (c_j MS_j)^2 / \nu_j}$$

to approximate the effective degrees of freedom, then constructs variance/SD intervals from chi-square quantiles. Negative components, low degrees of freedom, and imbalance degrade the approximation.

The Sadler precision profile expresses the variance or SD as a function of concentration; `list_sadler()` lists 10 candidate forms, and `profile(model.no=...)` calls the VFP fit and selects by AIC. Different models extrapolate very differently at low concentrations; the concentration coverage and residuals must be checked.

16.6.2 ANOVA and Reference Intervals

`bottle_anova()` uses `stats::aov()` to decompose the between-group/within-group sums of squares; Bartlett tests variance homogeneity, the residual Shapiro–Wilk checks normality; Tukey HSD uses the studentized range to control the family-wise error rate of all pairwise comparisons at once.

The parametric reference interval is approximated as $\bar{x} \pm z_{1-\alpha/2}s$ under the normality assumption; the nonparametric percentile method uses R quantile type 6, with endpoints from the order statistics and endpoint intervals computed by binomial ordering or bootstrap paths. `interval` is the coverage and `ci` is the endpoint confidence; the two have different meanings. Sample selection bias cannot be repaired by any interval algorithm.

16.7 QC, Stability, and Equation Fitting Algorithms

16.7.1 QC and Stability

The Levey–Jennings standardization is $z_i = (x_i - \mu_0)/\sigma_0$; the Westgard rules combine these z values across single points, consecutive same-side/same-direction runs, or multiple levels in the same batch. `qc_chart()` evaluates in run order, and `youden_plot()` uses the target means/SDs of two control

materials to form quadrant and ellipse-style judgments; the order, target values, and lot boundaries are inputs to the algorithm.

The node relative bias is $100(x_t - x_0)/x_0$ (relative) or $x_t - x_0$ (absolute). `stability_regression()` fits a time trend, and `stability_time()` finds the intersection of the confidence boundary/trend with the allowable limit. MKT is the Arrhenius-weighted temperature history:

$$T_k = -\frac{E_a}{R} \left[\log \left\{ \sum_i w_i \exp(-E_a/RT_i) \right\} \right]^{-1}.$$

The Arrhenius relationship is $k = Ae^{-E_a/(RT)}$; zero-order is $C_t = C_0 - kt$, first-order is $\log C_t = \log C_0 - kt$. `mkt(ea=83.144)`, `arrhenius(order, target_temp, limit)` map these quantities. Temperature must be converted to Kelvin for the formulas, and extrapolation is limited by the assumption of an unchanged reaction mechanism.

16.7.2 Equation Fitting

Linear/polynomial are estimated by least squares; exponential and 4PL/5PL use `nlsLM`/constrained optimization. A typical 4PL is

$$y = d + \frac{a - d}{1 + (x/c)^b},$$

and 5PL adds an asymmetry exponent g , but the specific parameter names and directions in the package are governed by `list_equation()`. Weighted fitting minimizes $\sum w_i (y_i - f(x_i, \theta))^2$. AIC approximates $-2 \log L + 2k$; residuals reveal heteroscedasticity, curvature, and anomalous points.

The mean confidence interval reflects the uncertainty of $E(Y|X)$, while the prediction interval also includes single-observation error. Nonlinear inverse prediction requires numerical root-finding; small signal errors near a flat plateau can translate into enormous concentration errors. Starting values, bounds, weights, equation versions, convergence codes, and failed models should all enter the audit record.

16.8 Module-to-Guideline Mapping

Topic	Main functions	Corresponding CLSI guideline
Outliers and normality	<code>outliers_test()</code> , <code>normal_test()</code>	General (pre-analysis steps of the EP guidelines)
Method comparison	<code>mcr()</code> series	EP09
Qualitative evaluation	<code>fourfold_table</code> series	EP12
ROC	<code>roc()</code> series	EP24 (reference)
Precision	<code>precision()</code> series	EP05
Bottle/lot ANOVA	<code>bottle_anova()</code>	EP15
Reference intervals	<code>reference_interval()</code>	EP28-A3c
Stability	<code>stability_*</code> (), <code>arrhenius()</code> , <code>mkt()</code>	EP25
QC	<code>qc_chart()</code> , <code>youden_plot()</code>	Westgard rules
Equation fitting	<code>fit_equation()</code> series	EP06
Analytical specificity	<code>lob_lod_loq()</code>	EP17

16.9 Scope of Use

- Formulas and statistics are valid only under reasonable assumptions; when assumptions are not met, the results cannot be applied mechanically.

- “Statistically significant” and “mathematically optimal” are not the same as “clinically/product acceptable”; acceptance criteria must be defined before analysis.
- Performance on training data (such as ROC AUC, regression goodness of fit) is apparent performance; it should be validated on independent data before generalization.
- All computations should be conducted under the guidance of the protocol, standards, and professional judgment, and reviewed by appropriate personnel.

Chapter 17

AI-Automated Analysis

17.1 Overview

`ivdtools-analysis` is an AI-powered analysis Skill (current version 0.1.0) for in-vitro diagnostic (IVD) evaluation data. It uses the R package `ivdtools` as its statistical engine and, starting from CSV, TSV, Excel, or RDS data, performs data quality checks and statistical analysis, then generates reproducible Chinese HTML reports, R code, result tables, and figures. It works in conversational agent environments such as Claude Code (`/ivdtools-analysis`) and OpenAI Codex (`$ivdtools-analysis`).

Scope statement: This Skill is intended for statistical analysis and evidence compilation; it does not constitute clinical validation, product release, or regulatory approval. Decisions involving clinical, regulatory, or release matters should be independently reviewed by appropriate professionals.

The value of using the Skill in a conversation is not to “replace statistical judgment,” but to standardize the repetitive technical steps: data precheck, parameter confirmation, statistical computation, result verification, and Chinese report generation are executed through a fixed workflow, while data sources, parameters, exclusion records, warnings, result tables, figures, and the software environment are all saved together, reducing the risk of omissions and manual copy errors.

17.2 Applicable Analysis Tasks

The Skill covers 10 categories of analysis tasks, all powered by the corresponding modules of `ivdtools` as their statistical engine:

Analysis Category	Typical Tasks
Method comparison	Descriptive statistics, correlation, OLS/WLS/Deming/weighted Deming/Passing-Bablok, Bland-Altman, bias at medical decision levels
ROC analysis	Single/multi-marker ROC, AUC, cutoff values, sensitivity/specificity, multivariate Logistic
Qualitative agreement	2×2 tables, sensitivity/specificity/predictive values, overall agreement, Kappa, McNemar
Precision	Variance components, SD/CV, confidence intervals, profile results
Analytical sensitivity	sadler fit, limit of blank/limit of detection/limit of quantitation establishment
Reference intervals	Percentile, parametric, and robust methods with confidence intervals
Stability	Condition trend, test/reference comparison, shelf life, MKT, Arrhenius, stability design

Analysis Category	Typical Tasks
Quality control	Levey–Jennings plots, Westgard rule violation identification
Curve fitting	Curve search, nonlinear/weighted fitting, model comparison, residual diagnostics
Data checks	Outlier candidates, normality tests, bottle-to-bottle ANOVA, Youden plots
Sample size	Bland–Altman, proportion tests, proportion confidence intervals, stability design sample size

17.3 Standard Workflow

The Skill’s analysis follows a fixed process, with a fixed output directory structure, to facilitate verification and traceability.

17.3.1 1. Create a Dedicated Output Directory

The default directory is named `<input-file-name>-ivdtools-analysis-YYYYMMDD-HHMMSS`, located outside the Skill directory, and does not overwrite existing directories. Source data is always kept read-only and is never modified.

17.3.2 2. Data Precheck

Generate a data inspection JSON with `inspect_data.R`:

```
Rscript scripts/inspect_data.R \
  --input /data/example.csv \
  --encoding UTF-8 \
  --output /output/data-inspection.json
```

The precheck reports the number of rows, duplicate rows/column names, per-column type and proportion of missing values, number of unique values, candidate binary fields and candidate ID fields, category levels, and numeric ranges, helping to uncover data quality issues before analysis.

17.3.3 3. Determine the Analysis Plan

Select the analysis type based on the research objective and confirm parameters that cannot be safely inferred from the data. The Skill **will not fabricate** clinical cutoff values, allowable bias, stability limits, or acceptance criteria on its own; if such parameters affect the conclusions and cannot be safely derived from the data or the study plan, the user will be asked to confirm them.

17.3.4 4. Execute the Analysis and Render the Report

The Skill copies `assets/report-template.Rmd` into the output directory, creates an `analysis.R` with explicit parameters and `ivdtools::` namespaced calls to perform the analysis, then renders the Chinese HTML report. The rendering process preserves warnings and messages rather than silently suppressing them.

17.3.5 5. Verify the Results

Before delivery, verify that the total input row count matches the rows excluded at each step, that missing/non-numeric/duplicate records have been reported, that the positive class and direction are correct, that models converge, that the HTML values match `results/*.csv`, and that `sessionInfo()` and the actual package versions are recorded.

17.4 Output Contents

A standard delivery directory contains:

```
analysis.R           # Reusable analysis code
report.Rmd           # Report source file
report.html          # Chinese HTML report
analysis-manifest.json # Manifest of data sources, parameters, and artifacts
session-info.txt      # R environment and package versions
results/*.csv         # Machine-readable result tables
figures/*.png         # Analysis figures
```

The Chinese report includes, at minimum, the analysis objective, data source, data quality findings, methods and parameters, result tables, figures, exclusions and warnings, interpretation of results, limitations, and reproducibility information.

17.5 Installation and Usage

17.5.1 Runtime Environment

R 4.1.0, a matching version of `ivdtools`, Pandoc, and dependent packages such as `ggplot2`, `VCA`, `VFP`, `rmarkdown`, and `knitr`. Run the environment check first:

```
Rscript scripts/check_environment.R
```

An output of `STATUS: READY` indicates readiness; otherwise, missing items are listed. Installing/supplementing dependencies writes to the R library and **may only be executed with explicit user authorization** (`Rscript scripts/install_ivdtools.R --yes`).

17.5.2 Use in Claude Code

The Skill is registered as an invocable skill; call it directly in the conversation:

```
/ivdtools-analysis analyze the Results worksheet of /data/method_comparison.xlsx.
The sample ID is sample_id, the candidate method is candidate, and the reference method is reference.
Per the study plan, perform Deming regression (lambda=1) and difference-type Bland-Altman analysis
with a 95% confidence level, and output a reproducible Chinese HTML report.
```

17.5.3 Use in OpenAI Codex

Name `$ivdtools-analysis` in the conversation and attach the data and analysis requirements:

```
Please use $ivdtools-analysis to analyze /data/roc.csv.
sample_id is the sample ID, truth is the binary reference result where positive is the positive
marker1 and marker2 are the markers under evaluation. Please output AUC, cutoff values, sensitivity,
specificity, ROC plots, and a Chinese HTML report. The cutoff derived from the data is exploratory.
```

17.5.4 Recommended Details When Submitting a Task

1. Data file path (for Excel workbooks, also specify the worksheet);
2. Analysis objective and study design;
3. Mapping of fields such as sample ID, candidate/reference method, outcome, time, and lot number;
4. Positive class, direction, paired/repeated structure, and precision design formula;
5. Confidence level, power, statistical method, and plan-specified parameters;
6. Acceptance criteria such as medical decision levels, allowable bias, stability limits, and QC target mean/SD.

17.6 Data Handling Principles

- Preserve the original column names; do not automatically call `make.names()` to rename them;
- Do not default to imputing missing values, deleting outliers, averaging repeated measurements, or reversing the outcome direction;
- Outlier testing only produces candidate records for review and does not constitute a basis for automatic deletion;
- Keep a record of the rationale for excluded records and, where applicable, report sensitivity analyses with and without them;
- State the operations and rationale for any data transformation, exclusion, aggregation, and post-hoc analysis choices;
- Statistical significance does not equal clinical acceptability; acceptability conclusions can only be drawn against acceptance limits specified in the plan or provided by the user.

17.7 Frequently Asked Questions

- **Version mismatch:** When the environment check reports an `ivdtools` version mismatch, do not directly overwrite the newer installed version; choose a new writable R library, install the target version there after authorization, and verify it with `--library`.
- **Excel cannot be prechecked:** Confirm that `readxl` is installed; when there are multiple worksheets, use `--sheet` to select explicitly.
- **HTML cannot be rendered:** Confirm that `rmarkdown` and `knitr` are installed and that `rmarkdown::pandoc_available()` is `TRUE`; review the actual warnings or errors recorded in `report.Rmd` rather than silently ignoring them.
- **Binary direction is reversed:** Check the positive class, factor levels, marker direction, and test/reference mapping, and record the direction change and rationale before re-analyzing.
- **Outliers or repeated measurements present:** Do not automatically delete or average; first determine the handling approach based on the raw records, experimental design, and preset rules.

Chapter 18

References

18.1 Introduction

This document summarizes the correspondence between the international guidelines (the CLSI EP series) for each analysis module of `ivdtools` and the main methodological references. The correspondences are used to assist understanding and analysis-plan design; they **do not constitute a regulatory determination**; for registration, review, or accreditation matters, implement according to the applicable standard in force at the relevant time.

About domestic (Chinese) standards: Domestic standards fall into two categories — WS/T are health-industry standards (mainly aimed at clinical-laboratory practice), and YY are pharmaceutical/medical-industry standards (aimed at medical devices/in vitro diagnostic products; YY/T is recommended). For the same topic there may be both a laboratory-practice standard and a product-performance standard; when choosing, first clarify the applicable object (laboratory workflow or product evaluation).

18.2 Module-to-CLSI-Guideline Mapping

The correspondence between the modules of `ivdtools` and the guidelines of CLSI (Clinical and Laboratory Standards Institute) is as follows:

Module	Main entry	CLSI guideline	Main functions
Method comparison	<code>mcr()</code>	EP09	regression, correlation, Bland–Altman, bias, outliers
Precision	<code>precision()</code>	EP05	variance components, confidence intervals, Sadler profile
Qualitative analysis	<code>raw_to_table()</code> etc.	EP12	four-fold table, diagnostic metrics, Kappa, McNemar
Reference intervals	<code>reference_interval()</code>	EP28	percentile/parametric/robust methods
Stability	<code>stability_*</code> (), <code>arrhenius()</code> , <code>mkt()</code>	EP25	bias, regression, time-to-limit, MKT, Arrhenius
Linearity/curve fitting	<code>fit_equation()</code> series	EP06	linear/polynomial/4PL/5PL fitting
Analytical sensitivity/detection limit	<code>lob_lod_loq()</code>	EP17	LoB/LoQ, Sadler variance model

Module	Main entry	CLSI guideline	Main functions
Bottle/lot ANOVA	<code>bottle_anova()</code>	EP15	one-way/nested ANOVA, Tukey
ROC	<code>roc()</code> series	EP24 (reference)	AUC, cutoff, multivariate regression
Quality control	<code>qc_chart()</code> , <code>youden_plot()</code>	Westgard rules	Levey–Jennings, Westgard, Youden
Sample size	<code>sample_size_*</code>	General	agreement/proportion/one-arm test sample sizes
Outliers and normality	<code>outliers_test()</code> , <code>normal_test()</code>	General (pre-analysis steps)	Grubbs/ESD/Dixon/IQR, normality tests

Before actual application, be sure to verify the officially published text and, if necessary, consult the competent authority or a qualified assessment body.

18.3 Methodological References

The main statistical methods of `ivdtools` refer to the following publications (included in the package DESCRIPTION):

- Bland J M, Altman D G. Statistical methods for assessing agreement between two methods of clinical measurement. *Lancet*, 1986, 327(8476): 307–310. doi:10.1016/S0140-6736(86)90837-8
- Passing H, Bablok W. A new biometrical procedure for testing the equality of measurements from two different analytical methods. *Journal of Clinical Chemistry and Clinical Biochemistry*, 1983, 21(11): 709–720. doi:10.1515/cclm.1983.21.11.709
- Linnet K. Evaluation of regression procedures for methods comparison studies. *Clinical Chemistry*, 1993, 39(3): 424–432. doi:10.1093/clinchem/39.3.424
- Hanley J A, McNeil B J. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 1982, 143(1): 29–36. doi:10.1148/radiology.143.1.7063747
- Horn P S, Pesce A J, Copeland B E. A robust approach to reference interval estimation and evaluation. *Clinical Chemistry*, 1998, 44(3): 622–631. doi:10.1093/clinchem/44.3.622
- Westgard J O, Barry P L, Hunt M R, et al. A multi-rule Shewhart chart for quality control in clinical chemistry. *Clinical Chemistry*, 1981, 27(3): 493–501. doi:10.1093/clinchem/27.3.493
- Lu M J, Zhong W H, Liu Y X, et al. Sample size for assessing agreement between two methods of measurement by Bland–Altman method. *The International Journal of Biostatistics*, 2016, 12(2). doi:10.1515/ijb-2015-0039

18.4 Usage Suggestions

1. **Defer to the applicable standard:** Standards differ across countries/regions and applicable objects (laboratory vs product); first determine the applicable framework.
2. **Versions and supersession:** Standards are updated or superseded; cite the version and implementation date when referencing.
3. **Separate statistics from compliance:** This package provides statistical computation; whether all requirements of a given standard are met is determined by the complete validation/evaluation plan.
4. **Record and verify:** Write the standard list, versions, and DOI/number into the analysis plan for review and audit.